Spring Boot

The Basics

Build a REST API from zero: the Spring container and dependency injection, web controllers, validation and errors, JPA and databases, configuration, testing, security basics, one small project, and an interview kit.

Spring Core · Dependency Injection · REST · Validation · JPA · Config
Testing · Security intro · Actuator · Docker · Project · Interviews

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

Version basis

Spring Boot 3.5 line and Spring Boot 4 notes, on Java 21 or newer (Java 25 LTS recommended)
Spring and Maven code could not be run where this was written; listings follow the official docs
Build with Maven or Gradle; start a project at start.spring.io

How to use this guide

- **Who this is for.** You know core Java (classes, collections, streams, exceptions) and want to build backend services with Spring Boot. The guide keeps to the basics you need on day one and in a first interview.
- **Every chapter follows the same rhythm:** why it exists, how it works, code, what goes wrong, a key-points box, and a **Check yourself** quiz. Coloured boxes: NOTE, TIP, WATCH OUT and GOTCHA (mistakes that cost days), REAL WORLD, INTERVIEW ANGLE.
- **Running the code.** Create a project at start.spring.io (Maven, Java 21 or 25, dependencies Spring Web, Spring Data JPA, H2, Validation) and run it with `./mvnw spring-boot:run`. In a dark terminal box, lines beginning with `$` are commands and the other lines are output.
- **Honesty about the code.** The environment where this guide was written could not download Spring artefacts, so the listings were checked against the official Spring documentation and not executed. Output in terminal boxes is typical output. Run each example yourself and treat any difference as a lesson.
- **Versions.** Examples target Spring Boot 3.5 and note where Spring Boot 4 (Spring Framework 7, Jackson 3) differs.

Contents

Part 1: Spring Boot Foundations 4

1.1 What Spring and Spring Boot Are 4

1.2 The Spring Container and Dependency Injection 9

1.3 Bean Lifecycle and Proxies 13

1.4 Configuration 15

1.5 Auto-Configuration and Starters 18

Part 2: Building REST APIs 22

2.1 Spring MVC and REST Controllers 22

2.2 Validation and Exception Handling 28

2.3 REST API Design in Practice 33

Part 3: Data Access and Testing 38

3.1 Spring Data JPA basics 38

3.2 Relationships, transactions and the N+1 problem 43

3.3 Testing Spring Boot applications 48

Part 4: Securing and Shipping 54

4.1 Spring Security basics 54

4.2 Observability and shipping 60

Part 5: Project: Library API 67

5.1 Plan and scaffold 67

5.2 Domain, repository and service 71

5.3 REST layer with validation and error handling 80

5.4 Security and tests 84

5.5 Run it and call it 89

Part 6: Interview Preparation and Study Plan 92

6.1 Interview Q&A; 92

6.2 Cheat Sheets 98

6.3 A Ten-Day Study Plan 101

6.4 Mixed Quiz and Next Steps 102

PART 1

Spring Boot Foundations

Wiring, configuration and auto-configuration: the machinery under every Spring Boot application.

In this part

1.1 What Spring and Spring Boot Are
1.2 The Spring Container and Dependency Injection
1.3 Bean Lifecycle and Proxies
1.4 Configuration
1.5 Auto-Configuration and Starters

1.1 What Spring and Spring Boot Are

Every non-trivial Java program is a graph of objects that need each other: a controller needs a service, the service needs a repository, the repository needs a data source. In plain Java you build that graph by hand with `new`, and the code that creates objects gets mixed with the code that uses them. Tests become hard (you cannot swap the real database for a fake), and every change to a constructor ripples through the program.

Spring Framework solves this with **Inversion of Control (IoC)**: you describe your classes and what they need, and a container creates the objects and hands each one its collaborators. Passing collaborators in from outside is called **dependency injection (DI)**. The objects the container manages are called **beans**. Around that core, Spring offers web (MVC), data access, transactions, security, testing and more.

Spring Boot is an opinionated layer on top of Spring Framework. Plain Spring needed pages of XML or Java configuration before the first request could be served. Boot removes that boilerplate with four ideas:

- **Starters**: single dependencies such as `spring-boot-starter-web` that pull in a tested, compatible set of libraries.
- **Auto-configuration**: Boot looks at what is on your classpath and what you defined, and configures sensible defaults (a web server, a JSON mapper, a connection pool) for you.
- **Embedded server**: Tomcat runs inside your application, so you ship one jar and start it with `java -jar`.
- **Production features**: Actuator gives health checks and metrics; externalised configuration lets one jar run in many environments.

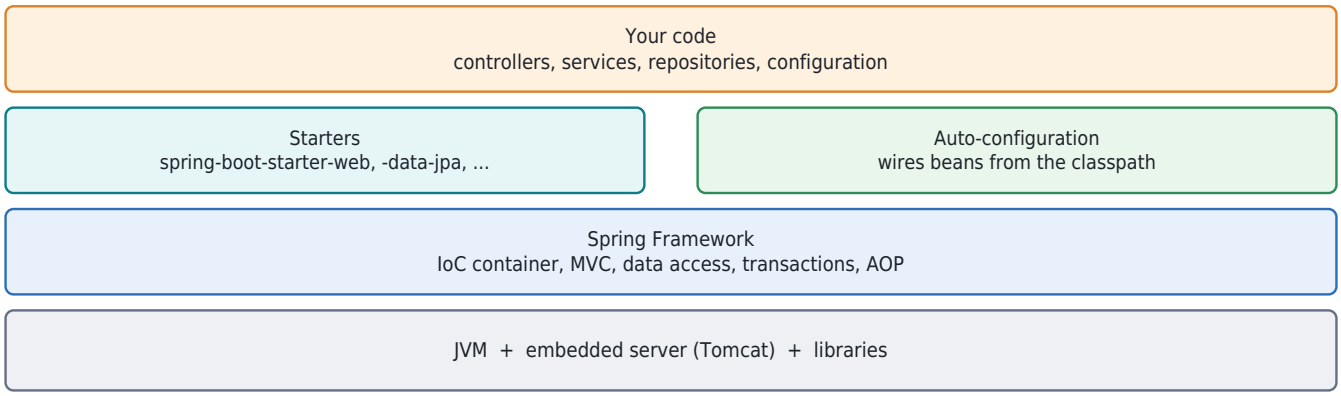


Figure 1.1. Spring Boot sits on Spring Framework: starters bring libraries, auto-configuration wires them, and your code is only the top layer.

1.1.1 Version map

Versions move, so check before you start a project. At the time of writing (October 2026) the current Spring Boot release is 4.1.x, built on Spring Framework 7.0.x, and it requires Java 17 or later (it supports up to Java 26). The previous major line, Spring Boot 3.5.x (Spring Framework 6.2), is still widely used in industry and in existing code bases.

Boot line	Spring Framework	Minimum Java	Notes
`3.5.x`	6.2	17	Very common in existing projects. Jackson 2 (`com.fasterxml.jackson`).
`4.x`	7.0	17	Current. Modularised starters, Jackson 3 (`tools.jackson`) by default.

Boot lines you will meet.

This guide writes code that works on both lines. Where they differ you will see a short note. Always confirm exact numbers at spring.io/projects/spring-boot and in the "System Requirements" page of the reference documentation; they change with each release.

> **NOTE**
> Spring Boot 4 and Jackson 3 Jackson 3 moved its packages from `com.fasterxml.jackson` to `tools.jackson` (the annotations module keeps the old package). You only notice this if you import Jackson classes directly, for example a custom serializer. Everything in this guide uses Boot's auto-configured JSON support and never imports Jackson types.

1.1.2 Creating a project

The standard way is **start.spring.io** (Spring Initializr), also built into IntelliJ IDEA and VS Code. Choose Maven, Java, the latest stable Boot version, a group (`com.example`) and artifact (`library`), packaging Jar, Java 21 (or 17 or 25), and add the dependencies **Spring Web** and **Spring Boot Actuator**. Download the zip and open it. If you cannot reach the site, you can write the files below by hand; they are all it takes.

The pom.xml

pom.xml XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version> <!-- use the latest 4.x or 3.5.x -->
    <relativePath/>
  </parent>

  <groupId>com.example</groupId>
  <artifactId>library</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <properties>
    <java.version>21</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

What to notice:

- The **parent** `spring-boot-starter-parent` fixes the versions of hundreds of libraries (Spring, Jackson, Hibernate, Tomcat, JUnit...). That is why the dependencies below have no `<version>`: the parent's dependency management supplies a combination that has been tested together.
- A **starter** is just a dependency whose purpose is to drag in the right transitive dependencies. `spring-boot-starter-web` brings Spring MVC, an embedded Tomcat and Jackson.
- The **Spring Boot Maven plugin** builds the executable "fat" jar and provides `spring-boot:run`.
- Under Boot 4 some starters were split into smaller modules (for example, the web starter also exists in a REST-focused form); the plain starters used in this guide still work. If a starter name from an older tutorial fails to resolve, look at the migration guide on the Spring Boot wiki.

Project layout

```
library/
├── mvnw, mvnw.cmd              Maven wrapper scripts (no Maven install needed)
├── pom.xml
└── src/
    ├── main/
    │   ├── java/com/example/library/
    │   │   └── LibraryApplication.java     main class, in the ROOT package
    │   └── resources/
    │       ├── application.properties      configuration
    │       ├── static/                     static files served as-is
    │       └── templates/                  server-side templates (not used in an API)
    └── test/java/com/example/library/
        └── LibraryApplicationTests.java
```

Put the main class in the root package of your code (`com.example.library`) and every other class in sub-packages (`...library.book`, `...library.loan`). The reason is component scanning, explained in the next chapter.

The first application

LibraryApplication.java JAVA

```java
package com.example.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
public class LibraryApplication {

    public static void main(String[] args) {
        SpringApplication.run(LibraryApplication.class, args);
    }

    @GetMapping("/hello")
    public String hello() {
        return "Hello, Spring Boot";
    }
}
```

This listing needs the Spring dependencies, so it was not run while writing this guide. Run it on your machine:

Run from source, then call the endpoint (typical output) TERMINAL

```
$ ./mvnw spring-boot:run
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v4.1.1)

... Tomcat started on port 8080 (http) with context path '/'
... Started LibraryApplication in 1.9 seconds (process running for 2.3)
$ curl http://localhost:8080/hello
Hello, Spring Boot
```

The three lines of real code do a lot: `@SpringBootApplication` turns on auto-configuration and scanning, `SpringApplication.run` creates the container and starts Tomcat, and `@RestController` plus `@GetMapping` register a web endpoint. The `./mvnw` script is the **Maven wrapper**: it downloads the Maven version the project expects, so nobody needs to install Maven.

Building and running the executable jar

Package and run (typical output) TERMINAL

```
$ ./mvnw clean package
$ ls target/*.jar
target/library-0.0.1-SNAPSHOT.jar
$ java -jar target/library-0.0.1-SNAPSHOT.jar --server.port=9090
```

`package` produces an **executable jar** (often called a fat jar): your classes plus every dependency, including Tomcat, laid out so that Boot's own launcher can start it. One file, one command, no application server to install. Command-line arguments such as `--server.port=9090` override configuration, a topic covered in the configuration chapter.

INTERVIEW ANGLE

What is the difference between Spring and Spring Boot? Spring Framework is the core: the IoC container, DI, AOP, MVC, data and transaction support. Spring Boot is a layer that makes Spring quick to use: starters for dependency management, auto-configuration that configures beans from the classpath and properties, an embedded server for self-contained jars, and production features such as Actuator. Boot does not replace Spring; every Boot application is a Spring application.

GOTCHA

Mixing Boot versions Do not override the version of a single Spring or Jackson library next to the Boot parent. Mixed versions cause errors such as `NoSuchMethodError` at startup. Change the Boot version and let dependency management follow.

Check yourself

1. Name the four ideas that make Boot "opinionated Spring".
2. Why do the dependencies in the pom above carry no `<version>`?
3. Where must the class with `@SpringBootApplication` live, and why?
4. What is inside an executable jar that is not in a normal jar?
5. You run `java -jar app.jar --server.port=9090`. Which port does Tomcat use if `application.properties` says 8081?

Answers

1. Starters, auto-configuration, an embedded server, and production features (Actuator, externalised configuration).
2. The `spring-boot-starter-parent` parent supplies dependency management, a tested set of versions.
3. In the root package of the application, because component scanning starts at that package and covers its sub-packages.
4. All the dependency jars (nested) plus Boot's launcher classes, so `java -jar` works with no extra classpath.
5. 9090: command-line arguments have higher precedence than `application.properties`.

KEY POINTS

- Spring is an IoC container plus libraries; Boot adds starters, auto-configuration, an embedded server and Actuator.
- The Boot parent POM manages library versions; you add starters without versions.
- Put the `@SpringBootApplication` class in the root package.
- `./mvnw spring-boot:run` runs from source; `./mvnw package` plus `java -jar` runs the executable jar.
- Check the current Boot version and Java requirement at spring.io before starting.

1.2 The Spring Container and Dependency Injection

1.2.1 The problem and the idea

Consider a service that needs a repository. Without DI the service builds its own:

JAVA

```java
class BookService {
    private final BookRepository repository = new JdbcBookRepository(); // hard-wired
}
```

`BookService` now knows which implementation to use and how to construct it. You cannot test it with a fake repository, and if `JdbcBookRepository` later needs a `DataSource`, every caller changes. With DI the class only declares what it needs:

JAVA

```java
class BookService {
    private final BookRepository repository;

    BookService(BookRepository repository) { // somebody else decides which one
        this.repository = repository;
    }
}
```

The mental picture: a **container** holds a registry of beans. At startup it reads your class definitions, works out the dependency graph, creates each bean in the right order, and passes dependencies into constructors. Your code never calls `new` on a managed class.

1.2.2 A 40-line container, run on plain Java

To see that there is no magic, here is a toy container. It takes a class, inspects the constructor with reflection, recursively creates the parameter types, and caches each instance (so every bean is a singleton). Real Spring does far more, but this is the core of constructor injection.

MiniDi.java JAVA

```java
import java.lang.reflect.Constructor;
import java.util.HashMap;
import java.util.Map;

public class MiniDi {

    static class MiniContainer {
        private final Map<Class<?>, Object> singletons = new HashMap<>();

        @SuppressWarnings("unchecked")
        <T> T get(Class<T> type) {
            Object existing = singletons.get(type);
            if (existing != null) {
                return (T) existing;
            }
            try {
                Constructor<?> ctor = type.getDeclaredConstructors()[0];
                Class<?>[] paramTypes = ctor.getParameterTypes();
                Object[] args = new Object[paramTypes.length];
                for (int i = 0; i < paramTypes.length; i++) {
                    args[i] = get(paramTypes[i]);
                }
                Object created = ctor.newInstance(args);
                singletons.put(type, created);
                return (T) created;
            } catch (ReflectiveOperationException e) {
                throw new IllegalStateException("Cannot create " + type.getSimpleName(), e);
            }
        }
    }

    static class BookRepository {
        String findTitle(int id) { return "Book#" + id; }
    }
```

MiniDi.java (continued) JAVA

```java
    static class BookService {
        private final BookRepository repository;
        BookService(BookRepository repository) { this.repository = repository; }
        String describe(int id) { return "Service says: " + repository.findTitle(id); }
    }

    static class BookController {
        private final BookService service;
        BookController(BookService service) { this.service = service; }
        String handle(int id) { return service.describe(id); }
    }

    public static void main(String[] args) {
        MiniContainer container = new MiniContainer();
        BookController controller = container.get(BookController.class);
        System.out.println(controller.handle(7));
        System.out.println(container.get(BookService.class) == container.get(BookService.class));
    }
}
```

Run on JDK 21 TERMINAL

```
$ java MiniDi.java
Service says: Book#7
true
```

Asking for `BookController` made the container build `BookRepository`, then `BookService`, then the controller, and the second line proves the service is a single shared instance. What this toy lacks: interfaces with several implementations, scopes, configuration, cycles detection, lifecycle callbacks and proxies. Spring has all of them.

1.2.3 ApplicationContext and beans

The Spring container is the `ApplicationContext` interface (the older, smaller one is `BeanFactory`). `SpringApplication.run` returns it. A **bean** is any object created and managed by it. Spring needs to know which classes are beans; there are two ways to tell it.

Stereotype annotations (your own classes)

Put an annotation on the class and component scanning finds it.

Annotation	Meaning
`@Component`	A generic managed bean.
`@Service`	Business logic. Same behaviour as `@Component`, but documents intent.
`@Repository`	Data access. Also enables translation of persistence exceptions to Spring's `DataAccessException` hierarchy.
`@Controller`	A web controller returning views. `@RestController` = `@Controller` + `@ResponseBody`.
`@Configuration`	A class that declares beans with `@Bean` methods.

Stereotypes. All are specialisations of `@Component`.

@Configuration and @Bean (classes you do not own)

You cannot add `@Component` to a library class such as `Clock`. Instead, write a factory method:

AppConfig.java JAVA

```java
import java.time.Clock;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {

    @Bean
    public Clock clock() {
        return Clock.systemUTC();
    }
}
```

The method name becomes the bean name, the return type is the bean type, and parameters of a `@Bean` method are injected, so one bean can depend on another. Injecting a `Clock` into services (instead of calling `Instant.now()`) also makes time testable.

1.2.4 Constructor injection

Spring supports three injection styles. Prefer the first.

JAVA

```java
@Service
public class LoanService {

    private final BookRepository books;      // final: cannot be forgotten or replaced
    private final Clock clock;

    // single constructor: @Autowired is optional since Spring 4.3
    public LoanService(BookRepository books, Clock clock) {
        this.books = books;
        this.clock = clock;
    }
}
```

Field injection (`@Autowired private BookRepository books;`) looks shorter, but the fields cannot be `final`, you cannot construct the class in a unit test without reflection or a Spring context, and the list of dependencies hides inside the class so a class with eight of them does not look wrong. Constructor injection makes dependencies explicit and mandatory, and a service with too many constructor parameters shows you the design problem. Setter injection is for genuinely optional dependencies.

INTERVIEW ANGLE

Why is constructor injection preferred over field injection? Dependencies can be final and immutable; the object is never half-initialised; it is testable with plain `new` and mocks, no container needed; missing dependencies fail at startup; and a large constructor is a visible hint that a class does too much. Field injection also hides circular dependencies that constructor injection reports immediately.

1.2.5 More than one candidate: @Primary and @Qualifier

If two beans match a requested type, Spring throws `NoUniqueBeanDefinitionException`. You resolve the ambiguity in one of two ways.

JAVA

```java
public interface Notifier { void send(String message); }

@Component
@Primary                        // the default choice when nobody asks for a specific one
public class EmailNotifier implements Notifier {
    public void send(String message) { /* send e-mail */ }
}

@Component("sms")               // explicit bean name
public class SmsNotifier implements Notifier {
    public void send(String message) { /* send SMS */ }
}

@Service
```

(continued) JAVA

```java
public class OverdueService {
    private final Notifier notifier;
    public OverdueService(@Qualifier("sms") Notifier notifier) { // pick by name
        this.notifier = notifier;
    }
}
```

Without `@Qualifier`, `OverdueService` would get `EmailNotifier` because of `@Primary`. You can also inject `List<Notifier>` to receive all implementations, or `Map<String, Notifier>` keyed by bean name.

1.2.6 Component scanning rules

`@SpringBootApplication` includes `@ComponentScan` for the package of the annotated class and everything below it. A `@Service` in `com.example.library.book` is found; one in `com.example.other` is silently ignored and you get a `NoSuchBeanDefinitionException` for a missing bean at startup. Fix by moving the class (best) or by widening the scan with `@SpringBootApplication(scanBasePackages = "...")`. Classes without a stereotype annotation (or a `@Bean` method) are not beans.

1.2.7 Scopes

Scope	Behaviour
`singleton`	The default. One shared instance per container.
`prototype`	A new instance every time the bean is requested from the container.
`request`	One instance per HTTP request (web applications only).
`session`	One instance per HTTP session (web applications only).

Bean scopes.

Choose a non-singleton scope with `@Scope("prototype")`. Most beans stay singletons: services, repositories and controllers are stateless collaborators, which is what makes sharing safe.

GOTCHA
Singletons are shared between threads Tomcat handles many requests at once, all calling the same controller and service instances. A mutable field in a singleton is shared state: `private int counter;` incremented in a service method loses updates under load, and `private SimpleDateFormat fmt;` breaks randomly. Keep singletons stateless (state lives in method variables or in the database), or use thread-safe types such as `AtomicInteger` and `ConcurrentHashMap`. Injecting a prototype bean into a singleton also gives you one instance only (created once at injection), which surprises people; inject an `ObjectProvider<T>` to get a fresh one each time.

Check yourself

1. What does the code below do on startup, assuming both classes are scanned? `@Component class A { A(B b) {} }` and `@Component class B { B(A a) {} }`
2. Two beans implement `Notifier` and neither is marked `@Primary`. What happens and how do you fix it?
3. Your `@Service` class is in a package that is a sibling of the main class's package. Is it a bean?
4. A singleton service has `private List<String> cache = new ArrayList<>();` that every request adds to. What is wrong?
5. When do you write a `@Bean` method instead of using `@Component`?

Answers

1. Startup fails with `BeanCurrentlyInCreationException` (a circular dependency through constructors). Break the cycle by redesigning, for example extracting a third class.
2. `NoUniqueBeanDefinitionException`. Mark one `@Primary`, or inject with `@Qualifier("name")`, or inject a `List<Notifier>`.
3. No: scanning covers only the main class's package and below. Move it or set `scanBasePackages`.
4. `ArrayList` is not thread-safe and the state is shared by all requests, so there are data races. Make the service stateless or use a thread-safe structure or external store.
5. For classes you do not control (library types) or when construction needs custom logic or parameters.

KEY POINTS

- The container creates beans and injects dependencies; you declare, it wires.
- Use stereotypes for your classes, `@Bean` methods for third-party types.
- Prefer constructor injection: final fields, testable, explicit.
- `@Primary` and `@Qualifier` resolve several candidates; scanning starts at the main class's package.
- Beans are singletons by default, so keep them stateless or thread-safe.

1.3 Bean Lifecycle and Proxies

1.3.1 From class to usable bean

For each singleton bean the container goes through the same steps. Knowing them explains most "why is this null" and "why did my annotation do nothing" bugs.

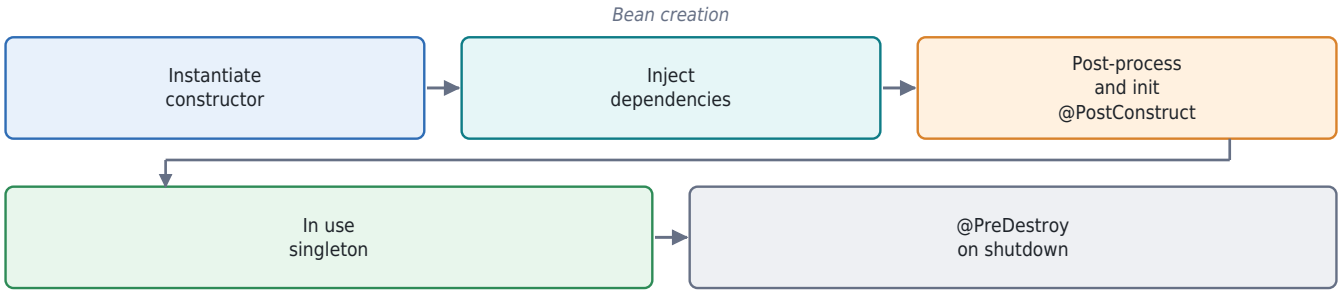


Figure 1.2. The bean lifecycle: instantiate, inject dependencies, post-processors and init callbacks, use, then destroy callbacks on shutdown.

1. **Instantiate** the object (calls the constructor; constructor dependencies are resolved first).
2. **Populate** properties (setter and field injection, if used).
3. **Aware callbacks** (for example `BeanNameAware`), rarely needed.
4. `BeanPostProcessor.postProcessBeforeInitialization`.
5. **Initialisation**: the method annotated `@PostConstruct`, then `InitializingBean.afterPropertiesSet`, then a custom `initMethod`.
6. `BeanPostProcessor.postProcessAfterInitialization`: this is where proxies are created.
7. The bean is ready and lives until shutdown.
8. **Destruction**: `@PreDestroy`, then `DisposableBean.destroy`, when the context closes.

CacheWarmer.java JAVA

```java
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.stereotype.Component;

@Component
public class CacheWarmer {

    private final BookRepository books;

    public CacheWarmer(BookRepository books) {
        this.books = books;
        // do NOT use fields of injected beans for heavy work here; keep constructors light
    }

    @PostConstruct              // runs once, after the dependencies are injected
    void warm() {
        System.out.println("Warming cache with " + books.count() + " books");
    }

    @PreDestroy                 // runs once on graceful shutdown
    void close() {
        System.out.println("Releasing resources");
    }
}
```

Note the `jakarta.annotation` package. Since Spring Boot 3 everything moved from `javax.*` to `jakarta.*`; copy-pasting an old `javax.annotation.PostConstruct` import is a classic error.

Prototype beans are only created and initialised by the container; it does not call their destroy callbacks.

1.3.2 BeanPostProcessor: how Spring adds behaviour

A `BeanPostProcessor` is a bean that sees every other bean after creation and may modify or **replace** it. Many Spring features are post-processors: `@Autowired` handling, `@PostConstruct` handling, and the ones that create proxies for `@Transactional`, `@Async`, `@Cacheable` and method security. You will rarely write one, but you should know that these annotations work through the container, not through the language.

1.3.3 Proxies: what @Transactional really does

Annotations such as `@Transactional` are only markers. The behaviour is added by wrapping your bean in a **proxy**: an object with the same type that receives each call, does something before (open a transaction), calls your real method, then does something after (commit or roll back).

```
caller ──► [ proxy ]  begin tx ──► [ real LoanService ] ──► commit / rollback
           (what the container actually injects)
```

When another bean receives a `LoanService`, it is really holding the proxy. Spring creates proxies either with the JDK's `java.lang.reflect.Proxy` (when the bean implements interfaces) or with CGLIB, which generates a subclass; Spring Boot uses CGLIB by default (`spring.aop.proxy-target-class=true`). Consequences of the subclass approach: the class and its advised methods cannot be `final`, and the advised method must be `public` (or at least visible to the proxy) to be intercepted.

The self-invocation gotcha

JAVA

```java
@Service
public class LoanService {

    public void borrowAll(List<Long> bookIds) {
        for (Long id : bookIds) {
            borrow(id);          // this.borrow(...): a direct call, the proxy is bypassed
        }
    }

    @Transactional
    public void borrow(Long bookId) {
        // ... never runs inside a transaction when called from borrowAll
    }
}
```

The proxy only intercepts calls that come from outside through it. A call from one method to another inside the same object uses `this`, which is the real object, so `@Transactional`, `@Async`, `@Cacheable` and `@PreAuthorize` on `borrow` are silently skipped. No exception is thrown; the bug shows up as missing rollbacks. Fixes, in order of preference: move the annotated method to another bean and inject it; put the annotation on the outer method; or (last resort) inject the bean into itself.

WATCH OUT

Annotations that need a proxy fail silently `@Transactional` on a private method, on a `final` method or called through `this` does nothing and reports nothing. If a transaction or async behaviour "does not work", check the call path first, annotation spelling second.

INTERVIEW ANGLE

Explain the Spring bean lifecycle and how @Transactional works. The container instantiates the bean, injects dependencies, runs BeanPostProcessors and init callbacks (`@PostConstruct`), the bean serves requests, and `@PreDestroy` runs at shutdown. `@Transactional` works through AOP: a post-processor wraps the bean in a proxy that opens a transaction before the method and commits or rolls back after it. Because the proxy only sees external calls, self-invocation bypasses the transaction.

Check yourself

1. In which order run the constructor, `@PostConstruct` and field injection?
2. Why does `@Transactional` on a method called by another method of the same class not start a transaction?
3. What goes wrong with this: `@Transactional public final void save()`?
4. Which package is `PostConstruct` in on Spring Boot 3 and 4?
5. A prototype-scoped bean has a `@PreDestroy` method. Will it run at shutdown?

Answers

1. Constructor, then field/setter injection, then `@PostConstruct`.
2. The call goes through `this`, bypassing the proxy that opens the transaction.
3. A CGLIB proxy cannot override a final method, so the advice does not apply (no transaction).
4. `jakarta.annotation`.
5. No. The container does not manage the destruction of prototype beans.

KEY POINTS

- Lifecycle: instantiate, inject, post-processors, `@PostConstruct`, use, `@PreDestroy`.
- `@Transactional`, `@Async`, `@Cacheable` work through proxies created by post-processors.
- Self-invocation, `private` and `final` methods bypass the proxy silently.
- Use `jakarta.*` imports in Boot 3 and 4.

1.4 Configuration

Code that is identical in every environment still needs different settings in each: a port, a database URL, a feature flag, a password. Spring Boot's **externalised configuration** lets you keep those values outside the compiled jar so one build runs everywhere.

1.4.1 application.properties and application.yml

Boot reads `src/main/resources/application.properties` (or `application.yml`) from the classpath. The two formats are equivalent; YAML is more readable for nested keys but is whitespace-sensitive.

application.properties PROPERTIES

```
server.port=8080
spring.application.name=library
library.loan-days=14
library.max-loans-per-member=5
logging.level.com.example.library=DEBUG
```

application.yml YAML

```
server:
  port: 8080
spring:
  application:
    name: library
library:
  loan-days: 14
  max-loans-per-member: 5
logging:
  level:
    com.example.library: DEBUG
```

Use one format per project. Boot has hundreds of built-in keys; the "Common Application Properties" appendix of the reference documentation lists them.

1.4.2 Reading values: @Value and @ConfigurationProperties

@Value for one-off values

```java
@Service
public class LoanService {
    private final int loanDays;

    public LoanService(@Value("${library.loan-days:14}") int loanDays) { // 14 = default
        this.loanDays = loanDays;
    }
}
```

`@Value` is quick, but strings scattered across classes are easy to mistype, are not validated together, and give no IDE help.

@ConfigurationProperties for groups (preferred)

Bind a prefix to a typed object. A Java record works well because it is immutable and Boot binds it through the canonical constructor.

LibraryProperties.java

```java
import jakarta.validation.constraints.Min;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.context.properties.bind.DefaultValue;
import org.springframework.validation.annotation.Validated;

@ConfigurationProperties(prefix = "library")
@Validated
public record LibraryProperties(
        @DefaultValue("14") @Min(1) int loanDays,
        @DefaultValue("5") @Min(1) int maxLoansPerMember) {
}
```

LibraryApplication.java

```java
@SpringBootApplication
@ConfigurationPropertiesScan          // finds all @ConfigurationProperties classes
public class LibraryApplication {
    public static void main(String[] args) {
        SpringApplication.run(LibraryApplication.class, args);
    }
}
```

Then inject `LibraryProperties` into any bean like any other dependency. What to notice:

- **Relaxed binding**: `library.loan-days` (kebab case, recommended in files), `library.loanDays` and the environment variable `LIBRARY_LOANDAYS` all bind to `loanDays`.
- With `@Validated` and a Bean Validation implementation on the classpath (`spring-boot-starter-validation`), an invalid value such as `library.loan-days=0` stops the application at startup with a clear message. Failing fast beats failing in production.
- Instead of `@ConfigurationPropertiesScan` you can annotate a `@Configuration` class with `@EnableConfigurationProperties(LibraryProperties.class)`.

	@Value	@ConfigurationProperties
Best for	One or two simple values	A group of related settings
Type safety	Strings with placeholders	A typed class or record
Validation	None built in	Bean Validation with `@Validated`
Relaxed names	Limited	Full

@Value versus @ConfigurationProperties.

1.4.3 Profiles

A **profile** is a named set of beans and properties that is active only in some environments. Per-profile files are `application-dev.properties`, `application-prod.properties`; they are loaded on top of the base file when the profile is active.

application-dev.properties — PROPERTIES

```
spring.datasource.url=jdbc:h2:mem:library
logging.level.root=INFO
logging.level.com.example.library=DEBUG
```

Activate profiles with `spring.profiles.active=dev` in a file, with the environment variable `SPRING_PROFILES_ACTIVE=prod`, or on the command line: `java -jar app.jar --spring.profiles.active=prod`. Beans can be restricted too:

JAVA

```java
@Configuration
@Profile("dev")
public class DevDataConfig {
    // beans that exist only when the dev profile is active, e.g. a data loader
}
```

TIP

Keep prod-only settings out of the base file Put safe defaults for local development in `application.properties`, and override in `application-prod.properties` or, better, with environment variables supplied by the platform. Never rely on a profile name to protect something dangerous such as `ddl-auto=create`.

1.4.4 Environment variables and the command line

Any property can be given from outside. Environment variables use upper-case with underscores (the dot becomes _, a dash is dropped or turned into _): `SERVER_PORT=9090`, `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/library`. This is the normal way containers receive configuration.

Override configuration from outside (typical) — TERMINAL

```
$ SERVER_PORT=9090 java -jar target/library.jar
$ java -jar target/library.jar --server.port=9091 --library.loan-days=21
```

1.4.5 Property precedence

When the same key appears in several places, the higher source wins. Condensed from the reference documentation, from highest to lowest priority:

1. Test properties (`@SpringBootTest(properties=...)`, `@TestPropertySource`).
2. Command-line arguments (`--server.port=9091`).
3. `SPRING_APPLICATION_JSON` and Java system properties (`-Dserver.port=9092`).
4. OS environment variables.
5. Profile-specific files outside the jar, then inside the jar (`application-prod.properties`).
6. Plain `application.properties` outside the jar, then inside the jar.
7. `@PropertySource` annotations and application defaults (`SpringApplication.setDefaultProperties`).

Check the exact and complete ordering in "Externalized Configuration" in the reference before relying on a corner case such as a config import. The rule of thumb is enough for work: the closer to the running process, the stronger. An external file placed next to the jar (or `./config/application.properties`) overrides the one packaged inside.

1.4.6 Secrets

Never commit passwords, API keys or JWT signing keys to `application.properties` in Git. Provide them at runtime as environment variables, container secrets, or from a secrets manager (Vault, a cloud provider's service, Kubernetes Secrets), and reference them with a placeholder: `spring.datasource.password=${DB_PASSWORD}`. Remember that Actuator's `env` endpoint can expose configuration; keys with words such as `password`, `secret`, `key` and `token` are masked by default, but do not depend on that as your only protection.

INTERVIEW ANGLE

How do you manage configuration across environments in Spring Boot? Keep one artefact. Base defaults live in `application.properties`; environment-specific values come from profile files, environment variables or command-line arguments, in that rising order of priority. Group settings with `@ConfigurationProperties` (validated) instead of scattered `@Value`s, and keep secrets out of the repository, supplying them at runtime.

Check yourself

1. `application.properties` has `server.port=8080` and you start with `SERVER_PORT=9090 java -jar app.jar --server.port=9091`. Which port wins?
2. Why might `library.loan-days=0` crash the app at startup in the example above?
3. What is wrong here? `@Value("${library.loan-days}") private int days;` where the key is absent from all sources.
4. How do you activate the `prod` profile in a Docker container without changing the image?
5. Name one reason to prefer a record with `@ConfigurationProperties` over several `@Value` fields.

Answers

1. 9091: command-line arguments beat environment variables.
2. `@Min(1)` on the record with `@Validated` fails binding validation, so the context fails fast.
3. No default is given, so startup fails with an unresolved placeholder error. Use `${library.loan-days:14}` or a properties record with `@DefaultValue`.
4. Set the environment variable `SPRING_PROFILES_ACTIVE=prod` on the container.
5. Type safety, grouping, validation at startup and immutability (one place to see all settings).

KEY POINTS

- Configuration lives outside the code: files, environment variables, command line.
- Prefer `@ConfigurationProperties` records (validated) to loose `@Value`s.
- Profiles select per-environment files and beans; activate with `SPRING_PROFILES_ACTIVE`.
- Precedence: tests, command line, system properties, environment, profile files, base files.
- Secrets come from the runtime environment, never from Git.

1.5 Auto-Configuration and Starters

Auto-configuration is the feature that makes Boot feel like magic: add a dependency and a working set of beans appears. It is not magic, it is ordinary Spring `@Configuration` classes that are switched on by conditions. Understanding it lets you debug "why is this bean here" and "why is this bean missing".

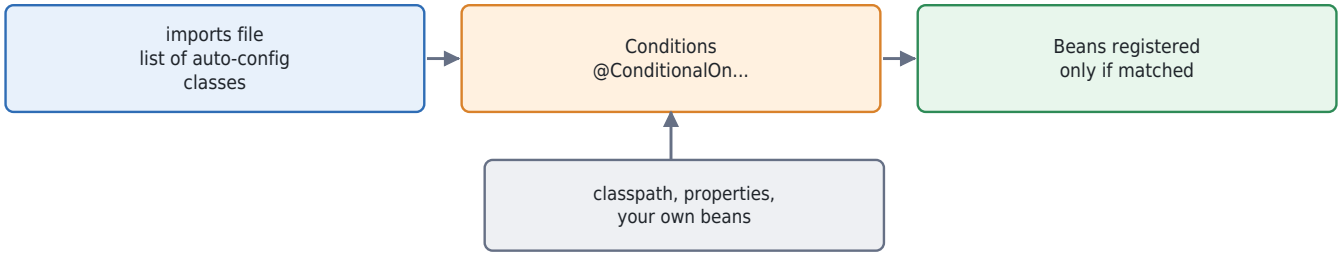


Figure 1.3. Boot reads the list of auto-configuration classes, evaluates each class's conditions against the classpath and your beans, and registers only those that match.

1.5.1 What @SpringBootApplication really is

```java
@SpringBootConfiguration      // marks this class as a @Configuration (the root config)
@EnableAutoConfiguration      // imports the candidate auto-configuration classes
@ComponentScan                // scans this package and below for your own beans
public @interface SpringBootApplication { /* simplified */ }
```

So `@SpringBootApplication` is three annotations in one. `@EnableAutoConfiguration` loads the list of auto-configuration classes from every jar on the classpath. Each jar that provides some lists them in a text file:

META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports

```
org.springframework.boot.autoconfigure.web.servlet.WebMvcAutoConfiguration
org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
com.example.hello.HelloAutoConfiguration
```

(One fully qualified class name per line. The older `spring.factories` mechanism was replaced by this file in Boot 2.7/3.0. In Boot 4 the framework's own auto-configurations were split into per-technology modules, so some built-in class names and packages differ from older tutorials, but the file name and the mechanism are the same.)

1.5.2 Conditions

An auto-configuration class is guarded by conditional annotations. The most important:

Annotation	The configuration applies only if...
`@ConditionalOnClass`	a class is on the classpath (the library was added)
`@ConditionalOnMissingClass`	a class is absent
`@ConditionalOnBean`	a bean of that type already exists
`@ConditionalOnMissingBean`	no such bean exists: the application's own definition always wins
`@ConditionalOnProperty`	a property has a given value (feature flags)
`@ConditionalOnWebApplication`	the app is a web application

Common conditions.

`@ConditionalOnMissingBean` is the key to Boot's "back off" behaviour. Boot defines a default `ObjectMapper`, `DataSource` or `RestClient.Builder` only if you have not declared your own. To customise something, you therefore usually declare a bean of the same type and Boot steps aside.

1.5.3 Seeing what happened: the conditions report

Run with `--debug` (or `debug=true` in properties). Boot prints the **conditions evaluation report**.

Print the report (typical output, shortened) TERMINAL

```
$ java -jar target/library.jar --debug
============================
CONDITIONS EVALUATION REPORT
============================

Positive matches:
-----------------
   WebMvcAutoConfiguration matched:
      - @ConditionalOnClass found required classes 'jakarta.servlet.Servlet', ...

Negative matches:
-----------------
   DataSourceAutoConfiguration:
      Did not match:
         - @ConditionalOnClass did not find required class 'javax.sql.DataSource' ...
```

Positive matches say what was configured and why, negative matches say what was not and which condition failed. When you ask "why is there no `DataSource`?" the answer is in the negative section, typically a missing dependency. With Actuator on the classpath, the `/actuator/conditions` endpoint shows the same information.

1.5.4 Excluding auto-configuration

Sometimes you want to turn one off: for example, a project that has a JDBC driver on the classpath but does not want Boot to create a `DataSource` yet.

JAVA

```java
@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)
public class LibraryApplication { /* ... */ }
```

or in properties:
`spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration`
(use the class name that matches your Boot version; check the conditions report for the exact name).

1.5.5 Starters: what they are and are not

A starter contains no logic. It is a dependency descriptor (a pom) that pulls in a library together with its Boot auto-configuration and compatible companions. The rule: **starter = dependencies, auto-configuration = behaviour.** Naming: official starters are `spring-boot-starter-xyz`; third-party ones should be `xyz-spring-boot-starter`.

Starter	You get
`spring-boot-starter-web`	Spring MVC, embedded Tomcat, Jackson
`spring-boot-starter-data-jpa`	Spring Data JPA, Hibernate, a connection pool (HikariCP)
`spring-boot-starter-validation`	Bean Validation (Hibernate Validator)
`spring-boot-starter-security`	Spring Security with default protection
`spring-boot-starter-actuator`	Health, metrics, info endpoints
`spring-boot-starter-test`	JUnit 5, AssertJ, Mockito, Spring Test

Starters used in this guide.

1.5.6 Writing a tiny auto-configuration

The mechanism is easiest to remember after building it once. Imagine a shared library that offers a `Greeter` bean to any application that adds it. (Spring dependencies required; not run here.)

HelloProperties.java JAVA

```java
import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "hello")
public record HelloProperties(String greeting) {
    public HelloProperties {
        if (greeting == null) greeting = "Hello";
    }
}
```

HelloAutoConfiguration.java JAVA

```java
import org.springframework.boot.autoconfigure.AutoConfiguration;
import org.springframework.boot.autoconfigure.condition.ConditionalOnMissingBean;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;

@AutoConfiguration
@EnableConfigurationProperties(HelloProperties.class)
@ConditionalOnProperty(prefix = "hello", name = "enabled", matchIfMissing = true)
public class HelloAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean            // the app can define its own Greeter and win
    public Greeter greeter(HelloProperties props) {
        return new Greeter(props.greeting());
    }
}
```

Greeter.java JAVA

```java
public class Greeter {
    private final String greeting;

    public Greeter(String greeting) { this.greeting = greeting; }

    public String greet(String name) { return greeting + ", " + name; }
}
```

Register it by adding one line (`com.example.hello.HelloAutoConfiguration`) to `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` in the library's `src/main/resources`. Any application with that jar gets a `Greeter`; setting `hello.greeting=Hi` changes it, `hello.enabled=false` removes it, and declaring its own `Greeter` bean replaces it. Note that auto-configuration classes must not be found by the application's component scan; that is why they are listed in the imports file and live in a separate package.

NOTE
Why you rarely write one Inside a single application you just use `@Configuration` and `@ConditionalOnProperty` directly. Writing a real auto-configuration is for shared libraries and starters; the mechanism matters mostly so you can read Boot's own and debug it.

INTERVIEW ANGLE
How does Spring Boot auto-configuration work? `@EnableAutoConfiguration` (part of `@SpringBootApplication`) reads the list of candidate configuration classes from `META-INF/spring/...AutoConfiguration.imports` in every jar on the classpath. Each class is a normal `@Configuration` guarded by conditions such as `@ConditionalOnClass` (the library is present), `@ConditionalOnMissingBean` (you have not defined your own) and `@ConditionalOnProperty`. Only the classes whose conditions pass are applied. You inspect the result with `--debug` and customise by defining your own bean, by properties or by `exclude`.

GOTCHA
Auto-configuration and your own bean definitions If you define a bean and your version is ignored, check the type and name carefully: `@ConditionalOnMissingBean` backs off only for a bean of the matching type. If an auto-configured bean you did not expect appears, find the dependency that pulled it in with `./mvnw dependency:tree`.

Check yourself

1. Name the three annotations that make up `@SpringBootApplication`.
2. Where does Boot find the list of auto-configuration classes?
3. You define your own `ObjectMapper` bean. Why does Boot's default one not appear?
4. How do you find out why a `DataSource` bean was not created?
5. What is the difference between a starter and an auto-configuration?

Answers

1. `@SpringBootConfiguration`, `@EnableAutoConfiguration`, `@ComponentScan`.
2. In `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` files inside the jars on the classpath.
3. Boot's definition is guarded by `@ConditionalOnMissingBean`, so it backs off.
4. Start with `--debug` and read the negative matches (or `/actuator/conditions`); the usual cause is a missing dependency or property.
5. A starter is a dependency bundle with no logic; auto-configuration is the conditional configuration code that creates the beans.

KEY POINTS
- `@SpringBootApplication` = configuration + auto-configuration + component scan.
- Auto-configuration classes are listed in `AutoConfiguration.imports` and guarded by `@Conditional...` annotations.
- `@ConditionalOnMissingBean` lets your own beans override Boot's defaults.
- `--debug` prints the conditions report; use `exclude` to switch a class off.
- Starters bundle dependencies; they add no behaviour themselves.

PART 2

Building REST APIs

Controllers, JSON, validation, error handling and the design habits that make an HTTP API pleasant to use.

In this part

2.1 Spring MVC and REST Controllers
2.2 Validation and Exception Handling
2.3 REST API Design in Practice

2.1 Spring MVC and REST Controllers

Almost every Spring Boot job involves writing HTTP endpoints. A client (a browser, a phone app, another service) sends a request such as `GET /api/books/7`, and your code must turn that into a response, usually JSON. Doing this with raw sockets or even the bare Servlet API means parsing URLs, headers and bodies by hand. Spring MVC does that plumbing for you so a controller method can look like an ordinary Java method.

This chapter covers how a request travels through Spring MVC, how to map URLs to methods, how JSON conversion works, and how to structure the code in layers. Examples use a small book catalogue; the project in a later Part builds on the same shapes.

> **NOTE**
> Versions. Spring Boot 4.x (built on Spring Framework 7) is the current line; Boot 3.5.x is the last 3.x line. All code in this Part uses `jakarta.*` imports, constructor injection and records, and works on both unless a NOTE says otherwise. Boot 4 requires Java 17 or newer; Boot 3.5 also needs Java 17+. None of the listings in this Part were executed in the build environment (it cannot download Spring artefacts), so treat them as carefully checked examples, and any output shown as "typical output".

2.1.1 How a request flows through Spring MVC

The starter `spring-boot-starter-web` (named `spring-boot-starter-webmvc` in Boot 4; the old name still works as a compatibility alias) puts Spring MVC and an embedded Tomcat on the classpath. Boot starts Tomcat on port 8080 when the application starts, so there is no WAR file and no separate server to install.

Tomcat accepts the connection and hands the request to one servlet that Spring registers: the `DispatcherServlet`. It is the **front controller**: every request enters through it, and it delegates to specialised helpers.

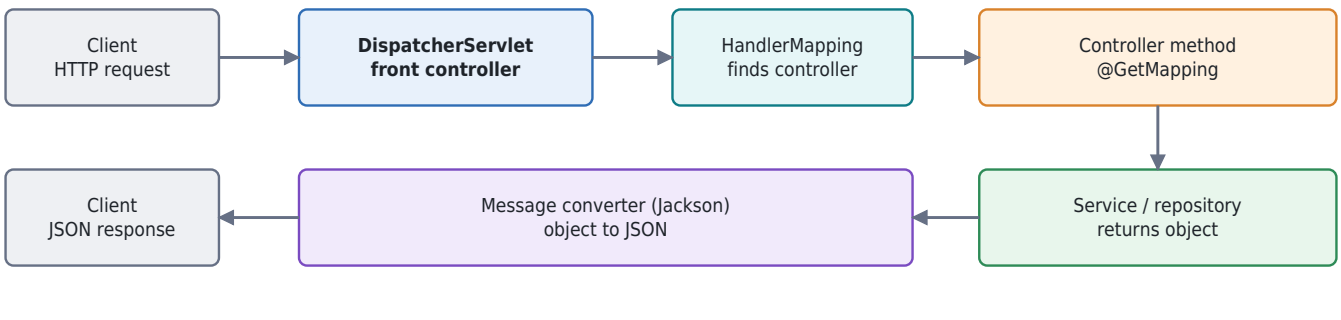


Figure 2.1. The DispatcherServlet looks up a handler, runs interceptors, converts arguments and the return value with message converters, and writes the response.

The steps, in order:

1. The `HandlerMapping` finds the controller method that matches the URL, the HTTP method and other conditions (such as `Content-Type`).
2. Filters (servlet level) have already run, and interceptors (`HandlerInterceptor`) run before the method is called.

3. A `HandlerAdapter` prepares the arguments. `@PathVariable`, `@RequestParam` and `@RequestBody` are resolved here; JSON is turned into Java objects by an `HttpMessageConverter` (Jackson).
4. Your method runs and returns an object.
5. The return value is converted back to JSON by a message converter and written to the response with the status code.
6. If any step throws, exception resolvers (your `@ExceptionHandler` methods, covered in the next chapter) turn the exception into a response.

INTERVIEW ANGLE
Describe what happens when a request reaches a Spring Boot application. Tomcat receives it and passes it through the servlet filter chain to the `DispatcherServlet`. It asks the handler mappings for the matching controller method, runs interceptors, resolves method arguments (converting the JSON body with Jackson), invokes the method, converts the return value to the response body with a message converter, and sends it. Exceptions are routed to exception handlers.

2.1.2 Your first controller

Annotate a class with `@RestController`. It combines `@Controller` and `@ResponseBody`, which means "every return value is written to the response body" instead of being treated as a view name.

BookController.java JAVA

```java
package com.example.library.book;

import java.util.List;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookService service;

    public BookController(BookService service) {
        this.service = service;
    }

    @GetMapping
    public List<BookResponse> list(@RequestParam(defaultValue = "") String title) {
        return service.search(title);
    }

    @GetMapping("/{id}")
    public BookResponse get(@PathVariable long id) {
        return service.get(id);
    }
}
```

What to notice: `@RequestMapping("/api/books")` on the class gives a common prefix; `@GetMapping` with no path means `GET /api/books`. The class has one constructor, so Spring injects `BookService` without any annotation. The controller contains no business logic, only translation between HTTP and Java.

Try it with curl (typical output):

Call the endpoints TERMINAL

```
$ curl -s "localhost:8080/api/books?title=java"
[{"id":1,"title":"Core Java","author":"A. Rao","year":2023}]
$ curl -s localhost:8080/api/books/1
{"id":1,"title":"Core Java","author":"A. Rao","year":2023}
```

2.1.3 Mapping annotations and parameters

Each HTTP verb has a shortcut annotation. They are all specialised forms of `@RequestMapping(method = ...)`.

Annotation	Reads from	Example
`@GetMapping`	`HTTP GET`	`@GetMapping("/{id}")`
`@PostMapping`	`HTTP POST`	`@PostMapping` creates a resource
`@PutMapping`	`HTTP PUT`	replace a whole resource
`@PatchMapping`	`HTTP PATCH`	partial update
`@DeleteMapping`	`HTTP DELETE`	remove a resource
`@PathVariable`	`URL segment`	`/books/{id}` gives `long id`
`@RequestParam`	`query string`	`?page=2` gives `int page`
`@RequestBody`	`request body`	JSON turned into a record
`@RequestHeader`	`a header`	`@RequestHeader("X-Trace-Id")`

Mapping and binding annotations.

Details worth knowing:

- Names are taken from the parameter name when the code is compiled with `-parameters`. Spring Boot's Maven parent enables that flag, so `@PathVariable long id` works. If you ever compile without it, write `@PathVariable("id")`.
- `@RequestParam` is required by default. A missing one gives HTTP 400. Use `required = false`, `defaultValue = "..."`, or `Optional<T>`.
- A path variable of the wrong type (`/books/abc` for a `long`) gives 400 automatically.
- Narrow a mapping with `consumes` and `produces`, for example `@PostMapping(consumes = "application/json")`. A wrong `Content-Type` then gives 415 (Unsupported Media Type).

2.1.4 DTOs, records and JSON

Jackson is the library that converts Java objects to JSON and back. Boot configures it for you. A **DTO** (data transfer object) is a class whose only job is to carry data across a boundary. Java records are ideal DTOs: immutable, short, and Jackson supports them.

BookRequest.java JAVA

```java
package com.example.library.book;

public record BookRequest(String title, String author, int year) { }
```

BookResponse.java JAVA

```java
package com.example.library.book;

public record BookResponse(long id, String title, String author, int year) {

    static BookResponse from(Book book) {
        return new BookResponse(book.getId(), book.getTitle(), book.getAuthor(), book.getYear());
    }
}
```

Why not return the database entity directly? Three reasons you can give in an interview:

1. **Leaking fields.** An entity may hold a password hash, an internal flag or a collection you did not mean to expose. A DTO exposes only what you list.
2. **Coupling.** If the JSON is your entity, renaming a column breaks every client. A DTO lets the API and the database evolve separately.
3. **Lazy loading.** Serialising an entity with lazy relations can trigger database queries during JSON writing, or throw `LazyInitializationException` (covered in the JPA Part).

Useful Jackson behaviour to know:

- Unknown JSON properties are ignored by default in Spring Boot (`FAIL_ON_UNKNOWN_PROPERTIES` is turned off), so adding a field on the client does not break the server.

- `LocalDate` and friends are written as ISO strings (`2026-10-12`), not numbers, because Boot disables `WRITE_DATES_AS_TIMESTAMPS`.
- Rename a field with `@JsonProperty("published_year")`, hide one with `@JsonIgnore`. Both annotations still live in the `com.fasterxml.jackson.annotation` package in Jackson 3.
- To change global behaviour use properties such as `spring.jackson.default-property-inclusion=non_null`. In Boot 4 the same namespace applies for Jackson 3; check the property reference when you upgrade.

> **NOTE**
> Jackson 3. Spring Boot 4 uses Jackson 3 by default. Its core packages moved from `com.fasterxml.jackson.databind` to `tools.jackson.databind`, and the main class `ObjectMapper` is created through `JsonMapper`. If you only write DTOs and controllers, as here, you notice nothing. You only touch the difference when you inject an `ObjectMapper` or write custom serialisers. Boot 4 also ships a Jackson 2 compatibility module for gradual migration.

2.1.5 Responses, status codes and ResponseEntity

Returning an object gives status 200. For anything else you control the status in one of two ways.

BookWriteController.java JAVA

```java
package com.example.library.book;

import java.net.URI;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

@RestController
@RequestMapping("/api/books")
public class BookWriteController {

    private final BookService service;

    public BookWriteController(BookService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<BookResponse> create(@RequestBody BookRequest request) {
        BookResponse created = service.create(request);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").buildAndExpand(created.id()).toUri();
        return ResponseEntity.created(location).body(created);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable long id) {
        service.delete(id);
    }
}
```

`ResponseEntity<T>` lets you set the status, headers and body together; here it returns 201 Created with a `Location` header pointing at the new resource, which is the REST convention. `@ResponseStatus` is the simpler choice when the status is always the same.

Create a book (typical output) TERMINAL

```
$ curl -i -X POST localhost:8080/api/books \
    -H "Content-Type: application/json" \
    -d '{"title":"Spring Basics","author":"M. Rao","year":2026}'
HTTP/1.1 201
Location: http://localhost:8080/api/books/2
Content-Type: application/json

{"id":2,"title":"Spring Basics","author":"M. Rao","year":2026}
```

GOTCHA

Returning `null` from a controller method gives an empty 200 response, not a 404. Return `ResponseEntity.notFound().build()` or, better, throw a custom exception that your error handler maps to 404 (next chapter).

2.1.6 Layering: controller, service, repository

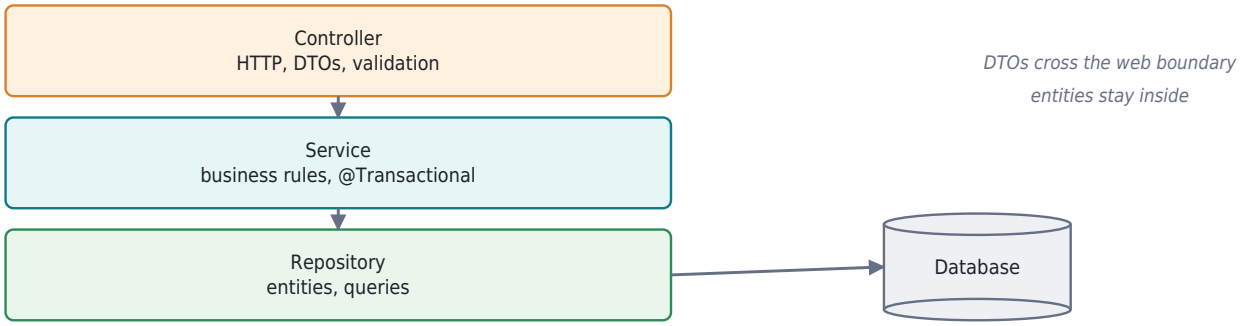


Figure 2.2. Each layer talks only to the one below it; DTOs cross the web boundary and entities stay inside the data layer.

A layered structure keeps each class small and testable.

- **Controller**: HTTP only. Parse input, call one service method, choose the status code. No business rules, no SQL.
- **Service**: business rules and transactions. Works with entities and throws domain exceptions (`BookNotFoundException`). Knows nothing about HTTP.
- **Repository**: reads and writes the database (Spring Data, next Part).

Each layer receives the next one through its constructor. The service in our example:

BookService.java JAVA

```java
package com.example.library.book;

import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class BookService {

    private final BookRepository repository;

    public BookService(BookRepository repository) {
        this.repository = repository;
    }

    @Transactional(readOnly = true)
    public List<BookResponse> search(String title) {
        return repository.findByTitleContainingIgnoreCase(title).stream()
                .map(BookResponse::from)
                .toList();
    }

    @Transactional(readOnly = true)
    public BookResponse get(long id) {
        return repository.findById(id)
                .map(BookResponse::from)
                .orElseThrow(() -> new BookNotFoundException(id));
    }
```

BookService.java (continued) JAVA

```
    @Transactional
    public BookResponse create(BookRequest request) {
        Book book = new Book(request.title(), request.author(), request.year());
        return BookResponse.from(repository.save(book));
    }

    @Transactional
    public void delete(long id) {
        if (!repository.existsById(id)) {
            throw new BookNotFoundException(id);
        }
        repository.deleteById(id);
    }
}
```

The mapping from entity to DTO happens inside the transactional service, so lazy fields can still be read. The controller never sees an entity.

TIP
Do not create an interface plus one implementation for every service by reflex. Spring does not need it (it can proxy classes). Add an interface when you truly have two implementations or need a seam for a test.

2.1.7 Common mistakes

- **Field injection** (`@Autowired` on a field) in controllers makes them hard to test. Use the constructor.
- **Fat controllers** that call repositories directly: business rules end up scattered and untestable.
- **Returning entities** from endpoints, as discussed.
- **Blocking forever**: a controller that calls a slow service without a timeout holds a Tomcat thread. Timeouts are covered in the API design chapter.
- **Wrong package for component scan**: `@SpringBootApplication` scans its own package and below. A controller placed in a sibling package is silently ignored, and you get 404 for every URL.

Check yourself

1. Which component receives every HTTP request first inside Spring MVC, and what is the pattern called?
2. What does `@RestController` add compared with `@Controller`?
3. What is wrong here, and what does the client see for `GET /api/books/abc`? `@GetMapping("/{id}") BookResponse get(@PathVariable long id)`
4. A controller method has return type `BookResponse` and returns `null`. What status code is sent?
5. Give two reasons not to return a JPA entity from a controller.
6. Where should the check "a member may not borrow more than 5 books" live?

Answers

1. The `DispatcherServlet`; it is the front controller pattern.
2. `@ResponseBody` on every method, so return values are serialised into the body instead of resolved as view names.
3. Nothing is wrong with the code; the path value `abc` cannot convert to `long`, so Spring answers 400 Bad Request.
4. 200 with an empty body. Throw an exception or use `ResponseEntity` to send 404.
5. It can leak internal fields and it couples the API to the schema; lazy relations may also fail or query during serialisation.
6. In the service layer, because it is a business rule independent of HTTP.

KEY POINTS

Remember these points.

- A request enters the `DispatcherServlet`, is routed to a controller method, and message converters (Jackson) translate JSON both ways.
- Use `@RestController`, the verb-specific mapping annotations, and bind inputs with `@PathVariable`, `@RequestParam`, `@RequestBody`.
- Use records as request and response DTOs; keep entities inside the service and repository layers.
- Set status deliberately: 201 with a `Location` header for creates, 204 for deletes, 404 through an exception.
- Controller handles HTTP, service handles rules and transactions, repository handles storage; wire them with constructor injection.

2.2 Validation and Exception Handling

Never trust input. A client can send an empty title, a negative year, or a 5 MB string. If your service code assumes good data, bad data ends as a `NullPointerException` deep in the stack and a confusing 500 response. The rule: **reject bad input at the edge with a clear 4xx message, and turn every failure into a predictable JSON error**.

2.2.1 Bean Validation

Bean Validation (Jakarta Validation, with Hibernate Validator as the implementation) lets you declare constraints as annotations. Add the starter `spring-boot-starter-validation` (same name in Boot 3.5 and 4).

BookRequest.java — JAVA

```java
package com.example.library.book;

import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record BookRequest(
        @NotBlank(message = "title is required")
        @Size(max = 200, message = "title must be at most 200 characters")
        String title,

        @NotBlank(message = "author is required")
        String author,

        @Min(value = 1450, message = "year must be 1450 or later")
        @Max(value = 2100, message = "year must be 2100 or earlier")
        int year) { }
```

Common constraints:

Annotation	Meaning
`@NotNull`	value must not be null
`@NotBlank`	string is not null and has a non-whitespace character
`@NotEmpty`	string, collection or map is not empty
`@Size(min, max)`	length of a string or size of a collection
`@Min / @Max`	numeric bounds
`@Positive / @PositiveOrZero`	greater than zero / zero or more
`@Email`	looks like an email address
`@Pattern(regexp)`	string matches a regular expression
`@Past / @Future`	date is before / after now
`@Valid`	validate the nested object too (cascade)

Frequently used constraints.

Constraints do nothing until you ask for validation. Put `@Valid` on the `@RequestBody` parameter:

CreateEndpoint.java JAVA

```java
@PostMapping
public ResponseEntity<BookResponse> create(@Valid @RequestBody BookRequest request) {
    BookResponse created = service.create(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(created);
}
```

If validation fails, Spring throws `MethodArgumentNotValidException` before your method body runs. Without a handler, Boot answers 400 with a default error body. Next we customise it.

Validating other inputs

For `@RequestParam` and `@PathVariable`, put `@Validated` on the controller class and constraints on the parameters (`@RequestParam @Min(1) int page`). Spring 6.1 and later (so both Boot 3.5 and 4) validate controller method parameters natively and raise `HandlerMethodValidationException` in that case, a different exception from the body one. Handle both. Validation also works on any Spring bean when the class is annotated `@Validated`, which is handy for service methods; that path throws `ConstraintViolationException`.

A custom constraint

When no built-in annotation fits, write your own: an annotation plus a validator.

Isbn.java JAVA

```java
package com.example.library.validation;

import jakarta.validation.Constraint;
import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;
import jakarta.validation.Payload;
import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Documented
@Constraint(validatedBy = Isbn.Validator.class)
@Target({ElementType.FIELD, ElementType.PARAMETER, ElementType.RECORD_COMPONENT})
@Retention(RetentionPolicy.RUNTIME)
public @interface Isbn {

    String message() default "must be a valid ISBN-13";

    Class<?>[] groups() default {};

    Class<? extends Payload>[] payload() default {};

    class Validator implements ConstraintValidator<Isbn, String> {
        @Override
        public boolean isValid(String value, ConstraintValidatorContext context) {
            if (value == null) {
                return true; // combine with @NotNull when the field is mandatory
            }
            String digits = value.replace("-", "");
            if (!digits.matches("\\d{13}")) {
                return false;
            }
            int sum = 0;
            for (int i = 0; i < 13; i++) {
                int d = digits.charAt(i) - '0';
                sum += (i % 2 == 0) ? d : 3 * d;
            }
            return sum % 10 == 0;
        }
    }
}
```

What to notice: the three attributes `message`, `groups`, `payload` are required by the specification. Treat `null` as valid and let `@NotNull` handle presence, so each rule has one job. Use it like any other: `@Isbn String isbn` in a record.

Validation groups let one DTO have different rules for create and update (`@NotNull(groups = OnCreate.class)`, `@Validated(OnCreate.class)`). They add complexity, so most teams prefer two small DTOs (`CreateBookRequest`, `UpdateBookRequest`) instead.

2.2.2 Handling exceptions in one place

Without a plan, every controller grows `try/catch` blocks. Spring lets you declare, once, how each exception type becomes a response.

- `@ExceptionHandler` on a method of a controller handles exceptions from that controller only.
- `@RestControllerAdvice` on a separate class applies to all controllers. This is the normal choice.

ProblemDetail (RFC 9457)

Every API invents its own error JSON unless you agree on a format. **RFC 9457** ("Problem Details for HTTP APIs", the successor of RFC 7807) defines one, content type `application/problem+json`, with standard fields: `type`, `title`, `status`, `detail`, `instance`, plus any extra properties you add. Spring has the class `ProblemDetail` built in (Framework 6+). Setting `spring.mvc.problemdetails.enabled=true` makes Spring's own MVC exceptions (such as a missing parameter or wrong media type) use that format automatically. In Boot 3.5 the property defaults to false, so set it explicitly; check the reference for your version if you rely on the default.

The advice class

ApiExceptionHandler.java JAVA

```java
package com.example.library.web;

import com.example.library.book.BookNotFoundException;
import com.example.library.book.DuplicateIsbnException;
import java.net.URI;
import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ProblemDetail;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

@RestControllerAdvice
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(ApiExceptionHandler.class);

    @ExceptionHandler(BookNotFoundException.class)
    ProblemDetail notFound(BookNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Book not found");
        pd.setType(URI.create("https://api.example.com/problems/book-not-found"));
        return pd;
    }

    @ExceptionHandler(DuplicateIsbnException.class)
    ProblemDetail conflict(DuplicateIsbnException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, ex.getMessage());
        pd.setTitle("Duplicate ISBN");
        return pd;
    }

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex, HttpHeaders headers,
            HttpStatusCode status, WebRequest request) {
        List<FieldProblem> errors = ex.getBindingResult().getFieldErrors().stream()
                .map(f -> new FieldProblem(f.getField(), f.getDefaultMessage()))
                .toList();
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(
                HttpStatus.UNPROCESSABLE_ENTITY, "Validation failed");
        pd.setTitle("Invalid request");
        pd.setProperty("errors", errors);
        return ResponseEntity.unprocessableEntity().body(pd);
    }

    @ExceptionHandler(Exception.class)
    ProblemDetail unexpected(Exception ex) {
        log.error("Unhandled exception", ex);
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(
                HttpStatus.INTERNAL_SERVER_ERROR, "Something went wrong. Try again later.");
        pd.setTitle("Internal error");
        return pd;
    }

    record FieldProblem(String field, String message) { }
}
```

Supporting types:

BookNotFoundException.java JAVA

```java
package com.example.library.book;

public class BookNotFoundException extends RuntimeException {

    public BookNotFoundException(long id) {
        super("No book with id " + id);
    }
}
```

What to notice:

- Extending `ResponseEntityExceptionHandler` gives sensible handling for all Spring MVC's own exceptions (400, 404, 405, 415 and so on) rendered as `ProblemDetail`. We override only the validation one. Your own `@ExceptionHandler` methods are for your domain exceptions.
- The catch-all `Exception` handler logs the full stack trace on the server and returns a generic message. **Never put `ex.getMessage()` or a stack trace of an unexpected exception in the response**: it leaks class names, SQL and internals to attackers. Our domain exceptions return their message because we wrote those messages for clients.
- The 422 versus 400 choice for validation is a style decision. 400 (Bad Request) is what Spring uses by default; 422 (Unprocessable Content) says "the syntax was fine but the content breaks the rules". Pick one and be consistent.

Typical output for an invalid POST TERMINAL

```
$ curl -s -X POST localhost:8080/api/books -H "Content-Type: application/json" \
    -d '{"title":"","author":"X","year":1200}'
{"type":"about:blank","title":"Invalid request","status":422,"detail":"Validation failed",
"instance":"/api/books","errors":[{"field":"title","message":"title is required"},
{"field":"year","message":"year must be 1450 or later"}]}
```

2.2.3 Mapping exceptions to status codes

Situation	Status	Typical cause
Malformed JSON, wrong type	`400`	client sent bad syntax
Validation constraint violated	`400 or 422`	field rules broken
Not authenticated	`401`	missing or bad credentials
Authenticated but not allowed	`403`	role missing
Resource does not exist	`404`	custom exception thrown in service
Business conflict (duplicate, version clash)	`409`	unique ISBN already exists
Method not supported	`405`	`DELETE` on a read-only URL
Unexpected failure	`500`	bug; log it, hide details

Which exception becomes which status.

You can also annotate an exception class with `@ResponseStatus(HttpStatus.NOT_FOUND)`. It is quick for tiny apps, but it ties the domain class to HTTP and produces the default body, so the advice class above scales better.

INTERVIEW ANGLE
How do you handle exceptions globally in Spring Boot? Create a class annotated `@RestControllerAdvice` with `@ExceptionHandler` methods for each exception type. Each returns a `ProblemDetail` (RFC 9457) or `ResponseEntity` with the right status. Domain exceptions map to 404/409, validation errors to 400/422 with per-field details, and a catch-all handler logs the stack trace and returns a generic 500 without internals.

WATCH OUT
A `catch (Exception e) { e.printStackTrace(); }` in a controller swallows the failure and still returns 200. Let exceptions propagate to the advice class.

Check yourself

1. You added `@NotBlank` on a record component but blank titles are accepted. What did you most likely forget?
2. Which exception does `@Valid @RequestBody` raise on failure?
3. What is wrong with this handler? `@ExceptionHandler(Exception.class) String h(Exception e) { return e.toString(); }`
4. Which media type does a `ProblemDetail` use?
5. A user registers an email that already exists. Which status code fits best, and why?
6. What is the difference between `@NotNull`, `@NotEmpty` and `@NotBlank` for a string?

Answers

1. `@Valid` on the controller parameter, or the validation starter on the classpath.
2. `MethodArgumentNotValidException`.
3. It leaks internal details (`toString()` of any exception) and is returned as plain text with status 200 unless a status is set; log the error and return a generic `ProblemDetail`.
4. `application/problem+json`.
5. 409 Conflict, since the request is valid but clashes with current server state.
6. `@NotNull` forbids null only; `@NotEmpty` also forbids `""`; `@NotBlank` also forbids whitespace-only strings.

KEY POINTS

Remember these points.

- Declare constraints on DTOs, add `@Valid` at the controller, and include the validation starter.
- Centralise error mapping in one `@RestControllerAdvice` and answer in the `ProblemDetail` format (RFC 9457).
- Domain exceptions map to 404 or 409; validation maps to 400 or 422 with per-field messages.
- Never expose stack traces or raw exception messages for unexpected errors; log them server-side.

2.3 REST API Design in Practice

A working endpoint is not yet a good API. Clients depend on your URLs, status codes and error shapes for years, so small design decisions made now are expensive to change. This chapter collects the conventions that experienced teams follow.

2.3.1 Resources, verbs and idempotency

REST models your domain as **resources** (nouns) and uses HTTP methods as the verbs.

- Use plural nouns: `/api/books`, `/api/books/7`, `/api/books/7/loans`. Not `/getBook` or `/createLoan`.
- Nesting shows ownership but keep it to two levels. `/authors/3/books` is fine; `/authors/3/books/7/loans/9/items` is not.
- Use lowercase and hyphens in paths (`/book-copies`), and camelCase for JSON properties.
- Actions that are not CRUD can be a sub-resource or a verb endpoint when nothing fits: `POST /api/loans/9/return`.

An operation is **idempotent** if calling it many times has the same effect as calling it once. A **safe** operation also changes nothing. This matters because networks retry: clients, proxies and load balancers resend requests after timeouts.

Method	Safe	Idempotent	Typical use
GET	yes	yes	read one or a list
POST	no	no	create; run an action
PUT	no	yes	replace a whole resource
PATCH	no	not guaranteed	change some fields
DELETE	no	yes	remove (second call may return 404, state is the same)

HTTP methods and their guarantees.

A retried `POST /loans` could create two loans. For payments and similar cases, accept an `Idempotency-Key` header and remember processed keys. Mention this in interviews; it shows you think about retries.

2.3.2 Status code cheat table

Code	Meaning	Use when
200	OK	successful read or update with a body
201	Created	POST created a resource; send `Location`
204	No Content	successful delete or update with no body
400	Bad Request	malformed JSON, bad parameter type
401	Unauthorized	not authenticated (really "unauthenticated")
403	Forbidden	authenticated but not permitted
404	Not Found	unknown id or URL
409	Conflict	duplicate or version conflict
422	Unprocessable Content	valid syntax, rules violated
429	Too Many Requests	rate limit hit
500	Internal Server Error	unexpected bug
503	Service Unavailable	dependency down, overloaded

Status codes you will use most.

2.3.3 Pagination and sorting

Never return an unbounded list. A table with a million rows will exhaust memory and time out. Use query parameters: `GET /api/books?page=0&size=20&sort=year,desc`. Spring Data turns these into a `Pageable` argument automatically (details in the data access Part).

PagedController.java JAVA

```java
@GetMapping
public PageResponse<BookResponse> list(
        @PageableDefault(size = 20, sort = "title") Pageable pageable) {
    return service.page(pageable);
}

public record PageResponse<T>(List<T> items, int page, int size, long totalItems, int totalPages) {

    public static <T> PageResponse<T> of(Page<T> page) {
        return new PageResponse<>(page.getContent(), page.getNumber(), page.getSize(),
                page.getTotalElements(), page.getTotalPages());
    }
}
```

Returning your own `PageResponse` instead of Spring's `Page` keeps the JSON shape stable (Spring Data warns that serialising `PageImpl` directly is not a supported stable format). Cap `size` so a client cannot ask for 100000 rows: the property `spring.data.web.pageable.max-page-size` (default 2000) does it. Offset pagination slows on deep pages; for huge feeds, **keyset** pagination (`?after=lastId`) scales better.

2.3.4 Versioning

When you must break clients, version the API. The common options:

- **URI path**: `/api/v1/books`. Simplest, visible, cache friendly. Most popular.
- **Header**: `X-API-Version: 2` or a vendor media type `Accept: application/vnd.library.v2+json`. Cleaner URLs, harder to test with a browser.
- **Query parameter**: `?version=2`. Easy, but easy to forget.

Spring Framework 7 (Boot 4) adds first-class support: configure a resolver in `WebMvcConfigurer.configureApiVersioning(...)` and write `@GetMapping(path = "/{id}", version = "1.1+")`. On Boot 3.5 use path prefixes. Whatever you choose, avoid breaking changes in the first place: adding optional fields is safe, removing or renaming is not.

HATEOAS (hypermedia links in responses, from the `spring-boot-starter-hateoas` starter) completes the REST theory, but few real APIs use it. Know the term for interviews.

2.3.5 CORS

CORS (cross-origin resource sharing) is a browser rule: JavaScript on `https://app.example.com` may not call `https://api.example.com` unless the API says it allows that origin. It does not protect your API from curl; it only restricts browsers. Configure it centrally:

WebConfig.java JAVA

```java
package com.example.library.web;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins("https://app.example.com")
                .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE")
                .allowedHeaders("Authorization", "Content-Type")
                .maxAge(3600);
    }
}
```

For a single controller, `@CrossOrigin(origins = "...")` works. Avoid `allowedOrigins("*")` together with credentials. If Spring Security is on the classpath, you must also enable CORS in the security filter chain (see the security chapter), because the security filters run first.

2.3.6 Filters versus interceptors

Both run code around requests, at different layers.

Aspect	Servlet `Filter`	Spring `HandlerInterceptor`
Runs	before the DispatcherServlet	inside it, around the controller
Knows the handler	no	yes (`preHandle`, `postHandle`, `afterCompletion`)
Typical use	security, logging, compression, request ids	auth checks per handler, timing, locale
Registered as	a `@Component` bean or `FilterRegistrationBean`	`WebMvcConfigurer.addInterceptors`

Filter and interceptor compared.

Rule of thumb: use a filter for anything that must also apply to non-Spring-MVC requests (security, tracing ids), and an interceptor when you need handler information. Spring Security itself is a chain of filters.

2.3.7 Calling other services with RestClient

Your API will often call another HTTP service. `RestClient` (Spring 6.1+) is the modern synchronous client with a fluent API. It replaces `RestTemplate` for new code (which is in maintenance mode). Build one with a base URL and **always set timeouts**: the defaults can wait indefinitely, and a slow dependency would then exhaust your Tomcat threads.

IsbnClient.java JAVA

```java
package com.example.library.integration;

import java.net.http.HttpClient;
import java.time.Duration;
import org.springframework.http.client.JdkClientHttpRequestFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

@Component
public class IsbnClient {

    private final RestClient restClient;

    public IsbnClient(RestClient.Builder builder) {
        HttpClient httpClient = HttpClient.newBuilder()
```

IsbnClient.java (continued) JAVA

```
                .connectTimeout(Duration.ofSeconds(2))
                .build();
        JdkClientHttpRequestFactory factory = new JdkClientHttpRequestFactory(httpClient);
        factory.setReadTimeout(Duration.ofSeconds(3));
        this.restClient = builder
                .baseUrl("https://isbn.example.com")
                .requestFactory(factory)
                .build();
    }

    public IsbnInfo lookup(String isbn) {
        return restClient.get()
                .uri("/v1/isbn/{isbn}", isbn)
                .retrieve()
                .body(IsbnInfo.class);
    }

    public record IsbnInfo(String isbn, String title) { }
}
```

What to notice: the injected `RestClient.Builder` is pre-configured by Boot (message converters, observability), so build from it instead of calling `RestClient.create()`. `retrieve()` throws `HttpClientErrorException` / `HttpServerErrorException` for 4xx and 5xx responses; catch these and translate to your own exceptions. Newer Boot versions also offer shared timeout properties and declarative `@HttpExchange` client interfaces; check the reference for your version. For retries and circuit breaking, look at Resilience4j or Spring's `@Retryable` (core retry support moved into Spring Framework 7).

NOTE
`RestClient` is blocking. For reactive stacks use `WebClient` (Spring WebFlux), which is outside this guide.

2.3.8 Virtual threads and documentation

Virtual threads (Java 21+, see the concurrency Part of the Core Java book) let a blocking request handler wait cheaply. In Boot 3.2 and later, set one property and Tomcat plus the async executors use virtual threads:

application.properties PROPERTIES

```
spring.threads.virtual.enabled=true
```

It helps most for I/O-bound services that call databases and other services. It does not speed up CPU-bound work. Pitfalls: connection pools (a database pool of 10 still limits concurrency) and old code that holds `synchronized` blocks around I/O (pinning is much reduced from Java 24 on).

OpenAPI docs. The library `springdoc-openapi` (dependency `springdoc-openapi-starter-webmvc-ui`) reads your controllers and publishes an OpenAPI description at `/v3/api-docs` and an interactive page at `/swagger-ui.html`. Use it so frontend developers and testers can try the API without reading code. Pick the springdoc release that matches your Boot generation (the 2.x line targets Boot 3; Boot 4 needs the 3.x line). Protect or disable the page in production.

INTERVIEW ANGLE
What is the difference between PUT and PATCH, and why does idempotency matter? PUT replaces the whole resource with the body, so repeating it gives the same state (idempotent). PATCH applies a partial change and may not be idempotent (for example "increment by 1"). Idempotency matters because clients and proxies retry on timeouts; an idempotent operation is safe to repeat, while a non-idempotent POST needs an idempotency key to avoid duplicates.

REAL WORLD
Production APIs commonly add a request id filter that puts a UUID in the logging context and returns it in a response header, so a client can quote it when reporting a problem and you can find the exact log lines.

Check yourself

1. Which of these URLs follows REST conventions: `/api/getBooks`, `/api/books`, `/api/book/list`?
2. A client retries `PUT /books/5` after a timeout. Is anything wrong? What if it was `POST /books`?
3. What is wrong here?
   `RestClient.create().get().uri("https://slow.example.com").retrieve().body(String.class)`

4. Why does a browser report a CORS error while the same call works from curl?
5. What status should a successful `DELETE` return when there is no body?
6. What does `spring.threads.virtual.enabled=true` change?

Answers

1. `/api/books`; the verb comes from the HTTP method, the URL is a plural noun.
2. PUT is idempotent, so the retry is harmless. A repeated POST may create a duplicate unless you use an idempotency key.
3. No timeouts are set, so a slow server can hold the thread indefinitely; also it ignores Boot's pre-configured builder.
4. CORS is enforced only by browsers; the server must send allow headers for the calling origin.
5. 204 No Content.
6. Tomcat request handling and Spring's async task executors run on virtual threads instead of platform threads.

KEY POINTS

Remember these points.

- Name resources with plural nouns, pick HTTP verbs by semantics, and know which are idempotent.
- Return the right status code, paginate every list, and choose a versioning scheme before you need it.
- Configure CORS centrally; remember it is a browser-only rule. Filters wrap the whole servlet pipeline, interceptors wrap controllers.
- Use `RestClient` with explicit connect and read timeouts for outbound calls.
- `spring.threads.virtual.enabled=true` suits I/O-bound services; springdoc generates OpenAPI docs.

PART 3

Data Access and Testing

Talking to a database with Spring Data JPA, avoiding its classic traps, and proving your code works.

In this part

- **3.1** Spring Data JPA basics
- **3.2** Relationships, transactions and the N+1 problem
- **3.3** Testing Spring Boot applications

3.1 Spring Data JPA basics

Almost every backend application stores data in a relational database. Plain JDBC works, but you write the same code again and again: open a connection, prepare a statement, bind parameters, loop over a `ResultSet`, copy columns into objects, close everything. Object-relational mapping (ORM) removes that work. You describe how a class maps to a table, and a library generates the SQL. In the Spring world the stack has three layers, and interviewers like to hear you separate them.

Name	What it is
JPA (Jakarta Persistence)	A specification: annotations such as `@Entity` and an API (`EntityManager`). It is only interfaces and rules, no code that talks to a database.
Hibernate	The most common JPA implementation (the "provider"). It generates SQL, tracks changes and caches objects. Spring Boot uses it by default.
Spring Data JPA	A Spring layer on top of JPA. You declare an interface and Spring generates the repository implementation for you at startup.

Three names that are often confused.

The packages are `jakarta.persistence.*` in Spring Boot 3 and 4 (older tutorials show `javax.persistence`, which no longer works).

3.1.1 Dependencies and configuration

Add the starter and a driver. H2 is an in-memory database that is perfect for learning and tests; PostgreSQL is what you would run for real.

pom.xml (dependencies) XML

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>
</dependencies>
```

With H2 on the classpath and no datasource configured, Boot starts an embedded in-memory database automatically. For PostgreSQL you give the URL and credentials. Keep the password out of the file (see the chapter on configuration in Part 1) and read it from an environment variable.

application.properties PROPERTIES

```
spring.datasource.url=jdbc:postgresql://localhost:5432/library
spring.datasource.username=library
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
```

Boot builds a connection pool (HikariCP) for you. You normally do not set the dialect any more; Hibernate detects it from the connection.

3.1.2 Entities

An entity is a class that maps to a table. It needs `@Entity`, an `@Id` field, and a no-argument constructor (JPA creates objects by reflection; the constructor may be `protected`). Entities cannot be records, because Hibernate needs to set fields after construction and may subclass the class to build proxies.

Book.java JAVA

```
package com.example.library;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "books")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String title;

    @Column(nullable = false, unique = true, length = 20)
    private String isbn;

    private int copies;

    protected Book() { }            // required by JPA

    public Book(String title, String isbn, int copies) {
        this.title = title;
        this.isbn = isbn;
        this.copies = copies;
    }

    public Long getId() { return id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getIsbn() { return isbn; }
    public int getCopies() { return copies; }
    public void setCopies(int copies) { this.copies = copies; }
}
```

What to notice. `@GeneratedValue(strategy = IDENTITY)` lets the database assign the id (auto-increment column), so `id` is `null` until the row is inserted. `@Column` is optional: without it the field maps to a column with the same name. Boot's default naming strategy turns `copies` into `copies` and `firstName` into `first_name`. Do not use a primitive `long` for the id, because a new object must be able to say "I have no id yet".

> **NOTE**
> `GenerationType.SEQUENCE` is the usual choice on PostgreSQL because it lets Hibernate batch inserts; `IDENTITY` forces an immediate insert to learn the id. For a beginner project `IDENTITY` is simpler and fine.

3.1.3 The persistence context and entity states

Hibernate does not run SQL the moment you call a setter. It keeps loaded entities in a first-level cache called the **persistence context** (one per `EntityManager`, which in a Spring app usually means one per transaction). Every entity instance is in one of four states.

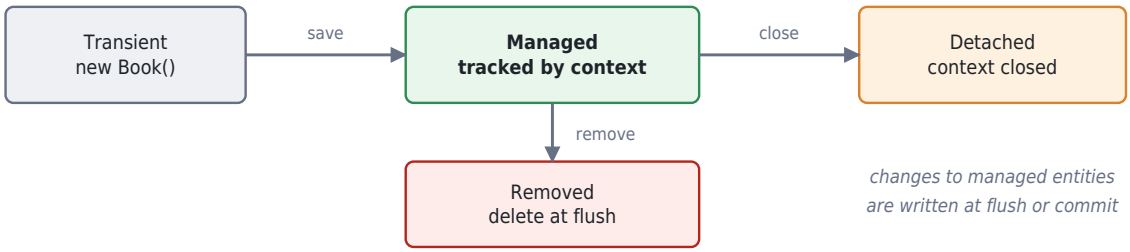


Figure 3.1. Entity lifecycle: only managed entities are tracked, and tracked changes are written at flush or commit.

- **Transient (new):** created with `new`, not known to JPA, no row.
- **Managed (persistent):** attached to the persistence context. Hibernate watches its fields ("dirty checking") and writes any change to the database when the transaction flushes.
- **Detached:** was managed, but its persistence context closed (typically the transaction ended). Changes are no longer tracked.
- **Removed:** scheduled for deletion at flush.

The consequence that surprises beginners: inside a transaction you do not have to call `save` after changing a managed entity. The change is saved automatically at commit.

BookService.java JAVA

```java
@Transactional
public void addCopies(Long bookId, int extra) {
    Book book = bookRepository.findById(bookId).orElseThrow();  // managed
    book.setCopies(book.getCopies() + extra);                   // dirty checking
    // no save() needed: UPDATE is issued when the transaction commits
}
```

3.1.4 Repositories

A Spring Data repository is an interface. You extend `JpaRepository<Entity, IdType>` and get CRUD and paging methods with no implementation code.

BookRepository.java JAVA

```java
package com.example.library;

import java.util.List;
import java.util.Optional;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface BookRepository extends JpaRepository<Book, Long> {

    Optional<Book> findByIsbn(String isbn);

    List<Book> findByTitleContainingIgnoreCase(String fragment);

    boolean existsByIsbn(String isbn);

    Page<Book> findByCopiesGreaterThan(int copies, Pageable pageable);

    @Query("select b from Book b where b.copies = 0")
    List<Book> findOutOfStock();

    @Query("select b from Book b where lower(b.title) like lower(concat('%', :q, '%'))")
    List<Book> search(@Param("q") String q);
}
```

At startup Spring creates a proxy class implementing this interface and registers it as a bean, so you inject `BookRepository` like any other dependency. The inherited methods you use most:

Method	Behaviour
`save(entity)`	INSERT if the entity is new, otherwise merge (UPDATE). Returns the managed instance: use the return value.
`findById(id)`	Returns `Optional<T>`; empty when no row.
`findAll() / findAll(Sort) / findAll(Pageable)`	All rows, sorted, or one page.
`existsById(id), count()`	Cheap checks.
`deleteById(id), delete(entity)`	Remove; `deleteById` on a missing id does nothing in current versions.
`saveAll(list)`	Save many in one call.

Common `JpaRepository` methods.

Derived query methods

Spring parses the method name: `findBy` + property + keyword. `findByTitleContainingIgnoreCase` becomes `where lower(title) like %?%`. Keywords include `And`, `Or`, `Between`, `LessThan`, `GreaterThan`, `In`, `IsNull`, `OrderBy`, `Top`/`First`. If you misspell a property, the application fails at startup with a clear message, which is a feature: you find the bug immediately. Once the name gets long (three conditions or more), switch to `@Query`.

@Query and JPQL

JPQL looks like SQL but speaks in entities and fields (`Book`, `b.title`), not tables and columns. Always bind values with named parameters (`:q`), never by concatenating strings, which would be an SQL injection hole. Add `nativeQuery = true` when you need real SQL.

Paging and sorting

Never return an unbounded list from an API. Pass a `Pageable` and get a `Page` with the content plus total counts.

PagingExample.java JAVA

```java
Pageable pageable = PageRequest.of(0, 20, Sort.by("title").ascending());
Page<Book> page = bookRepository.findByCopiesGreaterThan(0, pageable);

page.getContent();        // the 20 books on this page
page.getTotalElements();  // total rows matching (extra COUNT query)
page.getTotalPages();
page.hasNext();
```

In a controller you can simply declare a `Pageable` parameter and Spring fills it from `?page=0&size=20&sort=title,asc`. Return `Slice` instead of `Page` if you do not need the total, which saves the count query.

3.1.5 Creating the schema: ddl-auto, schema.sql and Flyway

Hibernate can create tables from your entities. The switch is `spring.jpa.hibernate.ddl-auto`:

Value	Effect
`none`	Do nothing. The default for a real database.
`validate`	Check that tables and columns match the entities; fail at startup if not. Good for production.
`update`	Try to alter tables to match. Never adds safety; can not rename or drop columns.
`create / create-drop`	Drop and recreate the schema on startup (and drop on shutdown for `create-drop`). Only for tests.

ddl-auto values.

Boot defaults to `create-drop` for an embedded database (H2) and `none` for everything else, which is why a demo "just works" and a PostgreSQL project does not create tables. In production the schema must be controlled by **versioned migration scripts**, not by Hibernate guessing. The standard tool is Flyway (Liquibase is the alternative). Add `flyway-core` (and `flyway-database-postgresql` for PostgreSQL on recent Flyway versions) and put numbered scripts in `src/main/resources/db/migration`; Boot runs them at startup before Hibernate validates.

db/migration/V1__create_books.sql SQL

```sql
create table books (
    id      bigint generated by default as identity primary key,
    title   varchar(200) not null,
    isbn    varchar(20)  not null unique,
    copies  integer      not null
);
```

`schema.sql` and `data.sql` in `src/main/resources` also work and are simple, but they have no history, no checksums and no ordering across environments. Use them for demos and tests, Flyway for anything deployed.

WATCH OUT

Never run `ddl-auto=update` or `create` against a production database. `create` deletes all data on the next restart. `update` silently drifts the schema away from what your team reviewed, and it cannot do destructive changes like renaming a column, so you end up fixing it by hand at 2 a.m. Use Flyway migrations plus `validate`.

INTERVIEW ANGLE

What is the difference between JPA, Hibernate and Spring Data JPA? JPA is the specification (annotations and the `EntityManager` API). Hibernate is an implementation of that specification that generates SQL and tracks entity changes. Spring Data JPA sits on top: it generates repository implementations from interfaces and derived method names, and calls JPA/Hibernate underneath. You can use Hibernate without Spring Data, but not Spring Data JPA without a JPA provider.

INTERVIEW ANGLE

Why does an update work without calling save()? A loaded entity inside a transaction is managed. At flush time Hibernate compares it with the snapshot taken at load time (dirty checking) and issues an UPDATE for any difference. `save()` is only required for new entities or for detached ones that you want to merge back.

Check yourself

1. Which of JPA, Hibernate and Spring Data JPA is only a specification?
2. A method in a `@Transactional` service does `book.setTitle("X");` on an entity it loaded with `findById` and returns. Is the title updated in the database? Why?
3. What happens at startup if you declare `List<Book> findByTitel(String t)` in the repository?
4. What is wrong with `@Query("select b from Book b where b.title = '" + t + "'")`, and what is the fix?
5. Which `ddl-auto` value would you use with Flyway in production, and why?
6. Can a Java `record` be an `@Entity`? Give one reason.

Answers

1. JPA (Jakarta Persistence). Hibernate implements it; Spring Data JPA builds on it.
2. Yes. The entity is managed, so dirty checking issues an UPDATE when the transaction commits; no `save` call is needed.
3. The context fails to start: Spring cannot find a property named `titel` on `Book` and reports it.
4. String concatenation allows SQL/JPQL injection (and it will not compile as an annotation value anyway if `t` is not a constant). Use a named parameter `:t` with `@Param("t")`.
5. `validate` (or `none`): Flyway owns the schema, Hibernate only checks that it matches.
6. No: JPA needs a no-arg constructor, mutable fields and the ability to subclass the class for lazy proxies; records are final and immutable. Use records for DTOs instead.

KEY POINTS

- JPA is the specification, Hibernate the provider, Spring Data JPA the repository layer on top.
- An entity needs `@Entity`, `@Id` and a no-arg constructor; use `jakarta.persistence` imports.
- Managed entities are dirty-checked: changes inside a transaction are written at commit without `save`.
- Derived query names are checked at startup; use `@Query` with named parameters for anything longer.
- Return `Page` or `Slice` with a `Pageable`, never unbounded lists.
- Production schemas come from Flyway migrations with `ddl-auto=validate`, not from Hibernate generation.

3.2 Relationships, transactions and the N+1 problem

Real data has relationships: an author writes many books, a member borrows many books, a book has many categories. JPA maps these with four annotations, and almost every serious JPA bug comes from using them without understanding two things: who owns the relationship, and when related data is loaded. This chapter covers both, then the transaction rules that make multi-step changes safe.

3.2.1 Mapping relationships

Think in terms of the **foreign key**. In a relational schema, the table that holds the foreign key column is the one that really stores the relationship. JPA calls it the **owning side**. The other side is optional, read-only convenience, and is marked with `mappedBy`.

Author.java JAVA

```java
@Entity
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // inverse side: no column here, the FK lives in the books table
    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Book> books = new ArrayList<>();

    protected Author() { }
    public Author(String name) { this.name = name; }

    public void addBook(Book book) {      // keep both sides in sync
        books.add(book);
        book.setAuthor(this);
    }
    public void removeBook(Book book) {
        books.remove(book);
        book.setAuthor(null);
    }
    public Long getId() { return id; }
    public String getName() { return name; }
    public List<Book> getBooks() { return books; }
}
```

Book.java (relationship part) JAVA

```java
@Entity
public class Book {
    // id, title, isbn as before

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "author_id")      // owning side: this table has the FK column
    private Author author;

    public Author getAuthor() { return author; }
    void setAuthor(Author author) { this.author = author; }
}
```

What to notice. `@ManyToOne` on `Book` is the owning side and creates the `author_id` column. `mappedBy = "author"` on `Author.books` says "the field `author` in `Book` owns this". If you set only `author.getBooks().add(book)` and forget `book.setAuthor(...)`, nothing is written, because the owner was never set. That is why we use the helper methods `addBook` and `removeBook`.

Annotation	Meaning	Default fetch
`@ManyToOne`	Many books, one author. FK in the "many" table.	EAGER
`@OneToMany`	One author, many books. Usually the inverse side of a `@ManyToOne`.	LAZY
`@OneToOne`	One to one, FK on one side.	EAGER

Annotation	Meaning	Default fetch
`@ManyToMany`	Students and courses. Needs a join table (`@JoinTable`).	LAZY

The four relationship annotations.

For `@ManyToMany`, the owning side declares `@JoinTable(name = "loan_books", joinColumns = ..., inverseJoinColumns = ...)`. In practice, when the join needs its own data (a loan has a date and a due date), do not use `@ManyToMany`: create a real `Loan` entity with two `@ManyToOne` fields. That is exactly what the project in Part 5 does.

3.2.2 Cascade and orphan removal

`cascade` passes an operation from parent to children. With `CascadeType.ALL` on `Author.books`, saving an author saves its new books, and deleting an author deletes its books. `orphanRemoval = true` also deletes a book removed from the list. Use cascading only for real parent-owns-child aggregates (order and its lines). Do not put `CascadeType.REMOVE` on a `@ManyToOne` or on `@ManyToMany`: deleting one book must never delete the shared author.

3.2.3 Lazy and eager fetching

Eager means "load the related data together with me". **Lazy** means "put a placeholder (a proxy) there and load the data only when somebody touches it". Lazy is almost always right, because eager loading drags whole object graphs into memory whether you need them or not. Notice the default for `@ManyToOne` and `@OneToOne` is EAGER, so set `fetch = FetchType.LAZY` explicitly, as we did.

The price of lazy loading is `LazyInitializationException`: the proxy can only load data while its persistence context is open (inside the transaction).

LazyBug.java JAVA

```java
public String authorNameOf(Long bookId) {
    Book book = bookRepository.findById(bookId).orElseThrow();   // tx ends here
    return book.getAuthor().getName();   // proxy touched after the tx is closed
}
// org.hibernate.LazyInitializationException: could not initialize proxy
//   [com.example.library.Author#1] - no Session
```

The fix is not "make everything eager". Load what you need inside the transaction: put `@Transactional` on the service method, or better, fetch exactly the data required with a query (below).

GOTCHA

Spring Boot enables `spring.jpa.open-in-view=true` by default. It keeps the persistence context open during the whole web request, so lazy loading appears to work even in your controller and even when the service has no transaction. It hides the bug above, leaks database access into the web layer, and holds a connection longer. Boot logs a warning at startup. Most teams set `spring.jpa.open-in-view=false` and fetch what they need in the service layer.

3.2.4 The N+1 select problem

This is the most asked JPA question in interviews. You load a list of N authors with one query, then read each author's books. Each access to a lazy collection fires its own query: 1 query for the list plus N for the collections.

NPlusOne.java JAVA

```java
@Transactional(readOnly = true)
public List<String> authorsWithCounts() {
    List<Author> authors = authorRepository.findAll();      // 1 query
    List<String> result = new ArrayList<>();
    for (Author a : authors) {
        result.add(a.getName() + ": " + a.getBooks().size());  // 1 query per author
    }
    return result;
}
```

Typical SQL log (spring.jpa.show-sql=true), 3 authors

```
select a1_0.id, a1_0.name from author a1_0
select b1_0.author_id, b1_0.id, b1_0.title from book b1_0 where b1_0.author_id=?
select b1_0.author_id, b1_0.id, b1_0.title from book b1_0 where b1_0.author_id=?
select b1_0.author_id, b1_0.id, b1_0.title from book b1_0 where b1_0.author_id=?
```

With 3 authors it is invisible. With 10,000 it is 10,001 round trips to the database and a slow endpoint that "worked fine on my machine". You find it by turning on SQL logging in development (`logging.level.org.hibernate.SQL=DEBUG`) and counting statements. There are three standard fixes.

Fix 1: join fetch

Tell JPQL to load the collection in the same query.

AuthorRepository.java JAVA

```java
public interface AuthorRepository extends JpaRepository<Author, Long> {

    @Query("select distinct a from Author a left join fetch a.books")
    List<Author> findAllWithBooks();
}
```

One SQL join, one round trip. Limits: you cannot page a collection `join fetch` in the database (Hibernate pages in memory and warns), and fetching two collections at once at the same time multiplies rows.

Fix 2: @EntityGraph

The same idea without writing JPQL. It works on derived methods too.

AuthorRepository.java (alternative) JAVA

```java
@EntityGraph(attributePaths = "books")
@Query("select a from Author a")
List<Author> findAllWithBooksGraph();

@EntityGraph(attributePaths = "books")
Optional<Author> findByName(String name);
```

Fix 3: DTO projection

If you only need a few fields, do not load entities at all. Select straight into a record. It is the fastest option, because there is no persistence context overhead and no lazy loading to trip over.

AuthorSummary.java JAVA

```java
public record AuthorSummary(Long id, String name, long bookCount) { }

// in AuthorRepository
@Query("""
       select new com.example.library.AuthorSummary(a.id, a.name, count(b))
       from Author a left join a.books b
       group by a.id, a.name
       """)
List<AuthorSummary> summaries();
```

A fourth, less common tool is `@BatchSize` (or the property `spring.jpa.properties.hibernate.default_batch_fetch_size=50`), which loads lazy collections for 50 parents at once, turning N queries into N/50. It is a cheap safety net, not a design.

INTERVIEW ANGLE

What is the N+1 problem and how do you fix it? One query loads N parent rows, then lazy access to a relationship on each parent fires N more queries. Fix by fetching the relationship in the initial query: `join fetch` in JPQL, `@EntityGraph`, or select a DTO projection that contains exactly the columns needed. Detect it by logging SQL and counting statements, or with a test that asserts the query count. Switching to EAGER does not fix it (Hibernate still issues the extra selects for `findAll`) and loads data you do not need.

3.2.5 equals and hashCode on entities

Entities live in sets and maps, and Hibernate compares them. Two rules keep you safe. Do not use Lombok `@Data` or an IDE-generated `equals` over all fields, since fields change and lazy collections would be loaded just to compute a hash. And do not base equality on a database-generated `id` naively, because it is `null` before the first save, so the hash code would change after persisting an entity already placed in a `HashSet`.

A pragmatic pattern: equality on a natural business key that never changes (the ISBN), or the id only when non-null with a constant hash code.

Book.java (equality by natural key) JAVA

```java
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Book other)) return false;
    return isbn != null && isbn.equals(other.isbn);
}

@Override
public int hashCode() {
    return java.util.Objects.hashCode(isbn);   // stable for the life of the object
}
```

Use `instanceof` rather than `getClass()` comparison, because Hibernate may hand you a subclass proxy. Do not include relationship fields in `toString` either, or you create infinite loops (author prints books, each book prints its author).

3.2.6 Transactions

A **transaction** groups several database operations so that they either all succeed or all roll back (atomicity). A bank transfer that debits one account and credits another is the classic example. In Spring you declare it with `@Transactional` on a service method.

LoanService.java JAVA

```java
@Service
public class LoanService {

    private final BookRepository books;
    private final LoanRepository loans;

    public LoanService(BookRepository books, LoanRepository loans) {
        this.books = books;
        this.loans = loans;
    }

    @Transactional
    public Loan borrow(Long bookId, Long memberId) {
        Book book = books.findById(bookId).orElseThrow(() -> new BookNotFound(bookId));
        if (book.getCopies() == 0) {
            throw new NoCopiesLeft(bookId);        // unchecked: rolls back
        }
        book.setCopies(book.getCopies() - 1);      // dirty checking
        return loans.save(new Loan(bookId, memberId));
    }

    @Transactional(readOnly = true)
    public List<Book> available() {
        return books.findByCopiesGreaterThan(0, Pageable.unpaged()).getContent();
    }
}
```

Spring wraps `LoanService` in a proxy (see the chapter on bean lifecycle and proxies in Part 1). The proxy begins a transaction before `borrow`, calls your code, then commits if it returns normally or rolls back if it throws.

The rules you must know

- **Rollback only on unchecked exceptions by default.** A `RuntimeException` or `Error` rolls back. A checked exception (such as `IOException`) commits! Declare `@Transactional(rollbackFor = Exception.class)` if checked exceptions should roll back.
- **Placement:** put it on public methods of Spring beans, in the service layer, not on repositories or controllers. A `private` or `final` method cannot be intercepted by the proxy.
- **Propagation** says what to do if a transaction already exists. The default `REQUIRED` joins the existing one, or starts a new one if none. So a service method calling another `@Transactional` method shares a single transaction, and an exception in the inner one marks the whole transaction rollback-only. `REQUIRES_NEW` suspends the current transaction and runs the inner call in an independent one (use it for audit logging that must survive a rollback). Other values (`SUPPORTS`, `MANDATORY`, `NEVER`, `NESTED`) are rare.
- `readOnly = true` is a hint: Hibernate skips dirty checking and the driver may optimise. Use it on query methods. It is not a security guarantee.

- **Isolation** controls what concurrent transactions see of each other: `READ_COMMITTED` (the default on PostgreSQL) prevents dirty reads, `REPEATABLE_READ` also keeps repeated reads stable, `SERIALIZABLE` behaves as if transactions ran one by one and is the slowest. Leave the default unless you can name the anomaly you are preventing.

Self-invocation: the proxy limit

Because the transaction logic lives in the proxy, a call from one method to another method of the same object never passes through the proxy.

SelfInvocation.java JAVA

```java
@Service
public class ReportService {

    public void monthly() {
        archive();                  // plain this.archive(): proxy bypassed, NO transaction
    }

    @Transactional
    public void archive() {
        // expected to run in a transaction, but does not when called from monthly()
    }
}
```

Fixes: move `archive` into another bean and inject it, or put `@Transactional` on the outer method as well. Calling from a controller or another bean works because that call goes through the proxy.

WATCH OUT

Catching an exception inside a `@Transactional` method and swallowing it prevents the rollback: the proxy sees a normal return and commits. Either rethrow, or deliberately design the catch as part of your business logic. Likewise, never keep a transaction open across a slow HTTP call to another service; it holds a database connection while waiting.

INTERVIEW ANGLE

Does @Transactional roll back on a checked exception? By default, no. Rollback happens for `RuntimeException` and `Error`; a checked exception commits the work done so far. Use `rollbackFor` to change that. A second favourite follow-up: "Why does @Transactional not work when I call the method from another method in the same class?" Because Spring applies it through a proxy, and a `this.` call skips the proxy.

Check yourself

1. In the `Author`/`Book` mapping above, which side owns the relationship and how do you know?
2. You write `author.getBooks().add(book);` and save the author, but `book.author_id` is null. Why?
3. What is wrong here, and what exception do you expect? `Book b = repo.findById(1L).get(); return b.getAuthor().getName();` in a method without `@Transactional`, with `open-in-view=false`.
4. A page lists 50 authors with their book counts and takes 51 SQL statements. Name the problem and two fixes.
5. A `@Transactional` method throws `new IOException("disk")`. Is the data rolled back?
6. Method `a()` of a bean calls `this.b()` which is annotated `@Transactional(propagation = REQUIRES_NEW)`. How many transactions are created by calling `a()` from outside?

Answers

1. `Book`, because it has the `@ManyToOne` with the `@JoinColumn`; the foreign key column lives in its table. `Author.books` is the inverse side (`mappedBy`).
2. The owning side `Book.author` was never set. Update both sides (helper method `addBook`).
3. `LazyInitializationException`: the author proxy is touched after the persistence context closed. Fetch the author in the query or add a transaction.
4. N+1 select. Fix with `join fetch` / `@EntityGraph`, or a DTO projection with `count` and `group by`.
5. No. Checked exceptions do not trigger rollback by default; add `rollbackFor = Exception.class`.
6. Zero, unless `a()` itself is transactional: the `this.b()` call bypasses the proxy so the annotation on `b` is ignored.

KEY POINTS

- The side with the foreign key owns the relationship; `mappedBy` marks the inverse side, and you must set both sides in code.
- Make associations LAZY (set it explicitly for `@ManyToOne`/`@OneToOne`) and fetch what you need with `join fetch`, `@EntityGraph` or a DTO projection.
- N+1 is one list query plus one query per row's lazy relation; find it by counting logged SQL.
- `LazyInitializationException` means you touched a proxy outside the transaction; consider disabling open-in-view.
- `@Transactional` rolls back on unchecked exceptions only, joins existing transactions by default (`REQUIRED`), and works through a proxy, so self-invocation is not covered.
- Base entity `equals`/`hashCode` on a stable business key, never on all fields or on relationship collections.

3.3 Testing Spring Boot applications

Tests are what let you change code without fear, and interviewers expect you to know the difference between a fast unit test and a slow integration test. Spring Boot gives you a rich set of test tools, but the most important idea is not a tool: test each thing at the cheapest level that can still catch the bug.

3.3.1 The test pyramid

The test pyramid

```
          /\
         /  \      few    end-to-end: whole app + real DB (@SpringBootTest, Testcontainers)
        /----\
       /      \    some   slice tests: one layer + Spring (@WebMvcTest, @DataJpaTest)
      /--------\
     /          \  many   unit tests: plain Java + JUnit + Mockito, no Spring at all
    /____________\
```

Unit tests run in milliseconds because no Spring context is started. Slice tests start only the beans of one layer (a few hundred milliseconds to a couple of seconds). Full `@SpringBootTest` tests start the whole application and cost seconds each, and Spring caches the context between tests with the same configuration to soften that.

The starter `spring-boot-starter-test` (added by start.spring.io as a `test` dependency) brings JUnit 5 (Jupiter), AssertJ, Mockito, Hamcrest, JSONassert, JsonPath and Spring's test support.

NOTE

Spring Boot 4 splits test support into per-technology test starters, for example `spring-boot-starter-webmvc-test` for MVC tests and `spring-boot-starter-data-jpa-test` for JPA tests. Generate the project on start.spring.io and tick the starters you use, and it adds the matching test starters. Spring Boot 3.5 only needs `spring-boot-starter-test`. The test annotations are the same names; in Boot 4 some of them live in new packages, so let the IDE fix imports. This chapter marks the differences.

3.3.2 Unit tests without Spring

Keep business logic in plain classes with constructor injection (as the previous Parts recommended), and you can test it with `new`. Mockito replaces the collaborators, AssertJ gives readable assertions.

LoanServiceTest.java JAVA

```java
package com.example.library;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

import java.util.Optional;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
```

LoanServiceTest.java (continued) JAVA

```java
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
class LoanServiceTest {

    @Mock BookRepository books;
    @Mock LoanRepository loans;
    @InjectMocks LoanService service;

    @Test
    void borrowReducesCopies() {
        Book book = new Book("Dune", "978-0", 2);
        when(books.findById(1L)).thenReturn(Optional.of(book));
        when(loans.save(any(Loan.class))).thenAnswer(inv -> inv.getArgument(0));

        service.borrow(1L, 7L);

        assertThat(book.getCopies()).isEqualTo(1);
    }

    @Test
    void borrowFailsWhenNoCopiesLeft() {
        when(books.findById(1L)).thenReturn(Optional.of(new Book("Dune", "978-0", 0)));

        assertThatThrownBy(() -> service.borrow(1L, 7L))
            .isInstanceOf(NoCopiesLeft.class);
        verify(loans, never()).save(any());
    }
}
```

What to notice. There is no Spring annotation in sight: `@ExtendWith(MockitoExtension.class)` is Mockito's JUnit 5 extension, `@Mock` creates the fakes and `@InjectMocks` calls the constructor with them. `when(...).thenReturn(...)` stubs a call; `verify(...)` checks an interaction happened (or, with `never()`, did not). Name tests after behaviour (`borrowFailsWhenNoCopiesLeft`), and follow arrange, act, assert.

TIP

Mockito by default (strict stubs under `MockitoExtension`) fails a test that stubs a call it never uses. That is useful: it flags dead setup. Do not mock what you do not own (such as `String` or a JDK collection) or simple value objects like `Book`; use real instances.

3.3.3 Controller tests with @WebMvcTest

`@WebMvcTest(BookController.class)` starts only the web layer: the controller, JSON conversion, validation, `@RestControllerAdvice`, and security filters. It does not create services or repositories, so you provide fakes with `@MockitoBean`. `MockMvc` then sends requests to the `DispatcherServlet` without opening a network port.

BookControllerTest.java JAVA

```java
package com.example.library;

import static org.hamcrest.Matchers.is;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(BookController.class)
class BookControllerTest {

    @Autowired MockMvc mvc;
    @MockitoBean BookService service;
```

BookControllerTest.java (continued) JAVA

```
    @Test
    void getReturnsBook() throws Exception {
        when(service.find(1L)).thenReturn(new BookResponse(1L, "Dune", "978-0", 2));

        mvc.perform(get("/api/books/1"))
           .andExpect(status().isOk())
           .andExpect(jsonPath("$.title", is("Dune")));
    }

    @Test
    void missingBookIs404() throws Exception {
        when(service.find(9L)).thenThrow(new BookNotFound(9L));

        mvc.perform(get("/api/books/9"))
           .andExpect(status().isNotFound());
    }

    @Test
    void invalidBodyIs400() throws Exception {
        mvc.perform(post("/api/books")
                .contentType(MediaType.APPLICATION_JSON)
                .content("{\"title\":\"\",\"isbn\":\"1\",\"copies\":-1}"))
           .andExpect(status().isBadRequest());
    }
}
```

What to notice. `@MockitoBean` (from Spring Framework 6.2) replaces the old `@MockBean`, which is removed in Boot 4; it registers a Mockito mock in the test's application context. The third test shows the value of this slice: validation and the error handler run, with no service and no database. In Boot 3.5 `@WebMvcTest` is in `org.springframework.boot.test.autoconfigure.web.servlet`; in Boot 4 it moved to `org.springframework.boot.webmvc.test.autoconfigure` and needs `spring-boot-starter-webmvc-test`. `get`/`post`/`jsonPath` are unchanged.

GOTCHA

If your app has Spring Security, `@WebMvcTest` includes the security filter chain, so requests return 401 or 403 until you authenticate. Add `@WithMockUser(roles = "ADMIN")` on the test (needs `spring-security-test`), or `.with(csrf())` for POSTs when CSRF is enabled. Do not "fix" it by deleting security from the test; you would stop testing the real rules.

3.3.4 Repository tests with @DataJpaTest

`@DataJpaTest` starts only JPA: entities, repositories, a `TestEntityManager`. It replaces your datasource with an embedded H2 database (if on the classpath) and wraps each test in a transaction that rolls back afterwards, so tests do not pollute each other.

BookRepositoryTest.java JAVA

```
package com.example.library;

import static org.assertj.core.api.Assertions.assertThat;

import java.util.Optional;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;

@DataJpaTest
class BookRepositoryTest {

    @Autowired BookRepository books;

    @Test
    void findsByIsbn() {
        books.save(new Book("Dune", "978-0", 3));

        Optional<Book> found = books.findByIsbn("978-0");

        assertThat(found).isPresent();
        assertThat(found.get().getTitle()).isEqualTo("Dune");
    }
```

BookRepositoryTest.java (continued) JAVA

```
    @Test
    void searchIgnoresCase() {
        books.save(new Book("Dune", "978-0", 3));
        assertThat(books.search("DUN")).hasSize(1);
    }
}
```

Test your own queries (derived methods with more than one condition, `@Query` strings, paging), not the `save` and `findById` that Spring Data already tested. In Boot 3.5 the annotation is in `org.springframework.boot.test.autoconfigure.orm.jpa`; in Boot 4 it moved to `org.springframework.boot.data.jpa.test.autoconfigure` (companion starter `spring-boot-starter-data-jpa-test`). Check the import your IDE offers.

WATCH OUT

An in-memory H2 database is not PostgreSQL. SQL that works on H2 can fail in production (JSON columns, `ilike`, sequences, locking). For queries that matter, run the test against the real engine with Testcontainers (below), or at least H2 in `MODE=PostgreSQL`.

3.3.5 Full-stack tests with @SpringBootTest

`@SpringBootTest` loads the complete application context. With `webEnvironment = RANDOM_PORT` it also starts the real embedded server on a random free port, and you call it over HTTP.

LibraryApiIT.java JAVA

```
package com.example.library;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class LibraryApiIT {

    @Autowired TestRestTemplate rest;

    @Test
    void createThenFetchBook() {
        var request = new CreateBookRequest("Dune", "978-0-441", 2);

        ResponseEntity<BookResponse> created =
            rest.postForEntity("/api/books", request, BookResponse.class);
        assertThat(created.getStatusCode()).isEqualTo(HttpStatus.CREATED);

        BookResponse fetched =
            rest.getForObject("/api/books/" + created.getBody().id(), BookResponse.class);
        assertThat(fetched.title()).isEqualTo("Dune");
    }
}
```

NOTE

In Boot 4, `@SpringBootTest` no longer provides `TestRestTemplate` or `MockMvc` automatically. Add `@AutoConfigureTestRestTemplate` (plus the `spring-boot-resttestclient` test dependency; the class lives in `org.springframework.boot.resttestclient`) or `@AutoConfigureMockMvc`. The newer `RestTestClient` (enabled with `@AutoConfigureRestTestClient`) is the recommended replacement for `TestRestTemplate` and has a fluent `expectStatus()` / `expectBody()` style like `WebTestClient`. In Boot 3.5 only the listing above is needed.

Full-stack tests are slow and blunt, so write few of them: one per main use case (create, read, borrow, return). They prove the pieces are wired together: component scanning, configuration, Flyway migrations, security, serialisation.

3.3.6 Real databases with Testcontainers

Testcontainers starts a throwaway Docker container (here PostgreSQL) for your tests, so tests run against the same engine as production. Spring Boot integrates it with `@ServiceConnection`, which configures the datasource from the running container, with no URL to write.

PostgresIT.java (needs Docker and testcontainers dependencies) JAVA

```java
@SpringBootTest
@Testcontainers
class PostgresIT {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpine");

    @Autowired BookRepository books;

    @Test
    void persistsAgainstRealPostgres() {
        Book saved = books.save(new Book("Dune", "978-0", 1));
        assertThat(books.findById(saved.getId())).isPresent();
    }
}
```

You need the `spring-boot-testcontainers`, `org.testcontainers:junit-jupiter` and `org.testcontainers:postgresql` test dependencies (Boot 4 packages differ slightly; the Testcontainers site documents the BOM versions) and a running Docker daemon. It is the best way to make Flyway migrations and native queries trustworthy. To keep the build fast, reuse one container across test classes with a shared static container to keep the build fast.

3.3.7 Which annotation for which job

Annotation	Starts	Use it for
`(none) + JUnit/Mockito`	Nothing	Business logic in services, mappers, validators
`@WebMvcTest`	Controllers, advice, JSON, filters	Status codes, JSON shape, validation, error responses
`@DataJpaTest`	Entities, repositories, embedded DB	Custom queries, mappings, constraints
`@JsonTest`	Jackson only	Serialisation of DTOs
`@SpringBootTest`	Whole application	A few end-to-end flows, wiring checks
`@MockitoBean`	Mock in the Spring context	Replace a bean (service, client) in a slice test
`@TestConfiguration`	Extra beans for a test	Fixed clocks, stub clients

Test annotations at a glance.

3.3.8 What to test

- **Business rules and branches**: the borrow limit, the "no copies left" path, boundary values (0, 1, max).
- **Contracts at the edge**: status codes, validation errors, error body, JSON field names. These are what clients depend on.
- **Your queries and mappings**: custom `@Query`, unique constraints, cascades.
- **Failure paths**: not found, duplicate, invalid input, unauthorised. Bugs live here.
- **Do not test**: getters and setters, Spring itself, or private methods. Test through the public behaviour.

Make tests independent (no test relies on another test's data), deterministic (inject a `Clock` instead of calling `LocalDate.now()`), and fast. Run them with `./mvnw test`; by convention Maven's Surefire plugin runs classes named `*Test` and Failsafe (`./mvnw verify`) runs `*IT`.

INTERVIEW ANGLE

What is the difference between @Mock and @MockitoBean (formerly @MockBean)? `@Mock` is plain Mockito: it creates a fake object, no Spring involved, ideal for unit tests with `@InjectMocks`. `@MockitoBean` creates a Mockito mock and registers it in the Spring `ApplicationContext`, replacing any bean of that type, so Spring injects the fake into the controller or service under test. `@MockBean` did the same job and was deprecated in Boot 3.4 and removed in Boot 4.

INTERVIEW ANGLE

Why prefer @WebMvcTest over @SpringBootTest for a controller? It starts only the web layer, so it is much faster and tests exactly what the controller is responsible for: mapping, validation, serialisation and error handling. Collaborators are mocked, so failures point at the controller. Use `@SpringBootTest` sparingly to prove the layers are wired together.

Check yourself

1. Which test type would you write for "returns 409 when the ISBN already exists" in the controller, and which annotation supplies the service?
2. What does this print? Why? `@DataJpaTest` test A saves a `Book` and ends; test B calls `books.count()` first thing.
3. A `@WebMvcTest` for a secured controller returns 401. Name two ways to fix the test without disabling security.
4. Why can `@DataJpaTest` with H2 give you false confidence, and what is the remedy?
5. What is wrong: `@Autowired private BookService service;` inside a class annotated only `@WebMvcTest(BookController.class)`?
6. What does `@SpringBootTest(webEnvironment = RANDOM_PORT)` do that the default mock environment does not?

Answers

1. A `@WebMvcTest` with MockMvc; `@MockitoBean BookService` throws the "duplicate" exception and the test expects status 409.
2. B sees 0 rows (if the table was empty before): each `@DataJpaTest` test runs in a transaction that is rolled back at the end, so test A's data is gone.
3. Authenticate the request with `@WithMockUser(...)` or a `jwt()` / `httpBasic()` request post-processor; add `csrf()` for state-changing requests when CSRF is on.
4. H2 differs from PostgreSQL in SQL dialect, types and locking. Run the repository tests on the real engine with Testcontainers.
5. `@WebMvcTest` does not create `@Service` beans, so the context fails with a missing bean. Use `@MockitoBean` for the service.
6. It starts a real embedded server on a random free port, so you test over actual HTTP using `TestRestTemplate` or a client, including servlet filters.

KEY POINTS

- Test at the cheapest level: many plain unit tests, some slice tests, few full `@SpringBootTest` runs.
- Unit tests need no Spring: constructor injection plus Mockito and AssertJ.
- `@WebMvcTest` + `MockMvc` + `@MockitoBean` tests the web layer; `@DataJpaTest` tests your queries with automatic rollback.
- `@MockitoBean` replaces `@MockBean`; in Boot 4 also check package changes and the added `@AutoConfigure...` annotations on `@SpringBootTest`.
- Use Testcontainers when the real database engine matters (migrations, native SQL).
- Test behaviour and failure paths, not getters or the framework.

PART 4

Securing and Shipping

Lock the API down, watch it run, and package it so it behaves the same on your laptop and in production.

In this part

4.1 Spring Security basics
4.2 Observability and shipping

4.1 Spring Security basics

Every API that is not a toy needs to answer two questions on every request: who is calling, and are they allowed to do this? Spring Security is the framework that answers both. It is a large library with a reputation for being confusing, mostly because people copy configuration they do not understand. The good news is that the core model is small: a chain of servlet filters, an `Authentication` object, and a few rules. Learn those and the rest is detail.

This chapter targets Spring Security 6.x (Spring Boot 3.5) and 7.x (Spring Boot 4). The configuration style shown here, a `SecurityFilterChain` bean using the lambda DSL, works on both.

4.1.1 What you get by adding the starter

Add `spring-boot-starter-security` to the pom and start the application without writing any security code. Boot auto-configures a default setup:

- Every endpoint requires authentication.
- There is one user named `user`, with a random password printed once in the startup log.
- Both a login form and HTTP Basic are enabled, so a browser gets a form and `curl -u` works.
- CSRF protection is on, so `POST`, `PUT` and `DELETE` without a CSRF token get `403`.
- Security headers (such as `X-Content-Type-Options`) are added to responses.

Typical output: the default user (the password differs every run) TERMINAL

```
$ ./mvnw spring-boot:run
Using generated security password: 8e557245-73e2-4286-969a-2f0f1d1e0c3b

This generated password is for development use only.
Your security configuration must be updated before running in production.
```

This default is a safe starting point, which is exactly why you must replace it: nobody ships an API whose only user is `user` with a random password.

4.1.2 The filter chain

Spring Security is not magic inside your controllers. It is a set of servlet filters that run before the `DispatcherServlet` ever sees the request. A single servlet filter, `DelegatingFilterProxy` (in Boot, a registered `FilterChainProxy`), hands each request to the first `SecurityFilterChain` whose request matcher matches. That chain is an ordered list of filters, each with one job.

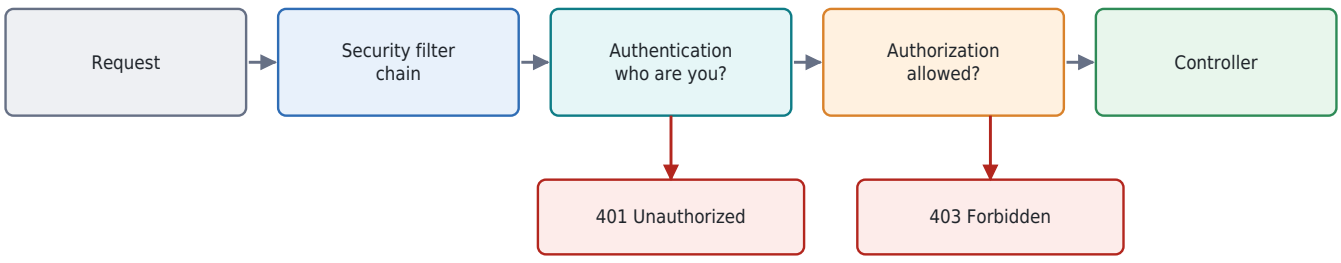


Figure 4.1. A request passes through the filter chain; each filter either lets it continue or stops it with 401 or 403 before any controller code runs.

The filters you should know by name:

Filter	Job
`CorsFilter`	Answers browser pre-flight requests; must run before authentication
`CsrfFilter`	Rejects state-changing requests without a valid CSRF token
`BasicAuthenticationFilter`	Reads the `Authorization: Basic` header and authenticates
`BearerTokenAuthenticationFilter`	Reads `Authorization: Bearer` and validates a JWT
`ExceptionTranslationFilter`	Turns security exceptions into 401 (not logged in) or 403 (not allowed)
`AuthorizationFilter`	Last filter: checks the request against your `authorizeHttpRequests` rules

Important filters in a typical chain.

When authentication succeeds, the result is stored in the `SecurityContext` (held in a thread-local for the duration of the request). Anywhere in your code you can read it with `SecurityContextHolder.getContext().getAuthentication()`, or let Spring inject it as a controller parameter (`Authentication`, `Principal`, or `@AuthenticationPrincipal`).

4.1.3 Authentication versus authorisation

These two words are constantly confused, and interviewers love the difference.

- **Authentication** (authn) proves who you are: a password, a token, a certificate. Failure means `401 Unauthorized` (the name is unfortunate; it really means unauthenticated).
- **Authorisation** (authz) decides what an identified user may do: roles and permissions. Failure means `403 Forbidden`.

An `Authentication` object carries a principal (the user), credentials (cleared after login), and a list of `GrantedAuthority` values such as `ROLE_ADMIN` or `SCOPE_books.read`. Authorisation rules are checks against that list.

4.1.4 Your own SecurityFilterChain

As soon as you define a `SecurityFilterChain` bean, Boot's default chain backs off and yours replaces it completely. Here is a realistic configuration for a stateless JSON API using HTTP Basic (we will swap Basic for JWT later):

SecurityConfig.java JAVA

```java
package com.example.shop.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

What to notice

- Rules are evaluated top to bottom and the first match wins. Put specific rules before general ones, and always end with `anyRequest()`. A broad `permitAll` placed first silently opens everything below it.
- `hasRole("ADMIN")` checks for the authority `ROLE_ADMIN`. Spring adds the `ROLE_` prefix for you, so storing `ROLE_ADMIN` and calling `hasRole("ROLE_ADMIN")` is a common double-prefix bug. Use `hasAuthority` when you want the exact string.
- `requestMatchers` replaced `antMatchers` and `mvcMatchers` (removed in Security 6). `WebSecurityConfigurerAdapter` is also gone: if a tutorial extends it, the tutorial is outdated.
- Requests that match no rule are not automatically denied in every setup, so the closing `anyRequest().authenticated()` (or `denyAll()`) is your safety net.

4.1.5 Users and passwords

Authentication needs a source of users. Spring Security abstracts it as `UserDetailsService`, a single method `loadUserByUsername`. Boot provides an in-memory one for the default user. For real applications you implement it against your user table:

DbUserDetailsService.java JAVA

```java
package com.example.shop.security;

import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

@Service
public class DbUserDetailsService implements UserDetailsService {

    private final AppUserRepository users;

    public DbUserDetailsService(AppUserRepository users) {
        this.users = users;
    }

    @Override
    public UserDetails loadUserByUsername(String username) {
        AppUser u = users.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("No such user"));
        return User.withUsername(u.getUsername())
            .password(u.getPasswordHash())      // already hashed
            .roles(u.getRole())                 // "ADMIN" becomes ROLE_ADMIN
            .build();
    }
}
```

`AppUser` and `AppUserRepository` are your own JPA entity and repository (see the Spring Data chapter). Boot picks up a single `UserDetailsService` bean automatically.

Never store plain-text passwords. Store a salted, deliberately slow hash. Spring Security's `PasswordEncoder` does both:

PasswordConfig.java JAVA

```java
package com.example.shop.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class PasswordConfig {

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();   // default strength 10
    }
}
```

You call `encoder.encode(raw)` when a user registers and Spring Security calls `matches(raw, stored)` at login. BCrypt generates a fresh random salt each time, so encoding the same password twice gives different strings, and the salt and cost are embedded in the hash itself. `PasswordEncoderFactories.createDelegatingPasswordEncoder()` stores hashes as `{bcrypt}$2a$...`, which lets you migrate algorithms later without a flag day.

GOTCHA
Two encodings of the same password never compare equal with `equals`. A beginner bug is `encode(raw).equals(stored)`. It is always false because of the random salt. Always call `passwordEncoder.matches(raw, stored)`. Another classic: seeding test users with a plain password in the database while the encoder is BCrypt gives "Encoded password does not look like BCrypt" warnings and a 401 on every login.

4.1.6 Basic, session or JWT?

The question "how does the client prove who it is on request number two?" has three common answers.

Style	How it works	Trade-offs
HTTP Basic	Username and password sent in a header on every request	Trivial to use and test; password travels on every call, so HTTPS is mandatory; fine for internal tools and demos
Session cookie	Login once, server stores a session, browser sends `JSESSIONID`	Natural for server-rendered sites; needs sticky or shared sessions to scale; needs CSRF protection
JWT bearer token	Client sends a signed token; server verifies the signature, no server-side session	Stateless and scales easily; hard to revoke before expiry; keep tokens short-lived

Authentication styles compared.

A **JWT** (JSON Web Token) is three Base64URL parts separated by dots: header, payload (claims such as `sub`, `exp`, `scope`), and a signature. The signature proves who issued it and that nobody changed it. The payload is encoded, not encrypted: never put secrets in it.

INTERVIEW ANGLE
How does JWT authentication work in a Spring Boot API? A login endpoint (or, better, a separate identity provider such as Keycloak or an OAuth2 authorization server) issues a signed, short-lived token. The client sends it as `Authorization: Bearer <token>`. A filter verifies the signature, expiry and issuer, builds an `Authentication` with the claims as authorities, and stores it in the `SecurityContext`. The server keeps no session, so any instance can serve any request. The downside is revocation: a leaked token stays valid until it expires, so use short lifetimes plus refresh tokens.

4.1.7 A JWT resource server

In modern Spring the recommended pattern is that your API is a **resource server**: it only validates tokens, and a dedicated identity provider issues them. Writing your own token-signing login endpoint is possible but easy to get subtly wrong. Add `spring-boot-starter-oauth2-resource-server` and point it at the issuer:

application.yml YAML

```yaml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.example.com/realms/shop
```

Boot downloads the issuer's public keys from its discovery document and uses them to verify signatures. The filter chain changes by one line:

JwtSecurityConfig.java JAVA

```java
package com.example.shop.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
```

JwtSecurityConfig.java (continued) JAVA

```java
@Configuration
public class JwtSecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**").permitAll()
                .requestMatchers("/api/admin/**").hasAuthority("SCOPE_admin")
                .anyRequest().authenticated())
            .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
        return http.build();
    }
}
```

By default, the `scope` claim becomes authorities with a `SCOPE_` prefix. If your provider puts roles in another claim (Keycloak uses `realm_access.roles`), configure a `JwtAuthenticationConverter` bean to map them. This is a sketch: details depend on your identity provider, so read its documentation and Spring Security's resource-server reference.

NOTE
Spring Boot does not ship a token issuer. If you need to run one yourself, use Spring Authorization Server or an off-the-shelf product. For the project in this guide we keep HTTP Basic with in-memory users, because it needs no extra infrastructure and lets you focus on the application.

4.1.8 Method security

URL rules are coarse. For rules that depend on the data ("only the owner or an admin may edit this"), put the check on the service method. Enable it once and annotate:

LoanService.java JAVA

```java
package com.example.shop.service;

import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.stereotype.Service;

@Service
public class LoanService {

    @PreAuthorize("hasRole('LIBRARIAN')")
    public void deleteLoan(long id) {
        // delete logic
    }

    @PreAuthorize("hasRole('LIBRARIAN') or #username == authentication.name")
    public String loansOf(String username) {
        return "loans of " + username;
    }
}
```

Add `@EnableMethodSecurity` to a configuration class (it turns on `@PreAuthorize` and `@PostAuthorize`; in Spring Security 6 and later it replaced `@EnableGlobalMethodSecurity`). The expression language can reach method parameters (`#username`) and the current `authentication`. Because this works through a proxy, it carries the same limits as `@Transactional`: calling a secured method from another method in the same class bypasses the check (see the chapter on bean lifecycle and proxies).

4.1.9 CSRF and CORS

CSRF (cross-site request forgery) is an attack on browsers that automatically attach cookies: a malicious page makes your logged-in browser send a request to your site. The defence is a secret token that a malicious page cannot read. Rule of thumb:

- Browser plus session cookie: keep CSRF enabled.
- Stateless API where the client sends the credential in a header (Basic or Bearer) that the browser does not attach on its own: disabling CSRF is reasonable. Do it deliberately, with that reason in mind, not because a `403` was annoying.

CORS (cross-origin resource sharing) is the browser rule that a page from one origin may not call another origin unless the server allows it. Security filters run before Spring MVC, so a CORS configuration that lives only in MVC will not stop the security chain from rejecting the browser's unauthenticated pre-flight `OPTIONS` request. Fix it by enabling CORS in the chain with `.cors(Customizer.withDefaults())` and publishing a `CorsConfigurationSource` bean (or let it find your MVC CORS configuration). The REST design chapter covers the CORS settings themselves.

4.1.10 Common mistakes

- Leaving the default generated user in place or committing real passwords to `application.yml`. Use environment variables or a secret store.
- Returning `403` or `200` where `401` is right, or the reverse: clients rely on the distinction to decide whether to log in again.
- Trusting the client: hiding a button is not authorisation. Enforce on the server.
- Disabling security "temporarily" for tests in the main configuration. Use `@WithMockUser` and the `spring-security-test` helpers in tests instead.
- Rule order mistakes (a broad `permitAll()` before specific rules).
- Logging `Authorization` headers or tokens.
- Long-lived JWTs with no way to revoke them.
- Exposing Actuator endpoints that reveal environment or heap dumps (next chapter).

Check yourself

1. A user logs in successfully but calls `/api/admin/users` and gets a response. Which status means "authenticated but not allowed", and which means "not authenticated"?
2. You store the role as `ROLE_ADMIN` and write `hasRole("ROLE_ADMIN")`. What goes wrong?
3. What is wrong with `if (encoder.encode(raw).equals(stored))`?
4. In this chain, can an anonymous caller read `/api/books`? `.authorizeHttpRequests(a -> a.anyRequest().authenticated().requestMatchers("/api/books").permitAll())`
5. Why can a stateless Bearer-token API usually disable CSRF, while a session-cookie site cannot?
6. Why does `@PreAuthorize` on a method not run when another method of the same bean calls it?

Answers

1. `403 Forbidden` is authenticated but not allowed; `401 Unauthorized` is not authenticated.
2. Spring adds `ROLE_` itself, so it looks for `ROLE_ROLE_ADMIN` and access is denied. Use `hasRole("ADMIN")` or `hasAuthority("ROLE_ADMIN")`.
3. BCrypt salts randomly, so two encodings differ and the comparison is always false. Use `matches(raw, stored)`.
4. No. First match wins and `anyRequest().authenticated()` is first. A rule placed after `anyRequest()` is unreachable.
5. The browser does not automatically attach a Bearer header, so a forged cross-site request carries no credential. Cookies are attached automatically, which is what CSRF exploits.
6. The call goes straight to `this`, bypassing the proxy that applies the security interceptor.

KEY POINTS

Key points

- Adding the starter secures everything with one random user; define your own `SecurityFilterChain` to take control.
- Security is a filter chain in front of MVC; rules match top to bottom, first match wins, end with `anyRequest()`.
- 401 means unauthenticated, 403 means authenticated but forbidden; `hasRole` adds the `ROLE_` prefix.
- Hash passwords with BCrypt via `PasswordEncoder`; compare with `matches`.
- For JWT, make the API a resource server and let an identity provider issue short-lived tokens.
- Disable CSRF only for stateless header-credential APIs; use `@PreAuthorize` for data-dependent rules, remembering it is proxy-based.

4.2 Observability and shipping

An application that works on your laptop has done about a third of its job. In production you must answer: is it alive, is it healthy, how fast is it, what did it log, and how do I package and start it identically everywhere? Spring Boot gives you most of the answers with Actuator, Micrometer and good packaging support. This chapter covers them, then a Dockerfile, a `docker-compose.yml` with PostgreSQL, and a deployment checklist.

4.2.1 Actuator

Spring Boot Actuator adds production-ready endpoints that expose the running application's state over HTTP (and JMX). Add the starter:

pom.xml (fragment) XML

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

By default over HTTP only `/actuator/health` is exposed. The most useful endpoints are:

Endpoint	What it shows
`health`	Up or down, plus details of database, disk and other checks
`info`	Build and git information, plus anything you add under `info.*`
`metrics`	Names of all Micrometer metrics; `/actuator/metrics/jvm.memory.used` gives one
`prometheus`	Metrics in Prometheus text format (needs the Prometheus registry dependency)
`loggers`	Read and change log levels at runtime
`env`	All configuration properties and their sources (sensitive)
`beans, mappings, conditions`	What is in the container, which URLs exist, why auto-configuration did or did not apply
`heapdump, threaddump`	A JVM heap or thread dump (very sensitive)

Common Actuator endpoints.

Exposing endpoints safely

Endpoints can be enabled and exposed separately. Exposure over the web is controlled with an include list:

application.yml YAML

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: when-authorized
      probes:
        enabled: true
  server:
    port: 9090
```

WATCH OUT

Never expose all endpoints on a public port. Setting `management.endpoints.web.exposure.include: "*"` is a favourite in tutorials. In production it publishes `env`, `heapdump` and `loggers` (which can change log levels) to anyone who can reach the port. Expose the minimum, secure the rest with Spring Security, and consider a separate `management.server.port` that is only reachable from inside your network. Heap dumps can contain passwords and tokens.

Note that `env` and `configprops` mask values of keys that look like secrets, but masking is a heuristic. Treat it as a safety net, not as permission.

Health and Kubernetes-style probes

Health is built from `HealthIndicator` beans that Boot registers automatically when the relevant class is present: a database check if there is a `DataSource`, disk space, and so on. With `management.endpoint.health.probes.enabled=true` (switched on automatically when running on Kubernetes) you get two groups:

- `/actuator/health/liveness`: is the process in a state it cannot recover from? If not, the orchestrator restarts it.
- `/actuator/health/readiness`: is it ready to take traffic? During start-up and graceful shutdown it reports out of service so the load balancer stops sending requests.

Keep liveness cheap and independent of external systems. If liveness checks the database and the database blips, the orchestrator restarts every healthy instance and makes the outage worse.

You can add your own check by implementing `HealthIndicator`:

LicenceHealthIndicator.java JAVA

```java
package com.example.shop.ops;

import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.stereotype.Component;

@Component
public class LicenceHealthIndicator implements HealthIndicator {

    @Override
    public Health health() {
        boolean valid = checkLicence();
        return valid
            ? Health.up().withDetail("licence", "valid").build()
            : Health.down().withDetail("licence", "expired").build();
    }

    private boolean checkLicence() {
        return true;   // call something real here
    }
}
```

The component name `licence` (from the class name minus `HealthIndicator`) becomes the key in the health response.

Typical output TERMINAL

```
$ curl -s localhost:9090/actuator/health
{"status":"UP","groups":["liveness","readiness"]}
```

> **NOTE**
> In Spring Boot 4 the Actuator is split into smaller modules and some package names moved, but the starter, the properties and the endpoint URLs above stay the same. Check the migration guide when an import does not resolve.

4.2.2 Metrics with Micrometer

Micrometer is the metrics facade Actuator uses, in the way SLF4J is a facade for logging. You record measurements in vendor-neutral terms and a registry ships them to Prometheus, Datadog, CloudWatch and others. Boot instruments the JVM (memory, GC, threads), the web server (`http.server.requests`, a timer by URI, method and status), the datasource pool and more, without any code from you.

Metric types you meet:

- **Counter**: only goes up (orders placed).
- **Gauge**: a value that goes up and down (queue size).
- **Timer**: counts events and records their duration (request latency).

To scrape with Prometheus, add `io.micrometer:micrometer-registry-prometheus` and expose the `prometheus` endpoint. Your own metric is a few lines:

CheckoutService.java JAVA

```java
package com.example.shop.ops;

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.springframework.stereotype.Service;

@Service
public class CheckoutService {

    private final Counter orders;

    public CheckoutService(MeterRegistry registry) {
        this.orders = Counter.builder("shop.orders.placed")
            .description("Orders successfully placed")
            .register(registry);
    }

    public void placeOrder() {
        // business logic
        orders.increment();
    }
}
```

Keep tag values low-cardinality. A tag such as `userId` creates one time series per user and can overwhelm your monitoring system.

4.2.3 Logging

Boot uses **SLF4J** as the API in your code and **Logback** as the default implementation. Create a logger per class and log with placeholders, which avoids building strings for log levels that are switched off:

ReportJob.java JAVA

```java
package com.example.shop.ops;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Component
public class ReportJob {

    private static final Logger log = LoggerFactory.getLogger(ReportJob.class);

    public void run(String name, int rows) {
        log.info("Report {} produced {} rows", name, rows);
        try {
            export(name);
        } catch (IllegalStateException e) {
            log.error("Export of report {} failed", name, e);   // exception last: full stack trace
        }
    }

    private void export(String name) {
        if (name.isBlank()) {
            throw new IllegalStateException("empty name");
        }
    }
}
```

Levels, from most to least severe: `ERROR`, `WARN`, `INFO` (the default), `DEBUG`, `TRACE`. Set them per package in configuration, and change them at runtime through the `loggers` endpoint when you need to investigate:

application.yml YAML

```yaml
logging:
  level:
    root: INFO
    com.example.shop: DEBUG
    org.hibernate.SQL: DEBUG
```

Logging rules that matter

- Do not log passwords, tokens, card numbers or full request bodies.
- Pass the exception as the last argument so you get the stack trace; do not log only `e.getMessage()`.
- In containers, log to the console (stdout) and let the platform collect it. Do not write log files inside a container.
- Use JSON in production so log systems can index fields. Since Spring Boot 3.4 this is built in: `logging.structured.format.console=ecs` (other values include `logstash` and `gelf`).
- A **correlation id** put in the MDC (`MDC.put("requestId", id)` in a filter) lets you follow one request across log lines. With Micrometer Tracing the trace and span ids are added for you.

4.2.4 Graceful shutdown

When an orchestrator stops your container it sends `SIGTERM`. Without graceful shutdown, in-flight requests are cut off. With it, the server stops accepting new connections, finishes active requests, then closes the application context.

application.yml — YAML

```yaml
server:
  shutdown: graceful
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s
```

Setting it explicitly is good practice even when your Boot version already defaults to it. Make the timeout shorter than the platform's kill grace period (Kubernetes defaults to 30 seconds), and make sure your container starts the JVM directly (not through a shell script that swallows the signal).

4.2.5 Packaging: the executable jar

`./mvnw package` runs your tests and produces `target/app-0.0.1-SNAPSHOT.jar`, an executable "fat" jar containing your classes, all dependencies and a launcher. Run it with `java -jar`. That jar is the unit of deployment: build it once and run the same artefact in every environment, changing only configuration.

Build and run the jar — TERMINAL

```
$ ./mvnw -q package -DskipTests
$ java -jar target/library-api-0.0.1-SNAPSHOT.jar --spring.profiles.active=prod
```

4.2.6 Docker

A **container image** packages the JRE, your jar and its start command. Two things make good Spring Boot images: a small base image, and layers ordered so that the rarely changing parts (dependencies) are cached and only your code layer is rebuilt.

Boot supports this with a layered jar: the jar's content is split into `dependencies`, `spring-boot-loader`, `snapshot-dependencies` and `application`. A multi-stage Dockerfile extracts the layers:

Dockerfile — DOCKERFILE

```dockerfile
FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN ./mvnw -q dependency:go-offline
COPY src/ src/
RUN ./mvnw -q package -DskipTests
RUN java -Djarmode=tools -jar target/*.jar extract --layers --launcher --destination /layers

FROM eclipse-temurin:21-jre
RUN useradd --system --create-home appuser
USER appuser
WORKDIR /app
COPY --from=build /layers/dependencies/ ./
COPY --from=build /layers/spring-boot-loader/ ./
COPY --from=build /layers/snapshot-dependencies/ ./
COPY --from=build /layers/application/ ./
EXPOSE 8080
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

What to notice

- Stage one builds with the JDK; stage two ships only the JRE. The final image has no Maven, no source code.
- `COPY pom.xml` plus `dependency:go-offline` before copying `src/` means dependency downloads are cached until the pom changes.
- The process runs as a non-root user.
- `extract --layers` with `-Djarmode=tools` is the current mechanism (Spring Boot 3.3 and later). Older tutorials use `-Djarmode=layertools ... extract`, and the launcher class was `org.springframework.boot.loader.JarLauncher` before Boot 3.2. If your Boot version is older than shown here, check the reference documentation for its exact command.
- Simpler alternative: `COPY target/app.jar /app/app.jar` and `ENTRYPOINT ["java","-jar","/app/app.jar"]`. It works but rebuilds the whole dependency layer on every code change.

Container-awareness: modern JVMs read the container memory limit. Rather than hard-coding `-Xmx`, set `-XX:MaxRAMPercentage=75` through the `JAVA_TOOL_OPTIONS` environment variable, leaving headroom for metaspace, threads and native memory.

Buildpacks: no Dockerfile at all

Spring Boot's Maven plugin can build an OCI image using Cloud Native Buildpacks:

Build an image with buildpacks TERMINAL

```
$ ./mvnw spring-boot:build-image -Dspring-boot.build-image.imageName=library-api:1.0
```

It needs a running Docker daemon, produces a layered, memory-tuned image, and gets security patches for the base layers by rebuilding. It is quick to adopt; a hand-written Dockerfile gives you more control. Both are legitimate.

4.2.7 Configuration in containers

Do not bake environment-specific values into the image. Spring Boot's relaxed binding maps environment variables to properties: `spring.datasource.url` is `SPRING_DATASOURCE_URL`, `server.port` is `SERVER_PORT`, and `library.jwt.secret` is `LIBRARY_JWT_SECRET`. Dots become underscores and everything is upper case. This lets you configure the same image per environment from the outside, as the earlier configuration chapter's precedence list explains (environment variables beat the packaged `application.yml`).

4.2.8 docker-compose with PostgreSQL

Compose runs several containers together. This file starts PostgreSQL and the application, and waits for the database to be healthy:

docker-compose.yml YAML

```yaml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: shop
      POSTGRES_PASSWORD: ${DB_PASSWORD:-changeme}
    volumes:
      - dbdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shop -d shop"]
      interval: 5s
      timeout: 3s
      retries: 10

  app:
    build: .
    depends_on:
      db:
        condition: service_healthy
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/shop
      SPRING_DATASOURCE_USERNAME: shop
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD:-changeme}
      SPRING_JPA_HIBERNATE_DDL_AUTO: validate
    ports:
```

docker-compose.yml (continued) YAML

```
      - "8080:8080"

volumes:
  dbdata:
```

What to notice

- Inside the Compose network the app reaches the database by its service name, `db`, not `localhost`. Mixing these up is the number one Compose error.
- `depends_on` with `service_healthy` waits for PostgreSQL to accept connections, not merely for its container to start.
- The password comes from the environment (`DB_PASSWORD`), with a development default only. Put real secrets in a secret manager or an untracked `.env` file.
- `ddl-auto: validate` makes Hibernate check the schema instead of changing it. Create the schema with Flyway or Liquibase migrations (see the Spring Data JPA chapter).
- Start with `docker compose up --build`.

4.2.9 Deployment checklist

Before you call it production-ready, walk through this list:

- Secrets come from the environment or a secret store; none are in the repository or the image.
- Profile `prod` is active; `ddl-auto` is `validate` or `none`; schema is managed by migrations.
- Actuator exposes only what you need, secured or on a private port; health probes are configured.
- Security: HTTPS terminated at the proxy or load balancer, CORS restricted to known origins, default user gone.
- Logs go to stdout, in JSON, with no sensitive data; error responses do not include stack traces.
- Graceful shutdown is on; container runs as non-root; memory settings suit the container limit.
- Connection pool size (`spring.datasource.hikari.maximum-pool-size`) matches what the database can handle across all instances.
- The jar was built by CI from a tagged commit, and the same image moves from test to production.
- You have verified a rollback path and a backup/restore for the database.

REAL WORLD
Teams commonly run this pipeline: push to Git, CI runs `./mvnw verify` (unit and integration tests, often with Testcontainers), builds an image, pushes it to a registry, and the platform (Kubernetes, ECS, a PaaS) rolls it out with readiness probes gating traffic. The pieces you learned here, health probes, graceful shutdown, environment-driven config, are what make that rollout safe.

INTERVIEW ANGLE
What is the difference between liveness and readiness probes, and why does it matter? Liveness answers "should this process be restarted?"; readiness answers "should it receive traffic right now?". A pod that is warming up or temporarily cannot reach a dependency should fail readiness (traffic is withheld) but pass liveness (it is not killed). Mixing them up causes restart storms.

Check yourself

1. Which single property turns on exposure of `metrics` and `prometheus` over HTTP?
2. What is wrong with `log.error("failed: " + e.getMessage())` compared with `log.error("failed", e)`?
3. In compose, the app uses `jdbc:postgresql://localhost:5432/shop` and cannot connect. Why?
4. How would you set `server.port` to 9000 in a container without rebuilding the image?
5. Why is it a bad idea to include a database check in the liveness probe?

Answers

1. `management.endpoints.web.exposure.include` (for example `health,info,metrics,prometheus`).
2. The first version loses the stack trace and builds the string even when the level is disabled. The second logs the full exception.
3. Inside the app container `localhost` is the app container itself. Use the service name `db`.

4. Set the environment variable `SERVER_PORT=9000` (or `--server.port=9000` as a command-line argument).
5. A short database outage would make the orchestrator restart healthy instances, turning a small blip into a bigger outage. Put dependency checks in readiness.

KEY POINTS

Key points

- Actuator gives health, metrics and more; expose the minimum and never expose everything publicly.
- Use liveness for "restart me" and readiness for "send me traffic"; keep liveness cheap.
- Micrometer is the metrics facade; keep tag cardinality low.
- Log with SLF4J placeholders to stdout, in JSON in production, with the exception as the last argument.
- Build one layered jar or image, enable graceful shutdown, and inject all environment-specific configuration through environment variables.
- In compose, services reach each other by service name; wait for health, not just start.

PART 5

Project: Library API

Everything from the earlier Parts in one small, complete service: REST, validation, JPA, security, tests and Docker.

In this part

- **5.1** Plan and scaffold
- **5.2** Domain, repository and service
- **5.3** REST layer with validation and error handling
- **5.4** Security and tests
- **5.5** Run it and call it

5.1 Plan and scaffold

Reading about Spring is not the same as building with it. In this Part you build a small but realistic service, **Library API**, in which librarians manage authors and books and members borrow and return copies. It is deliberately small enough to type in an afternoon, yet it uses the pieces a hiring manager expects to see: layered architecture, DTO records, Bean Validation, Spring Data JPA with relationships, a transactional business rule, `ProblemDetail` errors, role-based security, tests and a Docker Compose run.

> **NOTE**
> The listings in this Part are written to compile against Spring Boot 3.5.x and to carry over to Boot 4 with the small changes noted in the text. They could not be executed while this guide was written, so type them in, run `./mvnw verify`, and fix typos as you go; the structure is what matters. Any output shown is typical, not captured.

5.1.1 Requirements

- Librarians create and edit authors and books and see all loans.
- Anyone can browse authors and books (search by title, paged).
- A member who is logged in can borrow a book if a copy is available and can list their own loans.
- A librarian records the return of a loan.
- A book that has loans cannot be deleted; a book with no free copy cannot be borrowed; the ISBN is unique.

5.1.2 Endpoints

Method	Path	Who	Result
GET	/api/authors, /api/authors/{id}	anyone	200 list or one
POST	/api/authors	LIBRARIAN	201 with Location
GET	/api/books?title=&page=&size=	anyone	200 page of books
GET	/api/books/{id}	anyone	200 or 404
POST	/api/books	LIBRARIAN	201, 404 unknown author, 409 duplicate ISBN
PUT	/api/books/{id}	LIBRARIAN	200, 404, 409
DELETE	/api/books/{id}	LIBRARIAN	204, 404, 409 if loans exist
POST	/api/loans	MEMBER or LIBRARIAN	201, 404, 409 no copy
GET	/api/loans/mine	authenticated	200 own loans
GET	/api/loans	LIBRARIAN	200 all loans
POST	/api/loans/{id}/return	LIBRARIAN	200, 404, 409 already returned

The API surface.

5.1.3 Package layout

Package by layer, which is the simplest to follow at this size (larger systems often package by feature instead).

Project layout

```
library-api/
  pom.xml  Dockerfile  docker-compose.yml
  src/main/java/com/example/library/
    LibraryApplication.java
    domain/     Author  Book  Loan
    repository/ AuthorRepository  BookRepository  LoanRepository
    service/    AuthorService  BookService  LoanService
    web/        AuthorController  BookController  LoanController
                dto/ (records)   ApiExceptionHandler
    error/      NotFoundException  ConflictException
    config/     SecurityConfig  SecurityProps
  src/main/resources/
    application.yml
    db/migration/V1__init.sql
  src/test/java/com/example/library/
    web/BookControllerTest   service/LoanServiceTest
```

The dependency rule is one-directional: `web` calls `service`, `service` calls `repository`, and nothing calls upwards. Entities stay inside `domain` and `service`; controllers only see DTO records.

5.1.4 The pom

pom.xml XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.6</version>   <!-- use the latest 3.5.x from start.spring.io -->
    <relativePath/>
  </parent>

  <groupId>com.example</groupId>
  <artifactId>library-api</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <properties>
    <java.version>21</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
    <dependency>
      <groupId>org.flywaydb</groupId>
      <artifactId>flyway-core</artifactId>
    </dependency>
    <dependency>
      <groupId>org.flywaydb</groupId>
      <artifactId>flyway-database-postgresql</artifactId>
    </dependency>
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <scope>runtime</scope>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.security</groupId>
      <artifactId>spring-security-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
```

pom.xml (continued) XML

```
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

The parent's dependency management supplies the version of every dependency above, so no `<version>` tags appear. There is no H2 dependency on purpose: repository-level tests use the same PostgreSQL via Testcontainers in a real project (see the testing chapter), and here we keep the test suite to unit and web-slice tests that need no database.

NOTE
Spring Boot 4 split many starters. The web starter is now `spring-boot-starter-webmvc`, Flyway needs `spring-boot-starter-flyway`, and each technology has a matching `-test` starter. The simplest way to port: generate a project at start.spring.io with the same choices on the Boot 4 line and copy its dependency list. Application code in this Part is unaffected, apart from import packages for test-slice annotations.

5.1.5 Configuration and schema

application.yml YAML

```yaml
spring:
  application:
    name: library-api
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/library}
    username: ${DB_USER:library}
    password: ${DB_PASSWORD:library}
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
  flyway:
    enabled: true

server:
  shutdown: graceful

management:
  endpoints:
    web:
      exposure:
        include: health,info
  endpoint:
    health:
      probes:
        enabled: true

library:
  security:
    librarian-password: ${LIBRARIAN_PASSWORD:librarian-dev}
    member-password: ${MEMBER_PASSWORD:member-dev}
```

`${NAME:default}` reads an environment variable with a fallback, so local runs work with no setup while Compose or production override the values from outside. Flyway runs the SQL migration at start-up and Hibernate only validates that the entities match it:

V1__init.sql SQL

```sql
create table authors (
    id   bigint generated by default as identity primary key,
    name varchar(200) not null
);

create table books (
    id               bigint generated by default as identity primary key,
    title            varchar(300) not null,
    isbn             varchar(20)  not null unique,
    author_id        bigint       not null references authors (id),
    copies_total     int          not null check (copies_total >= 1),
    copies_available int          not null check (copies_available >= 0),
```

V1__init.sql (continued) SQL

```
    version          bigint       not null default 0
);

create table loans (
    id          bigint generated by default as identity primary key,
    book_id     bigint       not null references books (id),
    borrower    varchar(100) not null,
    loaned_at   date         not null,
    due_date    date         not null,
    returned_at date
);

create index idx_loans_borrower on loans (borrower);
create index idx_loans_book on loans (book_id);
```

Save it as `src/main/resources/db/migration/V1__init.sql`. The file name matters: `V`, version, two underscores, description. Never edit a migration that has run anywhere; add `V2__...` instead.

Check yourself

1. Why is `spring.jpa.open-in-view` set to `false`, and what does that force you to do?
2. What does `${DB_URL:jdbc:postgresql://localhost:5432/library}` do?
3. You need a new column next week. Do you edit `V1__init.sql`?
4. Why does the layout forbid controllers from using entities?

Answers

1. It stops the persistence session from staying open during view rendering, which avoids hidden lazy queries and long connection use. You must finish loading data (and map to DTOs) inside the service transaction.
2. It uses the environment variable `DB_URL` if set, otherwise the literal default after the colon.
3. No. Flyway checksums applied migrations and refuses to start if one changed. Add `V2__add_column.sql`.
4. Entities are persistence details with lazy relationships; exposing them couples the API to the schema and risks lazy-loading errors and accidental data leaks.

KEY POINTS

Key points

- Write requirements and an endpoint table first; it becomes your test checklist.
- Package by layer here, with one-way dependencies from web to service to repository.
- Flyway owns the schema, Hibernate only validates it, and configuration comes from environment variables with dev defaults.
- Never edit an applied migration.

5.2 Domain, repository and service

This chapter builds everything below the web layer: three entities, three repositories, the services and the error types.

5.2.1 Errors first

Two small exceptions express the business outcomes the web layer will translate to 404 and 409:

NotFoundException.java JAVA

```
package com.example.library.error;

public class NotFoundException extends RuntimeException {

    public NotFoundException(String what, Object id) {
        super(what + " " + id + " not found");
    }
}
```

ConflictException.java JAVA

```java
package com.example.library.error;

public class ConflictException extends RuntimeException {

    public ConflictException(String message) {
        super(message);
    }
}
```

Both are unchecked, which matters for `@Transactional`: a `RuntimeException` leaving a transactional method rolls the transaction back by default.

5.2.2 Entities

Author.java JAVA

```java
package com.example.library.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "authors")
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    protected Author() {
        // required by JPA
    }

    public Author(String name) {
        this.name = name;
    }

    public Long getId() { return id; }

    public String getName() { return name; }
}
```

Book.java JAVA

```java
package com.example.library.domain;

import com.example.library.error.ConflictException;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import jakarta.persistence.Version;

@Entity
@Table(name = "books")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String isbn;
```

Book.java (continued) JAVA

```java
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "author_id")
    private Author author;

    private int copiesTotal;
    private int copiesAvailable;

    @Version
    private long version;

    protected Book() {
        // required by JPA
    }

    public Book(String title, String isbn, Author author, int copies) {
        this.title = title;
        this.isbn = isbn;
        this.author = author;
        this.copiesTotal = copies;
        this.copiesAvailable = copies;
    }

    public void update(String title, String isbn, Author author) {
        this.title = title;
        this.isbn = isbn;
        this.author = author;
    }

    public void checkOut() {
        if (copiesAvailable == 0) {
            throw new ConflictException("No copy of '" + title + "' is available");
        }
        copiesAvailable--;
    }

    public void checkIn() {
        if (copiesAvailable >= copiesTotal) {
            throw new ConflictException("All copies of '" + title + "' are already in");
        }
        copiesAvailable++;
    }

    public Long getId() { return id; }
    public String getTitle() { return title; }
    public String getIsbn() { return isbn; }
    public Author getAuthor() { return author; }
    public int getCopiesTotal() { return copiesTotal; }
    public int getCopiesAvailable() { return copiesAvailable; }
}
```

Loan.java JAVA

```java
package com.example.library.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.time.LocalDate;

@Entity
@Table(name = "loans")
public class Loan {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "book_id")
```

Loan.java (continued) JAVA

```
    private Book book;

    private String borrower;
    private LocalDate loanedAt;
    private LocalDate dueDate;
    private LocalDate returnedAt;

    protected Loan() {
        // required by JPA
    }

    public Loan(Book book, String borrower, LocalDate loanedAt, LocalDate dueDate) {
        this.book = book;
        this.borrower = borrower;
        this.loanedAt = loanedAt;
        this.dueDate = dueDate;
    }

    public boolean isReturned() { return returnedAt != null; }

    public void markReturned(LocalDate day) { this.returnedAt = day; }

    public Long getId() { return id; }
    public Book getBook() { return book; }
    public String getBorrower() { return borrower; }
    public LocalDate getLoanedAt() { return loanedAt; }
    public LocalDate getDueDate() { return dueDate; }
    public LocalDate getReturnedAt() { return returnedAt; }
}
```

What to notice

- Fields have no setters; state changes go through behaviour (`checkOut`, `checkIn`, `markReturned`). The rule "you cannot borrow when no copy is free" lives in the entity, so it cannot be bypassed by a careless caller.
- Every association is `LAZY`. `@ManyToOne` defaults to `EAGER`, which is the usual cause of N+1 surprises, so we state it explicitly.
- `@Version` turns on optimistic locking. Two members borrowing the last copy at the same moment both read `copiesAvailable = 1`; the second `UPDATE` finds the version changed and fails with `ObjectOptimisticLockingFailureException`, which we map to 409.
- Camel-case fields map to snake-case columns (`copiesTotal` to `copies_total`) through Spring Boot's default naming strategy.

5.2.3 Repositories

BookRepository.java JAVA

```
package com.example.library.repository;

import com.example.library.domain.Book;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;

public interface BookRepository extends JpaRepository<Book, Long> {

    @EntityGraph(attributePaths = "author")
    Page<Book> findByTitleContainingIgnoreCase(String title, Pageable pageable);

    boolean existsByIsbn(String isbn);
}
```

LoanRepository.java JAVA

```
package com.example.library.repository;

import com.example.library.domain.Loan;
import java.util.List;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;
```

LoanRepository.java (continued) JAVA

```java
public interface LoanRepository extends JpaRepository<Loan, Long> {

    @EntityGraph(attributePaths = {"book", "book.author"})
    List<Loan> findByBorrowerOrderByLoanedAtDesc(String borrower);

    @EntityGraph(attributePaths = {"book", "book.author"})
    List<Loan> findAllByOrderByLoanedAtDesc();

    boolean existsByBookId(Long bookId);
}
```

`AuthorRepository` is a one-liner, `public interface AuthorRepository extends JpaRepository<Author, Long> {}`, in the same package. The `@EntityGraph` annotations tell Spring Data to join-fetch the author (and book) in the same query that loads the page. Without them, mapping 20 books to DTOs would trigger one extra query per book for `getAuthor().getName()`: the N+1 problem from the relationships chapter.

5.2.4 Request and response records

DTOs are records. Validation annotations on the request records are what `@Valid` in the controller will check:

BookRequest.java JAVA

```java
package com.example.library.web.dto;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

public record BookRequest(
        @NotBlank @Size(max = 300) String title,
        @NotBlank @Pattern(regexp = "[0-9-]{10,17}", message = "must be a valid ISBN") String isbn,
        @NotNull Long authorId,
        @Min(1) int copies) {
}
```

BookResponse.java JAVA

```java
package com.example.library.web.dto;

import com.example.library.domain.Book;

public record BookResponse(Long id, String title, String isbn, Long authorId,
                           String authorName, int copiesAvailable, int copiesTotal) {

    public static BookResponse from(Book b) {
        return new BookResponse(b.getId(), b.getTitle(), b.getIsbn(), b.getAuthor().getId(),
                b.getAuthor().getName(), b.getCopiesAvailable(), b.getCopiesTotal());
    }
}
```

The other DTOs follow the same pattern and are one line each (shown together here; put each record in its own file):

OtherDtos.java JAVA

```java
package com.example.library.web.dto;

import com.example.library.domain.Author;
import com.example.library.domain.Loan;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import java.time.LocalDate;
import java.util.List;
import org.springframework.data.domain.Page;

public class OtherDtos {

    public record AuthorRequest(@NotBlank @Size(max = 200) String name) { }

    public record AuthorResponse(Long id, String name) {
```

OtherDtos.java (continued) JAVA

```
        public static AuthorResponse from(Author a) {
            return new AuthorResponse(a.getId(), a.getName());
        }
    }

    public record LoanRequest(@NotNull Long bookId) { }

    public record LoanResponse(Long id, Long bookId, String bookTitle, String borrower,
                               LocalDate loanedAt, LocalDate dueDate, LocalDate returnedAt) {
        public static LoanResponse from(Loan l) {
            return new LoanResponse(l.getId(), l.getBook().getId(), l.getBook().getTitle(),
                    l.getBorrower(), l.getLoanedAt(), l.getDueDate(), l.getReturnedAt());
        }
    }

    public record PageResponse<T>(List<T> content, int page, int size, long totalElements) {
        public static <T> PageResponse<T> of(Page<T> p) {
            return new PageResponse<>(p.getContent(), p.getNumber(), p.getSize(),
                    p.getTotalElements());
        }
    }
}
```

From here on we use them as nested types of `OtherDtos` (`OtherDtos.LoanRequest`); in your own project, separate files are tidier. `PageResponse` exists because serialising Spring Data's `PageImpl` directly produces an unstable JSON shape that Spring warns against.

5.2.5 Services

BookService.java JAVA

```java
package com.example.library.service;

import com.example.library.domain.Author;
import com.example.library.domain.Book;
import com.example.library.error.ConflictException;
import com.example.library.error.NotFoundException;
import com.example.library.repository.AuthorRepository;
import com.example.library.repository.BookRepository;
import com.example.library.repository.LoanRepository;
import com.example.library.web.dto.BookRequest;
import com.example.library.web.dto.BookResponse;
import com.example.library.web.dto.OtherDtos.PageResponse;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@Transactional(readOnly = true)
public class BookService {

    private final BookRepository books;
    private final AuthorRepository authors;
    private final LoanRepository loans;

    public BookService(BookRepository books, AuthorRepository authors, LoanRepository loans) {
        this.books = books;
        this.authors = authors;
        this.loans = loans;
    }

    public PageResponse<BookResponse> search(String title, Pageable pageable) {
        String q = title == null ? "" : title;
        return PageResponse.of(books.findByTitleContainingIgnoreCase(q, pageable)
                .map(BookResponse::from));
    }

    public BookResponse get(long id) {
        return BookResponse.from(find(id));
    }

    @Transactional
    public BookResponse create(BookRequest req) {
        if (books.existsByIsbn(req.isbn())) {
            throw new ConflictException("ISBN " + req.isbn() + " already exists");
        }
        Book saved = books.save(new Book(req.title(), req.isbn(), author(req.authorId()),
                req.copies()));
        return BookResponse.from(saved);
    }

    @Transactional
    public BookResponse update(long id, BookRequest req) {
        Book book = find(id);
        if (!book.getIsbn().equals(req.isbn()) && books.existsByIsbn(req.isbn())) {
            throw new ConflictException("ISBN " + req.isbn() + " already exists");
        }
        book.update(req.title(), req.isbn(), author(req.authorId()));
        return BookResponse.from(book);   // dirty checking writes the UPDATE at commit
    }

    @Transactional
    public void delete(long id) {
        Book book = find(id);
        if (loans.existsByBookId(id)) {
            throw new ConflictException("Book " + id + " has loans and cannot be deleted");
        }
        books.delete(book);
    }

    private Book find(long id) {
        return books.findById(id).orElseThrow(() -> new NotFoundException("Book", id));
    }
```

BookService.java (continued) JAVA

```
    private Author author(long id) {
        return authors.findById(id).orElseThrow(() -> new NotFoundException("Author", id));
    }
}
```

Class-level `readOnly = true` is the default for the reads, and the three write methods override it with plain `@Transactional`. Note `update` never calls `save`: the entity is managed inside the transaction, so Hibernate notices the change and issues the `UPDATE` at commit (dirty checking). `PUT` here updates title, ISBN and author; copy counts change only through borrowing and returning.

`AuthorService` is the same shape (`list`, `get`, `create`) and is left as an exercise; its controller appears in the next chapter. The service that carries the real business rule is next:

LoanService.java JAVA

```
package com.example.library.service;

import com.example.library.domain.Book;
import com.example.library.domain.Loan;
import com.example.library.error.ConflictException;
import com.example.library.error.NotFoundException;
import com.example.library.repository.BookRepository;
import com.example.library.repository.LoanRepository;
import com.example.library.web.dto.OtherDtos.LoanResponse;
import java.time.Clock;
import java.time.LocalDate;
import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@Transactional(readOnly = true)
public class LoanService {

    static final int LOAN_DAYS = 14;

    private final LoanRepository loans;
    private final BookRepository books;
    private final Clock clock;

    public LoanService(LoanRepository loans, BookRepository books, Clock clock) {
        this.loans = loans;
        this.books = books;
        this.clock = clock;
    }

    @Transactional
    public LoanResponse borrow(long bookId, String borrower) {
        Book book = books.findById(bookId)
                .orElseThrow(() -> new NotFoundException("Book", bookId));
        book.checkOut();                                  // throws ConflictException if none left
        LocalDate today = LocalDate.now(clock);
        Loan loan = loans.save(new Loan(book, borrower, today, today.plusDays(LOAN_DAYS)));
        return LoanResponse.from(loan);
    }

    @Transactional
    public LoanResponse giveBack(long loanId) {
        Loan loan = loans.findById(loanId)
                .orElseThrow(() -> new NotFoundException("Loan", loanId));
        if (loan.isReturned()) {
            throw new ConflictException("Loan " + loanId + " was already returned");
        }
        loan.markReturned(LocalDate.now(clock));
        loan.getBook().checkIn();
        return LoanResponse.from(loan);
    }

    public List<LoanResponse> mine(String borrower) {
        return loans.findByBorrowerOrderByLoanedAtDesc(borrower).stream()
                .map(LoanResponse::from).toList();
    }
```

LoanService.java (continued) JAVA

```
    public List<LoanResponse> all() {
        return loans.findAllByOrderByLoanedAtDesc().stream().map(LoanResponse::from).toList();
    }
}
```

What to notice

- `borrow` changes two rows (the book's counter and a new loan) in one transaction: both happen or neither does. If `checkOut` throws, nothing is saved.
- `Clock` is injected, not `LocalDate.now()` called blindly. Production gets the system clock; tests pass `Clock.fixed(...)` and assert exact due dates.
- The borrower name is a parameter, not read from a security context inside the service. The controller will pass the authenticated user's name; the service stays free of web and security types and is easy to unit test.
- Provide the clock bean once, in the main class:

LibraryApplication.java JAVA

```
package com.example.library;

import java.time.Clock;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class LibraryApplication {

    public static void main(String[] args) {
        SpringApplication.run(LibraryApplication.class, args);
    }

    @Bean
    Clock clock() {
        return Clock.systemDefaultZone();
    }
}
```

INTERVIEW ANGLE

How do you prevent two users from borrowing the last copy at the same time? Add optimistic locking with a `@Version` field. Both transactions read the same version; the first commit increments it, and the second commit's `UPDATE ... WHERE version = ?` matches zero rows, so Hibernate throws an optimistic-locking exception, which the API maps to 409 and the client can retry. The alternative is pessimistic locking (`SELECT ... FOR UPDATE`), which blocks instead of failing and costs throughput under contention.

Check yourself

1. In `update`, there is no `books.save(book)`. Why does the change still reach the database?
2. What would break if `LoanService.borrow` caught the `ConflictException` and returned `null`?
3. Why are entity setters avoided here?
4. The read-only method `mine` calls `this.borrow(...)`, which has its own read-write `@Transactional`. Which settings apply to `borrow`?
5. What problem do the `@EntityGraph` annotations prevent?

Answers

1. The entity is managed in an open transaction, so Hibernate dirty-checks it and flushes an `UPDATE` at commit.
2. The transaction would commit with the decremented counter but no loan, silently corrupting the data. Throwing an unchecked exception triggers rollback.
3. Changes go through intention-revealing methods that enforce rules (`checkOut`), so invalid states are hard to create.
4. The caller's read-only transaction. Self-invocation bypasses the Spring proxy, so the annotation on `borrow` is ignored for that call.
5. N+1: one query for the list plus one per row to lazy-load the author or book.

KEY POINTS

Key points

- Put business rules in the domain and service layer, behind small, behaviour-named methods.
- Mark associations `LAZY` explicitly and fetch what you need with `@EntityGraph`.
- `@Transactional` on the service defines the unit of work; unchecked exceptions roll it back.
- `@Version` gives cheap protection against lost updates.
- Inject `Clock` for time-dependent logic so tests are deterministic.

5.3 REST layer with validation and error handling

The web layer should be thin: translate HTTP into a service call, and the result back into HTTP. No business rules, no entities, no repository calls.

5.3.1 Controllers

BookController.java JAVA

```java
package com.example.library.web;

import com.example.library.service.BookService;
import com.example.library.web.dto.BookRequest;
import com.example.library.web.dto.BookResponse;
import com.example.library.web.dto.OtherDtos.PageResponse;
import jakarta.validation.Valid;
import java.net.URI;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookService service;

    public BookController(BookService service) {
        this.service = service;
    }

    @GetMapping
    public PageResponse<BookResponse> search(
            @RequestParam(required = false) String title,
            @PageableDefault(size = 20, sort = "title") Pageable pageable) {
        return service.search(title, pageable);
    }

    @GetMapping("/{id}")
    public BookResponse get(@PathVariable long id) {
        return service.get(id);
    }

    @PostMapping
    public ResponseEntity<BookResponse> create(@Valid @RequestBody BookRequest request) {
        BookResponse created = service.create(request);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").buildAndExpand(created.id()).toUri();
        return ResponseEntity.created(location).body(created);
    }

    @PutMapping("/{id}")
    public BookResponse update(@PathVariable long id, @Valid @RequestBody BookRequest request) {
        return service.update(id, request);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

LoanController.java JAVA

```java
package com.example.library.web;

import com.example.library.service.LoanService;
import com.example.library.web.dto.OtherDtos.LoanRequest;
import com.example.library.web.dto.OtherDtos.LoanResponse;
import jakarta.validation.Valid;
```

LoanController.java (continued) JAVA

```java
import java.net.URI;
import java.security.Principal;
import java.util.List;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/loans")
public class LoanController {

    private final LoanService service;

    public LoanController(LoanService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<LoanResponse> borrow(@Valid @RequestBody LoanRequest request,
                                               Principal principal) {
        LoanResponse loan = service.borrow(request.bookId(), principal.getName());
        return ResponseEntity.created(URI.create("/api/loans/" + loan.id())).body(loan);
    }

    @GetMapping("/mine")
    public List<LoanResponse> mine(Principal principal) {
        return service.mine(principal.getName());
    }

    @GetMapping
    public List<LoanResponse> all() {
        return service.all();
    }

    @PostMapping("/{id}/return")
    public LoanResponse giveBack(@PathVariable long id) {
        return service.giveBack(id);
    }
}
```

`AuthorController` follows the `BookController` pattern with `GET /api/authors`, `GET /api/authors/{id}` and `POST /api/authors` over a small `AuthorService`; write it as practice. Three things to notice in the two controllers above:

- `@Valid` on the `@RequestBody` runs Bean Validation before the method body; invalid input never reaches the service.
- `Principal` is resolved by Spring MVC from the security context, so the borrower name comes from the authenticated user and cannot be spoofed in the request body. That is a security decision as much as a design one.
- `POST` returns `201 Created` with a `Location` header, `DELETE` returns `204`, and nothing returns an entity.

Set a ceiling on page size so a client cannot ask for a million rows: `spring.data.web.pageable.max-page-size: 100` in `application.yml`.

5.3.2 One place for errors

Services throw domain exceptions; one `@RestControllerAdvice` converts everything to RFC 9457 `ProblemDetail` JSON. Extending `ResponseEntityExceptionHandler` gives correct problem-detail responses for the standard Spring MVC exceptions (malformed JSON, wrong method, missing parameter) for free. We override only the validation case to add field details.

ApiExceptionHandler.java JAVA

```java
package com.example.library.web;

import com.example.library.error.ConflictException;
import com.example.library.error.NotFoundException;
import java.util.LinkedHashMap;
import java.util.Map;
import org.springframework.http.HttpHeaders;
```

ApiExceptionHandler.java (continued) JAVA

```java
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ProblemDetail;
import org.springframework.http.ResponseEntity;
import org.springframework.orm.ObjectOptimisticLockingFailureException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

@RestControllerAdvice
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(NotFoundException.class)
    ProblemDetail notFound(NotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Not found");
        return pd;
    }

    @ExceptionHandler(ConflictException.class)
    ProblemDetail conflict(ConflictException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, ex.getMessage());
        pd.setTitle("Conflict");
        return pd;
    }

    @ExceptionHandler(ObjectOptimisticLockingFailureException.class)
    ProblemDetail concurrentUpdate(ObjectOptimisticLockingFailureException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT,
                "The record was changed by someone else; please retry");
        pd.setTitle("Concurrent update");
        return pd;
    }

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex, HttpHeaders headers,
            HttpStatusCode status, WebRequest request) {
        Map<String, String> errors = new LinkedHashMap<>();
        for (FieldError fe : ex.getBindingResult().getFieldErrors()) {
            errors.putIfAbsent(fe.getField(), fe.getDefaultMessage());
        }
        ProblemDetail pd = ex.getBody();
        pd.setTitle("Validation failed");
        pd.setProperty("errors", errors);
        return handleExceptionInternal(ex, pd, headers, status, request);
    }
}
```

Typical output of a failed validation (the exact members can vary slightly by version):

Typical output TERMINAL

```
$ curl -s -u librarian:librarian-dev -H 'Content-Type: application/json' \
    -d '{"title":"","isbn":"x","authorId":1,"copies":0}' localhost:8080/api/books
{"type":"about:blank","title":"Validation failed","status":400,
 "detail":"Invalid request content.","instance":"/api/books",
 "errors":{"title":"must not be blank","isbn":"must be a valid ISBN",
 "copies":"must be greater than or equal to 1"}}
```

WATCH OUT

Do not add a catch-all `@ExceptionHandler(Exception.class)` to this advice without thought. It would also catch Spring Security's `AccessDeniedException` thrown by method security and turn a correct 403 into a 500. Leave unexpected errors to Boot's default error handling, which returns a generic 500 without a stack trace (the default for `server.error.include-stacktrace` is `never`), and log the real cause server-side.

NOTE
Security failures raised by the filter chain (401, 403 on URL rules) happen before MVC, so this advice never sees them; they come back with an empty body. That is acceptable for an API; if you want JSON bodies there, supply a custom `AuthenticationEntryPoint` and `AccessDeniedHandler`.

Check yourself

1. Which status does the API return when you `POST` a book with an ISBN that already exists, and which layer decides it?
2. Why is the borrower taken from `Principal` instead of the request body?
3. What is wrong with returning the `Book` entity from the controller (name two problems)?
4. A client sends malformed JSON. Which code path handles it in this project?

Answers

1. `409 Conflict`; `BookService.create` throws `ConflictException` and the advice maps it.
2. Anything in the body can be forged. The authenticated principal is trustworthy, so a member cannot borrow in someone else's name.
3. Lazy-loading errors outside a transaction (with open-in-view off), and coupling the public JSON contract to the database schema, which can also leak internal fields. Also risk of infinite recursion on bidirectional relations.
4. `ResponseEntityExceptionHandler` (the parent class) handles `HttpMessageNotReadableException` and returns a 400 `ProblemDetail`.

KEY POINTS
Key points

- Controllers translate: bind and validate input, call one service method, choose the status code.
- Throw domain exceptions in the service; map them centrally in one `@RestControllerAdvice` using `ProblemDetail`.
- Never take identity from the request body when the security context knows it.
- Return DTOs, `201` plus `Location` on create, `204` on delete.

5.4 Security and tests

5.4.1 Roles with HTTP Basic

Two users live in memory, and their passwords come from configuration (so they are environment variables in Compose). This is fine for a learning project; a real system would load users from a database or delegate to an identity provider, as the security chapter described.

SecurityProps.java JAVA

```java
package com.example.library.config;

import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "library.security")
public record SecurityProps(String librarianPassword, String memberPassword) {
}
```

SecurityConfig.java JAVA

```java
package com.example.library.config;

import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
```

SecurityConfig.java (continued) JAVA

```java
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
@EnableConfigurationProperties(SecurityProps.class)
public class SecurityConfig {

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    UserDetailsService users(SecurityProps props, PasswordEncoder encoder) {
        var librarian = User.withUsername("librarian")
                .password(encoder.encode(props.librarianPassword()))
                .roles("LIBRARIAN").build();
        var alice = User.withUsername("alice")
                .password(encoder.encode(props.memberPassword()))
                .roles("MEMBER").build();
        return new InMemoryUserDetailsManager(librarian, alice);
    }

    @Bean
    SecurityFilterChain chain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/books/**", "/api/authors/**").permitAll()
                .requestMatchers("/api/books/**", "/api/authors/**").hasRole("LIBRARIAN")
                .requestMatchers(HttpMethod.GET, "/api/loans").hasRole("LIBRARIAN")
                .requestMatchers("/api/loans/*/return").hasRole("LIBRARIAN")
                .anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

What to notice

- Order is everything. The public `GET` rule comes before the librarian rule for the same paths; swap them and nobody could browse anonymously. The final `anyRequest().authenticated()` covers `POST /api/loans` and `GET /api/loans/mine` for any logged-in user.
- CSRF is disabled because the API is stateless and uses a header credential, the justification from the security chapter, not a shortcut.
- A user who is logged in but lacks the role gets `403`; no credentials gives `401`.
- Passwords are encoded with BCrypt once at start-up. There is no plain-text password in the code, only configurable defaults for development.

5.4.2 Web-layer test

`@WebMvcTest` starts only the MVC slice: the controller, the advice and the security filter chain, with services replaced by mocks. It runs in well under a second per test class once the context is cached.

BookControllerTest.java JAVA

```java
package com.example.library.web;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
```

BookControllerTest.java (continued) JAVA

```java
import com.example.library.config.SecurityConfig;
import com.example.library.error.NotFoundException;
import com.example.library.service.BookService;
import com.example.library.web.dto.BookRequest;
import com.example.library.web.dto.BookResponse;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(BookController.class)
@Import(SecurityConfig.class)
class BookControllerTest {

    private static final String VALID_BOOK = """
            {"title":"Dune","isbn":"978-0441172719","authorId":7,"copies":3}
            """;

    @Autowired
    MockMvc mvc;

    @MockitoBean
    BookService service;

    @Test
    void getBookIsPublic() throws Exception {
        when(service.get(1L)).thenReturn(
                new BookResponse(1L, "Dune", "978-0441172719", 7L, "Frank Herbert", 2, 3));

        mvc.perform(get("/api/books/1"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.title").value("Dune"));
    }

    @Test
    void unknownBookIs404ProblemDetail() throws Exception {
        when(service.get(99L)).thenThrow(new NotFoundException("Book", 99L));

        mvc.perform(get("/api/books/99"))
                .andExpect(status().isNotFound())
                .andExpect(jsonPath("$.title").value("Not found"));
    }

    @Test
    void createWithoutLoginIs401() throws Exception {
        mvc.perform(post("/api/books").contentType(MediaType.APPLICATION_JSON).content(VALID_BOOK))
                .andExpect(status().isUnauthorized());
    }

    @Test
    @WithMockUser(roles = "MEMBER")
    void createAsMemberIs403() throws Exception {
        mvc.perform(post("/api/books").contentType(MediaType.APPLICATION_JSON).content(VALID_BOOK))
                .andExpect(status().isForbidden());
    }

    @Test
    @WithMockUser(roles = "LIBRARIAN")
    void createAsLibrarianReturns201WithLocation() throws Exception {
        when(service.create(new BookRequest("Dune", "978-0441172719", 7L, 3))).thenReturn(
                new BookResponse(5L, "Dune", "978-0441172719", 7L, "Frank Herbert", 3, 3));

        mvc.perform(post("/api/books").contentType(MediaType.APPLICATION_JSON).content(VALID_BOOK))
                .andExpect(status().isCreated())
                .andExpect(header().string("Location", "http://localhost/api/books/5"));
    }

    @Test
    @WithMockUser(roles = "LIBRARIAN")
    void invalidBodyIs400WithFieldErrors() throws Exception {
        String bad = "{\"title\":\"\",\"isbn\":\"x\",\"authorId\":7,\"copies\":0}";
```

BookControllerTest.java (continued) JAVA

```
        mvc.perform(post("/api/books").contentType(MediaType.APPLICATION_JSON).content(bad))
                .andExpect(status().isBadRequest())
                .andExpect(jsonPath("$.errors.title").exists())
                .andExpect(jsonPath("$.errors.copies").exists());
    }
}
```

Records give `BookRequest` a value-based `equals`, which is why `when(service.create(new BookRequest(...)))` matches the deserialised request. These six tests check routing, JSON, validation, error mapping and the security rules, all without a database.

> **NOTE**
> In Spring Boot 4 the slice annotations moved into feature-specific packages (for example `WebMvcTest` is under `org.springframework.boot.webmvc.test.autoconfigure`) and need the matching test starter. `@MockitoBean`, `MockMvc` and `@WithMockUser` are unchanged. Let your IDE fix the imports.

5.4.3 Service unit test

The business rule is tested with plain JUnit and Mockito: no Spring context at all, so it runs in milliseconds.

LoanServiceTest.java JAVA

```
package com.example.library.service;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

import com.example.library.domain.Author;
import com.example.library.domain.Book;
import com.example.library.domain.Loan;
import com.example.library.error.ConflictException;
import com.example.library.repository.BookRepository;
import com.example.library.repository.LoanRepository;
import com.example.library.web.dto.OtherDtos.LoanResponse;
import java.time.Clock;
import java.time.Instant;
import java.time.LocalDate;
import java.time.ZoneOffset;
import java.util.Optional;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

@ExtendWith(MockitoExtension.class)
class LoanServiceTest {

    @Mock
    LoanRepository loans;

    @Mock
    BookRepository books;

    LoanService service;

    @BeforeEach
    void setUp() {
        Clock clock = Clock.fixed(Instant.parse("2026-03-01T10:00:00Z"), ZoneOffset.UTC);
        service = new LoanService(loans, books, clock);
    }

    @Test
    void borrowTakesACopyAndSetsDueDate() {
        Book book = new Book("Dune", "978-0441172719", new Author("Frank Herbert"), 2);
        when(books.findById(1L)).thenReturn(Optional.of(book));
```

LoanServiceTest.java (continued) JAVA

```
        when(loans.save(any(Loan.class))).thenAnswer(inv -> inv.getArgument(0));

        LoanResponse loan = service.borrow(1L, "alice");

        assertThat(book.getCopiesAvailable()).isEqualTo(1);
        assertThat(loan.dueDate()).isEqualTo(LocalDate.of(2026, 3, 15));
        assertThat(loan.borrower()).isEqualTo("alice");
    }

    @Test
    void borrowFailsWhenNoCopyIsLeft() {
        Book book = new Book("Dune", "978-0441172719", new Author("Frank Herbert"), 1);
        book.checkOut();
        when(books.findById(1L)).thenReturn(Optional.of(book));

        assertThatThrownBy(() -> service.borrow(1L, "alice"))
                .isInstanceOf(ConflictException.class)
                .hasMessageContaining("Dune");
        verify(loans, never()).save(any());
    }
}
```

A natural third layer is a repository test with `@DataJpaTest` against PostgreSQL in Testcontainers (covered in the testing chapter): it would prove the Flyway migration and the derived queries work together. We omit it here so the suite needs no Docker daemon.

Check yourself

1. A member calls `GET /api/loans` and receives a response. What status, and which line of `SecurityConfig` decides it?
2. Why does `createAsMemberIs403` need `@WithMockUser` while `getBookIsPublic` does not?
3. What would the test `invalidBodyIs400WithFieldErrors` fail on if `ApiExceptionHandler` did not override `handleMethodArgumentNotValid`?
4. Why does `LoanServiceTest` pass a fixed `Clock`?

Answers

1. `403 Forbidden`, from `.requestMatchers(HttpMethod.GET, "/api/loans").hasRole("LIBRARIAN")` (the member is authenticated but lacks the role).
2. The first needs an authenticated user to get past authentication to authorisation; the public `GET` rule permits everyone.
3. The `$.errors.title` path: the default response has status 400 but no per-field `errors` property.
4. So "today" is fixed and the due date can be asserted exactly instead of depending on when the test runs.

KEY POINTS

Key points

- Security rules are ordered; public reads first, role rules next, `anyRequest().authenticated()` last.
- Test authorisation explicitly: 401 without login, 403 with the wrong role, success with the right one.
- `@WebMvcTest` plus `@MockitoBean` tests the HTTP contract quickly; plain Mockito tests the business rules even faster.
- Inject time (`Clock`) so tests are deterministic.

5.5 Run it and call it

5.5.1 Dockerfile and Compose

Use the multi-stage `Dockerfile` from the shipping chapter unchanged (it builds the jar inside the image). The Compose file runs PostgreSQL and the application, passing credentials as environment variables, which relaxed binding maps to the `${...}` placeholders and properties in `application.yml`:

docker-compose.yml YAML

```yaml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: library
      POSTGRES_USER: library
      POSTGRES_PASSWORD: ${DB_PASSWORD:-library}
    volumes:
      - dbdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U library -d library"]
      interval: 5s
      timeout: 3s
      retries: 10

  api:
    build: .
    depends_on:
      db:
        condition: service_healthy
    environment:
      DB_URL: jdbc:postgresql://db:5432/library
      DB_USER: library
      DB_PASSWORD: ${DB_PASSWORD:-library}
      LIBRARIAN_PASSWORD: ${LIBRARIAN_PASSWORD:-librarian-dev}
      MEMBER_PASSWORD: ${MEMBER_PASSWORD:-member-dev}
    ports:
      - "8080:8080"

volumes:
  dbdata:
```

Build and start everything TERMINAL

```
$ docker compose up --build -d
$ curl -s localhost:8080/actuator/health
{"status":"UP","groups":["liveness","readiness"]}
```

Without Docker you can run just the database (`docker compose up -d db`) and start the app from your IDE or with `./mvnw spring-boot:run`; the `localhost` defaults in `application.yml` then apply.

5.5.2 A curl walk-through

The responses below are typical output; ids and dates will differ. First, a librarian creates an author and a book:

Create an author and a book (typical output) TERMINAL

```
$ curl -s -u librarian:librarian-dev -H 'Content-Type: application/json' \
    -d '{"name":"Frank Herbert"}' localhost:8080/api/authors
{"id":1,"name":"Frank Herbert"}
$ curl -s -u librarian:librarian-dev -H 'Content-Type: application/json' \
    -d '{"title":"Dune","isbn":"978-0441172719","authorId":1,"copies":1}' \
    localhost:8080/api/books
{"id":1,"title":"Dune","isbn":"978-0441172719","authorId":1,"authorName":"Frank Herbert",
 "copiesAvailable":1,"copiesTotal":1}
```

Anyone can browse; a member borrows the only copy; a second attempt conflicts:

Browse, borrow, conflict (typical output) TERMINAL

```
$ curl -s 'localhost:8080/api/books?title=dun'
{"content":[{"id":1,"title":"Dune", ... }],"page":0,"size":20,"totalElements":1}
$ curl -s -u alice:member-dev -H 'Content-Type: application/json' \
    -d '{"bookId":1}' localhost:8080/api/loans
{"id":1,"bookId":1,"bookTitle":"Dune","borrower":"alice","loanedAt":"2026-03-01",
 "dueDate":"2026-03-15","returnedAt":null}
$ curl -s -u alice:member-dev -H 'Content-Type: application/json' \
    -d '{"bookId":1}' localhost:8080/api/loans
{"type":"about:blank","title":"Conflict","status":409,
 "detail":"No copy of 'Dune' is available","instance":"/api/loans"}
```

And the permission checks, printing only the status code:

Authorisation checks (typical output) TERMINAL

```
$ curl -s -o /dev/null -w '%{http_code}\n' -X POST localhost:8080/api/books
401
$ curl -s -o /dev/null -w '%{http_code}\n' -u alice:member-dev localhost:8080/api/loans
403
$ curl -s -u librarian:librarian-dev -X POST localhost:8080/api/loans/1/return
{"id":1,"bookId":1,"bookTitle":"Dune","borrower":"alice","loanedAt":"2026-03-01",
 "dueDate":"2026-03-15","returnedAt":"2026-03-01"}
```

If every call returns 401, check that the passwords you pass match the ones the container received, and remember that changing the environment requires recreating the container. If the application fails at start with a Flyway or schema validation error, the database volume probably holds an older schema: `docker compose down -v` removes it (and its data).

5.5.3 Extension ideas

Pick the ones that match what you want to learn.

- Replace the in-memory users with an `AppUser` table, a registration endpoint and a database-backed `UserDetailsService`.
- Switch to JWT: run Keycloak in Compose and turn the API into a resource server.
- Add a `GET /api/loans/overdue` endpoint with a JPQL query, and a scheduled job (`@Scheduled`) that logs overdue loans.
- Add a `@DataJpaTest` with Testcontainers PostgreSQL for the repositories and the Flyway migration.
- Add a Micrometer counter for loans created and expose `prometheus`.
- Add OpenAPI documentation with springdoc and a `v1` path prefix for versioning.
- Add a Redis cache for book searches and measure the difference.
- Add a members table, limits on concurrent loans per member, and a 422 response when the limit is hit.
- Deploy it to a free-tier container platform and add the URL to your CV and README.

REAL WORLD
A small, finished, tested and containerised project with a clear README (how to run, endpoint table, design decisions, how to run the tests) is worth more in an interview than a large unfinished one. Be ready to explain each choice: why DTOs, why optimistic locking, why 409 and not 400, why CSRF is disabled.

Check yourself

1. In Compose, why is the database URL `jdbc:postgresql://db:5432/library` rather than `localhost`?
2. You changed `LIBRARIAN_PASSWORD` in your shell but the running container still accepts the old one. Why?
3. A logged-in client calls `/api/book/1` and gets 404. What is the likely mistake?
4. What does `docker compose down -v` do that `down` does not?

Answers

1. Inside the API container `localhost` is the container itself; services reach each other by service name on the Compose network.
2. Environment variables are fixed when a container is created. Recreate it with `docker compose up -d --force-recreate api`.
3. The path is `/api/books/1` (plural); there is no mapping for `/api/book/1`.

4. It also deletes the named volumes, so the database starts empty next time.

KEY POINTS

Key points

- A project is finished when someone else can clone it and run it with one command.
- Docker Compose ties the app and PostgreSQL together; configuration enters through environment variables.
- Walk the API with curl, checking 401, 403, 404 and 409 as well as the happy path.
- Keep a README and a list of design decisions; they are what you talk about in interviews.

PART 6

Interview Preparation and Study Plan

Questions to practise out loud, reference tables to keep open, and a ten-day plan that ties the guide together.

In this part

- **6.1** Interview Q&A
- **6.2** Cheat Sheets
- **6.3** A Ten-Day Study Plan
- **6.4** Mixed Quiz and Next Steps

6.1 Interview Q&A

Interviewers rarely ask you to recite annotations. They ask "why" and "what happens if": why constructor injection, what happens when a transactional method calls another one on the same class, why a test passes alone and fails in a suite. This chapter collects about forty-five questions in the order interviews tend to go. Each answer is short on purpose: say it out loud in about thirty seconds, then be ready to go one level deeper. If an answer feels thin, reread the chapter named in the answer.

Answers describe Spring Boot 3.5 and the 4.x line. Where they differ, the difference is stated.

6.1.1 Core Spring: IoC and dependency injection

1. What is Inversion of Control, and what is dependency injection?

IoC means your code does not create and wire its own collaborators; a framework does it and calls your code. Dependency injection is the form of IoC Spring uses: a class declares what it needs (usually as constructor parameters) and the container supplies those objects. The payoff is loose coupling: you can swap a real `PaymentGateway` for a fake in a test without touching the class that uses it.

2. What is a bean, and what is the ApplicationContext?

A bean is simply an object whose creation, wiring and lifecycle are managed by the Spring container. The `ApplicationContext` is that container: it reads bean definitions (from component scanning, `@Bean` methods and auto-configuration), creates the beans, injects dependencies and publishes events. `BeanFactory` is the smaller base interface; in practice you always work with an `ApplicationContext`.

3. Constructor, setter or field injection: which and why?

Prefer constructor injection. It lets fields be `final`, makes required dependencies impossible to forget, works without Spring in a plain unit test (`new BookService(fakeRepo)`), and exposes circular dependencies at startup instead of hiding them. Field injection (`@Autowired` on a private field) needs reflection or the container to set up the object, so it hides how many dependencies a class has. Setter injection suits genuinely optional dependencies. A class with a single constructor needs no `@Autowired`.

4. What are the stereotype annotations?

`@Component` is the generic one. `@Service`, `@Repository` and `@Controller` (and `@RestController`) are specialisations that document the layer. Technically only a few add behaviour: `@Repository` enables exception translation to Spring's `DataAccessException` hierarchy, and `@Controller` marks a web handler. `@Service` is purely semantic.

5. Two beans match the type I inject. What happens?

Startup fails with `NoUniqueBeanDefinitionException`. Fix it with `@Primary` on the default candidate, `@Qualifier("name")` at the injection point, or by naming the parameter after the bean. If you inject `List<PaymentGateway>`, you get all of them, which is a clean way to build a strategy pattern.

6. What are the bean scopes, and what is the singleton gotcha?

`singleton` (the default: one instance per container), `prototype` (a new instance each time it is requested), and the web scopes `request`, `session` and `application`. Singleton does not mean thread-safe: one instance is shared by every request thread, so a singleton service must not keep per-request state in mutable fields. Keep beans stateless, or use immutable fields. Injecting a prototype or request-scoped bean into a singleton gives you only one instance unless you inject a scoped proxy or an `ObjectProvider`.

7. Describe the bean lifecycle.

Instantiate (call the constructor), populate dependencies, run `Aware` callbacks, `BeanPostProcessor.postProcessBeforeInitialization`, then `@PostConstruct` and `afterPropertiesSet`, then `postProcessAfterInitialization` (this is where AOP proxies are usually created), then the bean is in use. On shutdown, `@PreDestroy` and `destroy` run. Prototype beans are not destroyed by the container.

8. How does `@Transactional` actually work, and what is self-invocation?

Spring wraps the bean in a proxy during `postProcessAfterInitialization`. A call from outside goes through the proxy, which opens a transaction, calls the real method, then commits or rolls back. If method `a()` calls `this.b()` inside the same class, the call never touches the proxy, so `b()`'s `@Transactional` (or `@Async`, `@Cacheable`) is silently ignored. Fix by moving `b()` to another bean, or restructure so the annotated method is the entry point.

> **INTERVIEW ANGLE**
> "Why does my @Transactional method not roll back?" Three usual causes: the call came from the same class (proxy bypassed); the method is private or the class is final (cannot be proxied); or an exception was caught inside, or a checked exception was thrown (rollback happens by default only for `RuntimeException` and `Error`; use `rollbackFor` to change that). Naming all three shows you understand the proxy model.

9. What does `@Configuration` with `@Bean` give you that `@Component` does not?

It is how you register objects you do not own (a third-party `ObjectMapper`, a `RestClient`) or that need construction logic. In a full `@Configuration` class, Spring proxies the class so that one `@Bean` method calling another still returns the singleton instance.

6.1.2 Spring Boot and auto-configuration

10. What does Spring Boot add on top of Spring?

Four things: starters (curated dependency sets with versions managed by the parent or BOM), auto-configuration (beans created for you based on the classpath and properties), an embedded server (run a jar, no WAR deployment) and production features such as Actuator. It is opinionated defaults, not a different framework.

11. What does `@SpringBootApplication` do?

It is a shortcut for three annotations: `@SpringBootConfiguration` (a `@Configuration` class), `@EnableAutoConfiguration` (switch on auto-configuration) and `@ComponentScan` (scan the package of this class and below). That is why your main class must sit at the root of your package tree.

12. How does auto-configuration decide what to create?

Auto-configuration classes are listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` inside each starter's jar. Each class is guarded by conditions: `@ConditionalOnClass` (a library is on the classpath), `@ConditionalOnMissingBean` (back off if you defined your own), `@ConditionalOnProperty` (a switch). So if you declare a `DataSource` bean yourself, Boot's default one steps aside. Run with `--debug` to print the conditions report showing what matched and why.

13. How do you override or exclude an auto-configuration?

Define your own bean of that type (the default backs off), set the relevant property, or exclude the class: `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)` or `spring.autoconfigure.exclude=...`.

14. Where does configuration come from, and which source wins?

Properties come from many sources. From lowest to highest priority, the ones you meet daily are: defaults in code, `application.properties`/`yml` in the jar, profile-specific files (`application-prod.yml`), OS environment variables, command-line arguments (`--server.port=9090`). Higher wins. That is why a container can override a database URL with an environment variable without rebuilding the jar.

15. `@Value` or `@ConfigurationProperties`?

`@Value("${app.name}")` is fine for one or two values. `@ConfigurationProperties(prefix = "app")` binds a whole group into a typed object (a record works well), supports relaxed binding (`APP_MAX_LOANS` maps to `app.max-loans`), nesting, validation with `@Validated`, and is easier to test. Use it for anything more than a couple of keys.

16. What are profiles?

Named sets of configuration and beans, such as `dev`, `test`, `prod`. Activate with `spring.profiles.active=prod` (property, env var `SPRING_PROFILES_ACTIVE`, or `--spring.profiles.active`). Use `application-prod.yml` for property overrides and `@Profile("dev")` to include a bean only in that profile. Never put secrets in the files that go into git; read them from the environment or a secret store.

6.1.3 Spring MVC and REST

17. Walk through a request in Spring MVC.

The embedded Tomcat accepts the request and hands it to the `DispatcherServlet` (the front controller). It asks `HandlerMapping` which controller method matches the URL and verb, runs interceptors, resolves arguments (`@PathVariable`, `@RequestBody` via `HttpMessageConverter`/Jackson), invokes the method, and writes the return value back as JSON. Exceptions go to `HandlerExceptionResolver`s, which is where `@RestControllerAdvice` plugs in.

18. `@Controller` vs `@RestController`?

`@RestController` is `@Controller` plus `@ResponseBody`: return values are serialised straight to the response body instead of being treated as view names. For a JSON API, use `@RestController`.

19. `@PathVariable`, `@RequestParam`, `@RequestBody`: when do you use each?

`@PathVariable` identifies a resource (`/books/{id}`), `@RequestParam` carries optional filters, paging and sorting (`?page=0&size=20`), and `@RequestBody` carries the JSON payload of a POST/PUT/PATCH, deserialised by Jackson. Headers use `@RequestHeader`.

20. Why use DTOs instead of returning entities?

An entity mirrors your table and is managed by Hibernate; exposing it leaks columns (password hashes), couples your API to your schema, triggers lazy-loading problems during serialisation and invites mass-assignment bugs when clients send fields they should not control. A DTO, usually a `record`, is the explicit contract of the API and can change independently of the database.

21. How do you validate input and report errors?

Put Bean Validation annotations (`@NotBlank`, `@Size`, `@Email`, `@Min`) on the DTO and add `@Valid` on the `@RequestBody` parameter. A failure throws `MethodArgumentNotValidException`. A `@RestControllerAdvice` with `@ExceptionHandler` methods turns it, and your own exceptions, into a consistent response. Spring supports RFC 9457 `ProblemDetail` (`application/problem+json`) for the body. Map "not found" to 404, "duplicate" to 409, "invalid but well-formed" to 422 or 400, and never return stack traces.

22. PUT vs PATCH vs POST? What is idempotency?

An operation is idempotent if repeating it has the same effect as doing it once. GET, PUT and DELETE are idempotent; POST is not (two posts create two resources). PUT replaces the whole resource; PATCH changes part of it (and is only idempotent if you design it that way). A client that retries after a timeout is the reason this matters.

23. Which status codes do you return?

200 for success with a body, 201 plus a `Location` header on creation, 204 for success without a body (delete), 400 for malformed input, 401 not authenticated, 403 authenticated but not allowed, 404 not found, 409 conflict, 422 semantic validation failure, 500 for server bugs. The full table is in the cheat sheets.

24. How do you call another HTTP service?

Use `RestClient` (Spring 6.1+, synchronous fluent API) and set connect and read timeouts, because the default is to wait too long. `RestTemplate` still works but is in maintenance mode. For reactive code there is `WebClient`. Always handle failure: a slow downstream can exhaust your threads.

25. What changes with virtual threads?

Set `spring.threads.virtual.enabled=true` (Java 21+) and Boot runs request handling and some executors on virtual threads, so blocking calls (JDBC, `RestClient`) stop tying up scarce platform threads. It helps I/O-bound services; it does not speed up CPU-bound work, and you should still watch connection pool size.

6.1.4 Data access, JPA and transactions

26. JPA, Hibernate, Spring Data JPA: how do they relate?

JPA (now Jakarta Persistence) is a specification. Hibernate is the implementation Boot uses by default. Spring Data JPA sits on top: you declare an interface extending `JpaRepository<Book, Long>` and Spring generates the implementation, including derived queries such as `findByAuthorName`.

27. What are the entity states?

Transient (new, not known to the persistence context), managed/persistent (tracked by the `EntityManager`; changes are flushed automatically at commit, which is called dirty checking), detached (was managed, session closed) and removed. Understanding this explains why changing a managed entity inside a transaction needs no `save()` call.

28. Lazy vs eager fetching, and what is `LazyInitializationException`?

`LAZY` loads an association only when first touched; `EAGER` loads it immediately. Collections default to lazy and `@ManyToOne` defaults to eager, though lazy is usually what you want everywhere. The exception means you touched a lazy association after the session closed, typically while serialising an entity outside a transaction. Fix by fetching what you need in the query, or by mapping to a DTO inside the service.

29. What is the N+1 problem and how do you fix it?

Loading N parents with one query, then lazily loading each parent's children with N more queries. It looks fine with five rows and kills you with five thousand. Fixes: `join fetch` in a JPQL query, `@EntityGraph` on the repository method, or a DTO projection that selects exactly the columns needed. Turn on `spring.jpa.show-sql` (or SQL logging) in development to see it.

30. Which side owns a bidirectional relationship?

The side with the foreign key, which is the `@ManyToOne` side. The `@OneToMany(mappedBy = "author")` side is the inverse and Hibernate ignores changes made only there. Keep both sides in sync with helper methods like `author.addBook(book)`.

31. How does `@Transactional` propagation work, in short?

The default is `REQUIRED`: join the caller's transaction if one exists, otherwise start one. `REQUIRES_NEW` suspends the caller's transaction and runs in a separate one that commits independently. Use `readOnly = true` for query-only methods: it signals intent and lets Hibernate skip dirty checking. Rollback is automatic for unchecked exceptions only.

32. Why not `spring.jpa.hibernate.ddl-auto=update` in production?

Hibernate guesses schema changes, cannot rename or drop reliably, and gives no history or review. Use a migration tool (Flyway or Liquibase) with versioned SQL files and set `ddl-auto` to `validate` or `none`.

33. What is open-in-view?

`spring.jpa.open-in-view` (on by default) keeps the persistence session open until the response is rendered, which hides lazy-loading errors in controllers but holds a database connection longer and encourages queries from the web layer. Many teams turn it off and fetch explicitly in the service layer.

> **INTERVIEW ANGLE**
> "Should entities implement equals and hashCode?" Be careful. Using the generated database id breaks when it is null before saving, so entities in a `HashSet` change hash code after persist. Safe options: compare on a stable business key (an ISBN), or use an id-based `equals` with a constant `hashCode`. Never include lazy collections.

6.1.5 Security

34. What does Spring Security do by default?

Adding the starter secures every endpoint with form login and HTTP Basic, generates a random password for a user named `user` at startup (printed in the log), and enables CSRF protection and security headers. You then define a `SecurityFilterChain` bean to describe your rules.

35. Authentication vs authorisation?

Authentication answers "who are you?" (credentials checked against a `UserDetailsService` or a token validated). Authorisation answers "may you do this?" (roles and authorities checked by URL rules in the filter chain or by `@PreAuthorize("hasRole('ADMIN')")` on methods; method security needs `@EnableMethodSecurity`).

36. How are passwords stored?

Never in plain text. Hash with a slow, salted algorithm through a `PasswordEncoder`; `BCryptPasswordEncoder` is the common choice, and `PasswordEncoderFactories.createDelegatingPasswordEncoder()` stores a prefix like `{bcrypt}` so you can migrate algorithms later.

37. Sessions or JWT?

Session: the server stores login state and the browser sends a cookie. Simple, easy to revoke, needs sticky sessions or a shared store when scaled. JWT: the client sends a signed token on each request, so servers stay stateless, but revocation is hard and a leaked token works until it expires. Keep tokens short-lived. In Spring you configure a resource server (`oauth2ResourceServer(o -> o.jwt(...))`) rather than hand-parsing tokens.

38. When can you disable CSRF?

CSRF attacks rely on the browser automatically attaching credentials such as a session cookie. A stateless API that authenticates through an `Authorization` header, with no cookies, is not exposed in that way, so disabling CSRF there is reasonable. If you use cookie-based login, keep it on.

6.1.6 Testing

39. What is the difference between a unit test and a Spring test?

A unit test creates the class with `new` and fakes its dependencies with Mockito: no container, runs in milliseconds. A Spring test starts some or all of the application context, so it is slower but checks wiring, JSON mapping and SQL. Aim for many unit tests and fewer slice and full-context tests (the test pyramid).

40. What are test slices?

Annotations that load only part of the context. `@WebMvcTest` loads controllers, advice and filters, and you drive it with `MockMvc` while mocking the service. `@DataJpaTest` loads JPA with an embedded database and rolls back after each test. `@SpringBootTest` loads everything (with `webEnvironment = RANDOM_PORT` it starts a real server).

41. `@MockBean` or `@MockitoBean`?

`@MockitoBean` (Spring Framework 6.2) replaces `@MockBean`, which was deprecated in Boot 3.4 and removed in Boot 4. Use `@MockitoBean` on new code; it works in Boot 3.5 and 4.x. Import paths changed between lines, so check the import when migrating.

42. How do you test against a real database?

Testcontainers starts a throwaway PostgreSQL in Docker, so tests use the same engine as production instead of H2, whose SQL dialect can differ. Boot supports `@ServiceConnection` to wire the container's URL automatically.

6.1.7 Microservices basics

43. Monolith or microservices?

Start with a well-structured monolith. Microservices buy independent deployment and scaling at the price of network failures, distributed data, versioning and operations work. Split when team size or scaling needs force you to, along business boundaries.

44. How do services survive each other's failures?

Timeouts on every call, retries with backoff for idempotent calls only, circuit breakers (Resilience4j) to stop hammering a failing service, and fallbacks. Spring Framework 7 / Boot 4 adds core retry support; before that, Spring Retry or Resilience4j covered it.

45. What does Actuator give you in production?

HTTP endpoints for health (`/actuator/health`, with liveness and readiness probes for Kubernetes), metrics through Micrometer, environment, loggers and more. Expose only what you need (`management.endpoints.web.exposure.include=health,info`) and protect the rest; `env` and `heapdump` can leak secrets.

6.1.8 How to answer well

Give the one-sentence definition first, then the reason it exists, then a trade-off or pitfall. "Constructor injection, because dependencies become explicit and final and the class is testable without Spring; the only cost is more constructor parameters, which is itself a design smell worth noticing." That structure works for almost every question here.

If you do not know, say what you would check ("I would run with `--debug` and read the conditions report") rather than guessing. Interviewers value a debugging method more than a memorised sentence.

Check yourself

1. What is wrong with this service, and why does it fail silently? `public void a() { b(); } @Transactional public void b() { save(); }`
2. A class has two constructors and no `@Autowired`. What does Spring do?
3. Which scope would you pick for a service that holds a counter of requests per user, and why is a plain singleton field wrong?
4. Why does `ddl-auto=update` fail an architecture review?
5. A controller returns an entity with a lazy collection and throws `LazyInitializationException`. Name two fixes.
6. When may CSRF be disabled?

Answers

1. `a()` calls `b()` on `this`, bypassing the proxy, so no transaction starts. Move `b()` into another bean or annotate `a()`.
2. With several constructors and none marked `@Autowired`, Spring uses the no-argument constructor, and fails if there is none.
3. Do not keep per-user state in the singleton; store it in a database or cache, or a thread-safe structure such as `ConcurrentHashMap`. A mutable shared field is a race condition.
4. It changes the schema by guesswork, with no review, history or rollback. Use Flyway or Liquibase.
5. Map to a DTO inside the transactional service, or fetch the collection with `join fetch` or `@EntityGraph`.
6. For stateless APIs that authenticate with a header token and use no cookies.

KEY POINTS

- Answer in three beats: definition, reason, trade-off or pitfall.
- The proxy model explains `@Transactional`, `@Async` and `@Cacheable` surprises, including self-invocation.
- Constructor injection, DTOs, Flyway and stateless tokens are the "right default" answers.
- Auto-configuration is conditional and backs off when you define your own bean; `--debug` shows why.
- Name a debugging step when you are unsure; it shows method.

6.2 Cheat Sheets

These tables are for revision and for the night before an interview. They list the names you must recognise; the explanations are in the chapters on the container, MVC, data access and testing.

6.2.1 Annotations

Annotation	Meaning
`@SpringBootApplication`	Configuration + auto-configuration + component scan on the main class
`@Component, @Service, @Repository`	Register a bean found by scanning; the last also translates persistence exceptions
`@Configuration, @Bean`	A class of bean definitions; a method whose return value becomes a bean
`@Autowired`	Inject a dependency; optional on a single constructor
`@Qualifier, @Primary`	Choose among several beans of one type
`@Scope`	singleton (default), prototype, request, session
`@Profile`	Register a bean only in a named profile
`@Value, @ConfigurationProperties`	Bind one property; bind a group of properties to a typed object
`@PostConstruct, @PreDestroy`	Lifecycle callbacks after wiring and before shutdown
`@RestController`	Controller whose return values become the response body
`@GetMapping, @PostMapping, @PutMapping, @PatchMapping, @DeleteMapping`	Map an HTTP verb and path
`@PathVariable, @RequestParam, @RequestBody, @RequestHeader`	Bind parts of the request to parameters
`@Valid, @Validated`	Trigger Bean Validation
`@RestControllerAdvice, @ExceptionHandler`	Central error handling
`@ResponseStatus`	Fix the status code for a method or exception class
`@Transactional`	Run a method in a transaction through a proxy
`@PreAuthorize`	Method-level authorisation (needs method security enabled)

Core and web annotations.

6.2.2 Properties

Key	Purpose	Example
`server.port`	HTTP port	`8080`
`spring.application.name`	Name used in logs and metrics	`library-api`
`spring.profiles.active`	Active profiles	`dev`
`spring.datasource.url`	JDBC URL	`jdbc:postgresql://db:5432/lib`
`spring.datasource.username`	Database user	`lib`
`spring.jpa.hibernate.ddl-auto`	Schema handling	`validate`
`spring.jpa.show-sql`	Print SQL (development)	`true`
`spring.jpa.open-in-view`	Session spans the web request	`false`
`spring.flyway.enabled`	Run migrations at startup	`true`
`spring.threads.virtual.enabled`	Virtual threads (Java 21+)	`true`
`spring.mvc.problemdetails.enabled`	RFC 9457 error bodies	`true`
`server.shutdown`	Graceful shutdown	`graceful`
`management.endpoints.web.exposure.include`	Exposed actuator endpoints	`health,info`
`logging.level.root`	Default log level	`INFO`
`logging.level.com.example`	Per-package level	`DEBUG`

Common application.properties keys.

Environment variables use upper-case and underscores: `SPRING_DATASOURCE_URL` sets `spring.datasource.url`. Command-line arguments (`--server.port=9090`) beat environment variables, which beat files in the jar.

6.2.3 HTTP status codes

Code	Name	Typical use
`200`	OK	Successful GET, PUT, PATCH with a body
`201`	Created	POST that made a resource; add a `Location` header
`204`	No Content	Successful DELETE or update with no body
`400`	Bad Request	Malformed JSON, missing parameter, failed validation
`401`	Unauthorized	Not authenticated (missing or bad credentials)
`403`	Forbidden	Authenticated but not permitted
`404`	Not Found	No such resource
`405`	Method Not Allowed	Verb not supported on this path
`409`	Conflict	Duplicate key, version clash, state conflict
`415`	Unsupported Media Type	Wrong `Content-Type`
`422`	Unprocessable Content	Well-formed but semantically invalid
`429`	Too Many Requests	Rate limit exceeded
`500`	Internal Server Error	Unexpected bug; hide details
`503`	Service Unavailable	Overloaded or dependency down

Status codes an API uses.

6.2.4 JPA annotations

Annotation	Meaning
`@Entity`	Class mapped to a table
`@Table(name = "...")`	Choose table name, indexes, constraints
`@Id`	Primary key field
`@GeneratedValue`	Key generation: IDENTITY, SEQUENCE, UUID
`@Column`	Column name, length, nullable, unique
`@Enumerated(EnumType.STRING)`	Store enum names, not ordinals
`@ManyToOne, @OneToMany, @ManyToMany, @OneToOne`	Relationships; the foreign-key side owns it
`@JoinColumn`	Foreign key column on the owning side
`mappedBy`	Marks the inverse (non-owning) side
`cascade, orphanRemoval`	Propagate operations to children
`fetch = LAZY`	Load association on first use
`@Version`	Optimistic locking column
`@Query`	Custom JPQL or native query on a repository method
`@EntityGraph`	Fetch associations in one query to avoid N+1

JPA and Spring Data annotations (jakarta.persistence).

6.2.5 Test annotations

Annotation	Loads	Use for
`@ExtendWith(MockitoExtension.class)`	No Spring	Plain unit tests with Mockito
`@WebMvcTest`	Controllers, advice, filters	HTTP layer with MockMvc
`@DataJpaTest`	JPA, embedded database	Repository queries, mappings
`@SpringBootTest`	Full context	End-to-end through the app
`@MockitoBean`	Replaces a bean with a mock	Isolate a layer in a slice test
`@ActiveProfiles("test")`	Selects profile	Test-specific configuration
`@Testcontainers, @ServiceConnection`	Real database in Docker	Tests close to production
`@WithMockUser`	Fake authenticated user	Security rules in MockMvc tests

Test annotations and what they load.

6.2.6 Starters

Artifact	Brings
`spring-boot-starter-web`	Spring MVC, Jackson, embedded Tomcat (renamed spring-boot-starter-webmvc in 4.x)
`spring-boot-starter-validation`	Hibernate Validator for Bean Validation
`spring-boot-starter-data-jpa`	Spring Data JPA, Hibernate, connection pool
`spring-boot-starter-security`	Spring Security
`spring-boot-starter-actuator`	Health, metrics, operational endpoints
`spring-boot-starter-test`	JUnit 5, AssertJ, Mockito, Spring Test
`spring-boot-starter-oauth2-resource-server`	JWT validation for APIs
`spring-boot-starter-cache`	Caching abstraction
`spring-boot-starter-data-redis`	Redis client and templates

Common starters (group org.springframework.boot).

NOTE
In Boot 4 the starters were reorganised into smaller, technology-specific modules. The web starter is also available as `spring-boot-starter-webmvc`, and test starters became per-technology. Generate the project on start.spring.io and check the generated `pom.xml` rather than relying on memory.

Check yourself

1. Which status code means "logged in but not allowed"?
2. What does `SPRING_JPA_SHOW_SQL=true` set?
3. Which test annotation would you choose to test a custom `@Query`?
4. Which side of a bidirectional relationship carries `mappedBy`?

Answers

1. 403 Forbidden (401 means not authenticated).
2. The property `spring.jpa.show-sql`, through relaxed binding of environment variables.
3. `@DataJpaTest`.
4. The inverse side; the owning side has the foreign key (`@JoinColumn`).

KEY POINTS
- Memorise names, not syntax: you can always look up parameters.
- Properties map to environment variables by upper-casing and replacing dots and dashes with underscores.
- 401 is about identity; 403 is about permission.
- Pick the smallest test slice that exercises the behaviour you care about.

6.3 A Ten-Day Study Plan

One or two hours a day is enough if you type the code yourself. The plan assumes you can write core Java (classes, collections, streams) and have JDK 21 or newer plus Maven installed. Each day ends with a small deliverable; if you cannot produce it, repeat the day rather than moving on.

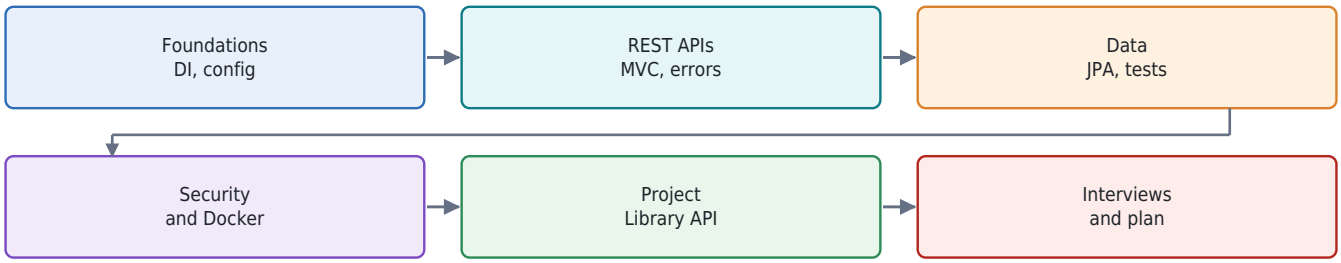


Figure 6.1. A route through the guide: foundations first, then the web layer, data, security and shipping, with the project tying them together.

Day	Study	Deliverable
1	Spring Boot Foundations: what Spring and Boot are, pom, layout, first app	Generate a project, run it, build and start the jar with `java -jar`
2	The container and dependency injection; bean lifecycle and proxies	Write a service and a repository wired by constructor; add `@PostConstruct` logging
3	Configuration, profiles, auto-configuration	Bind `@ConfigurationProperties` to a record; run once with `--debug` and read the report
4	Spring MVC and REST controllers	CRUD endpoints for books in memory, tested with curl
5	Validation, exception handling, REST design	`@Valid` DTOs and a `@RestControllerAdvice` returning `ProblemDetail`
6	Spring Data JPA: entities, repositories, queries	Swap the in-memory store for H2, then PostgreSQL

Day	Study	Deliverable
7	Relationships, transactions, N+1	Add authors and loans; reproduce and fix an N+1 query
8	Testing: unit, `@WebMvcTest`, `@DataJpaTest`	At least one test of each kind on your service
9	Spring Security, Actuator, Docker	Protect writes with a role; add a Dockerfile and a compose file with PostgreSQL
10	Finish the Library API project; interview Q&A aloud	Deployed locally with compose; answer 15 questions without notes

Day-by-day plan.

6.3.1 How to use each day

Read the chapter once quickly, then close it and build. Look things up only when you are stuck; that struggle is what makes the pieces stick. Keep a notes file with "things that surprised me": self-invocation, lazy loading, a missing slice bean. These become your best interview stories.

On days 4 to 8 keep one project growing instead of starting a new one each time; it is the same project you will polish on day 10. Commit after every working step, with messages that say why.

6.3.2 After day ten

Take the mixed quiz in the next chapter with the book closed. Every missed question names a chapter to reread. Then write a short README for your project (how to run it, the endpoint table, design decisions) and push it to GitHub. A runnable repository with tests and a clear README is worth more in an interview than a list of certificates.

Check yourself

1. Why does the plan put testing before security?
2. What is the deliverable on day 3 that proves you understand auto-configuration?
3. Why keep one project across days 4 to 8?

Answers

1. Tests let you change code safely; with them in place you can add security rules and see what you broke.
2. A `--debug` run where you can point to a condition that matched and one that did not, and explain why.
3. Each day then builds on the last, and you meet real integration problems instead of toy ones.

KEY POINTS

- Build every day; reading alone does not teach framework behaviour.
- Repeat a day instead of skipping it when the deliverable fails.
- Finish with a runnable, tested, documented repository.

6.4 Mixed Quiz and Next Steps

Thirty questions, grouped by topic. Answer on paper first, then check. The reply to each question is a sentence or two; if yours differs in substance, reread the named area.

6.4.1 Quiz: container, configuration, Boot

1. Which annotation combination does `@SpringBootApplication` stand for?
2. Why should the main class live in the root package?
3. What is the default scope of a bean?
4. A singleton service has a field `private int counter` incremented per request. What is the problem?
5. What does `@ConditionalOnMissingBean` let you do?
6. Which file lists a starter's auto-configuration classes?
7. Which wins: an environment variable or a value in `application.properties`?
8. How do you see why an auto-configuration did not apply?

Answers: container, configuration, Boot

1. `@SpringBootConfiguration`, `@EnableAutoConfiguration` and `@ComponentScan`.
2. Component scanning starts at that package and covers only it and its sub-packages.
3. Singleton.
4. Concurrent threads share it, causing a race condition; keep singletons stateless.
5. Define your own bean and have the default one back off.
6. `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`.
7. The environment variable.
8. Start with `--debug` (or `debug=true`) and read the conditions report.

6.4.2 Quiz: web and REST

1. Which component receives every request first?
2. What does `@RestController` add over `@Controller`?
3. Which status code does a successful POST creating a resource return, and which header goes with it?
4. Which exception does a failed `@Valid` on a request body throw?
5. Why should an API not return JPA entities directly?
6. Which verbs are idempotent: GET, POST, PUT, DELETE?
7. What is `ProblemDetail`?
8. Which class do you use for outgoing HTTP calls in modern Spring, and what must you always set?

Answers: web and REST

1. The `DispatcherServlet`.
2. `@ResponseBody` on every method, so results are written as the body.
3. 201 Created with a `Location` header.
4. `MethodArgumentNotValidException`.
5. It leaks schema and sensitive fields, couples API to database, and risks lazy-loading errors.
6. GET, PUT and DELETE.
7. The Spring representation of the RFC 9457 error body, served as `application/problem+json`.
8. `RestClient`; set connect and read timeouts.

6.4.3 Quiz: data and transactions

1. Who is the owning side of a one-to-many bidirectional relationship?
2. What is N+1, and name one fix?
3. When does `LazyInitializationException` occur?
4. Which exceptions roll back a `@Transactional` method by default?
5. Why does `this.save()` in the same class ignore `@Transactional`?
6. Why use Flyway instead of `ddl-auto=update`?
7. What does `readOnly = true` signal?
8. What does a Spring Data method named `findByTitleContainingIgnoreCase` do?

Answers: data and transactions

1. The many side (`@ManyToOne`), which holds the foreign key.
2. One query for parents plus one per parent for children; fix with `join fetch`, `@EntityGraph` or a DTO projection.
3. When a lazy association is accessed after the persistence session has closed.
4. `RuntimeException` and `Error`; checked exceptions do not unless `rollbackFor` says so.
5. The call bypasses the proxy that implements the transaction.
6. Versioned, reviewable migrations replace guesswork and keep history.
7. The method only reads, so Hibernate can skip dirty checking and the driver may optimise.
8. It generates a query for a title containing the argument, ignoring case.

6.4.4 Quiz: security, testing, operations

1. Which password encoder is the common default choice?
2. What is the difference between 401 and 403?
3. Which annotation tests only the web layer?
4. What replaced `@MockBean`?
5. Which actuator endpoints should be exposed publicly?
6. Why run tests on PostgreSQL with Testcontainers instead of H2?

Answers: security, testing, operations

1. `BCryptPasswordEncoder`, usually through the delegating encoder.
2. 401 is unauthenticated; 403 is authenticated but not allowed.
3. `@WebMvcTest`.
4. `@MockitoBean`.
5. Normally only `health` (and perhaps `info`); protect the rest.
6. The real engine behaves like production, so dialect and constraint differences surface in tests.

6.4.5 Scoring

27 to 30: ready to interview. 20 to 26: redo the weak group's chapters, then build the missing piece in the project. Below 20: repeat days 1 to 5 of the plan before returning.

6.4.6 Next steps

This guide covers the foundation. These are the usual next topics; each is a pointer, not a promise of detail here.

- **Microservices with Spring Cloud**: service discovery, configuration servers, API gateways, resilience patterns. Learn the monolith first.
- **Messaging with Kafka**: Spring for Apache Kafka for event-driven designs; learn partitions, consumer groups and idempotent consumers.
- **Caching with Redis**: `@Cacheable` and Spring Data Redis for read-heavy endpoints; learn eviction and staleness.
- **Docker and Kubernetes**: build images, health probes, config maps and secrets, rolling deploys.
- **System design**: capacity estimates, databases and indexes, queues, load balancing. Practise designing a library service at ten times the scale.
- **Observability**: Micrometer metrics, tracing with OpenTelemetry, structured logs.

Whatever you choose next, keep the habit that served you here: build a small version, test it, then explain it out loud.

Check yourself

1. Which quiz group was weakest for you, and which chapter will you reread?
2. What is one small project that would combine Kafka with this API?
3. Why learn the monolith before microservices?

Answers

1. Your own: write down the group and chapter name now.
2. For example, publish a "book borrowed" event from the loan service and consume it in a notification service.
3. Microservices multiply the problems (network, data, deployment) you must already handle in one process.

KEY POINTS

- Use the quiz to find gaps, then fix them by building.
- Learn one adjacent technology at a time, on top of the same project.
- Be able to explain every choice in your repository out loud.